

실전 데이터 모델링

Author: Cheongja Woo

email: cheongja.woo@gmail.com



교육 과정 구성

1일차



- 데이터 모델링 개요
- 식별자 선정 및 관계 선정 모델링 실습
- 이력관리 및 인터페이스 모델링 실습
- 논리 모델링의 일반화 및 구체화



2일차

- 엔티티 통합 및 분리 방안 실습
- 정규화 및 반정규화를 통한 모델 통합 방안
- 데이터 모델 구조에 따른 성능 향상을 위한 모델링 실습
- 대용량 데이터 특성을 고려한 모델링 실습
- 상세 물리 모델링의 이슈 관리





데이터 모델링 개요

데이터 모델링 개념과 기법 이해

데이터 모델링의 필요성과 기법, ERD 중심 정리

데이터 모델링의 중요성

정확한 데이터 분석

데이터 모델링은 데이터의 구조를 명확히 정의하여 데이터 분석의 정확성을 높입니다.

데이터 일관성 유지

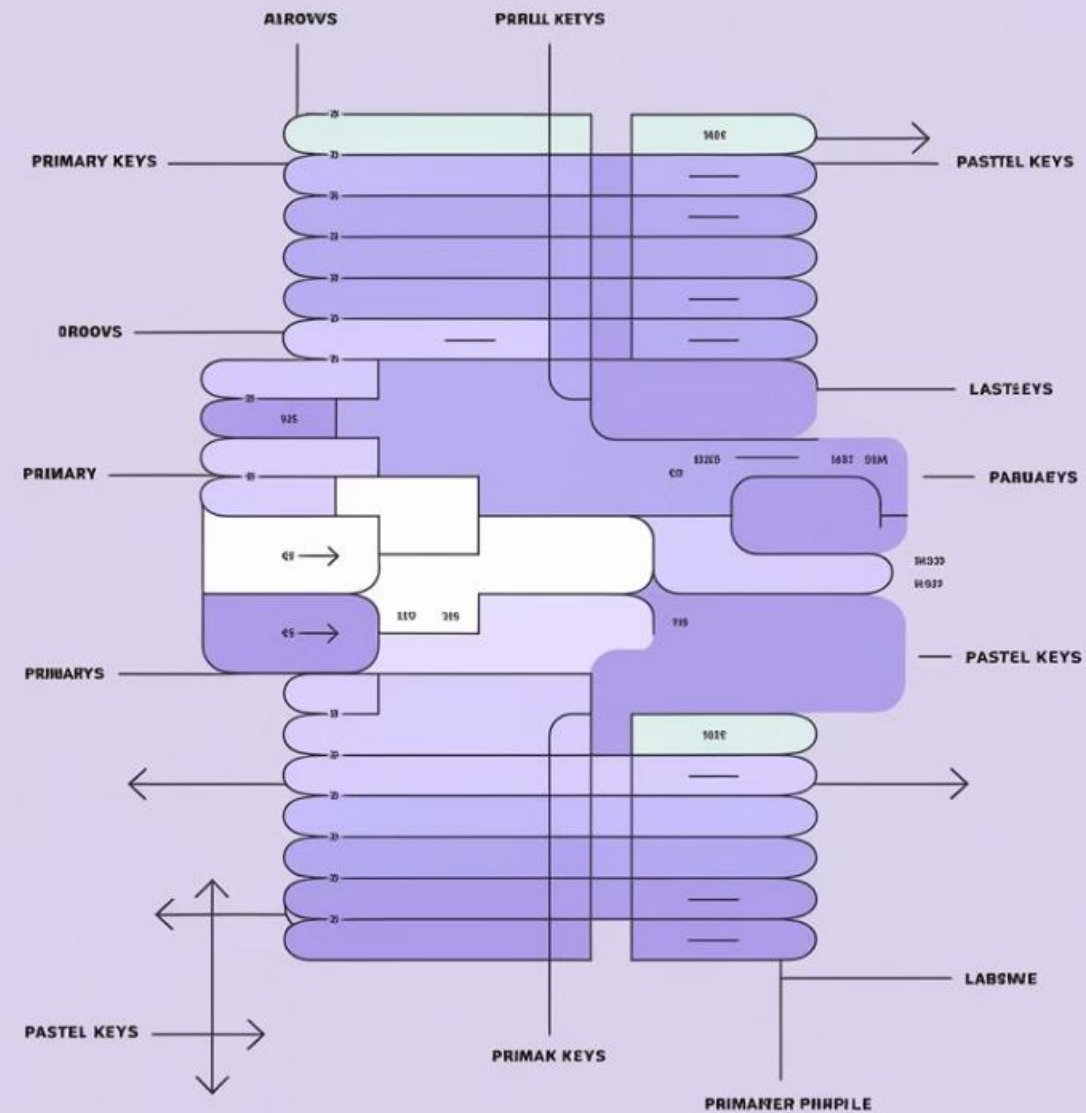
데이터 모델링을 통해 데이터의 일관성을 유지하여 데이터 무결성을 보장합니다.

효율적인 데이터 관리

데이터 모델링은 데이터베이스 설계를 최적화하여 데이터 관리 효율성을 향상시킵니다.

데이터 모델링의 개념

데이터 모델링은 데이터의 구조, 관계, 제약 조건을 표현하는 과정입니다. 데이터베이스 설계, 데이터 분석, 데이터 품질 관리 등 다양한 분야에서 중요한 역할을 합니다.





데이터 모델링 기법

1 ERD (Entity-Relationship Diagram)

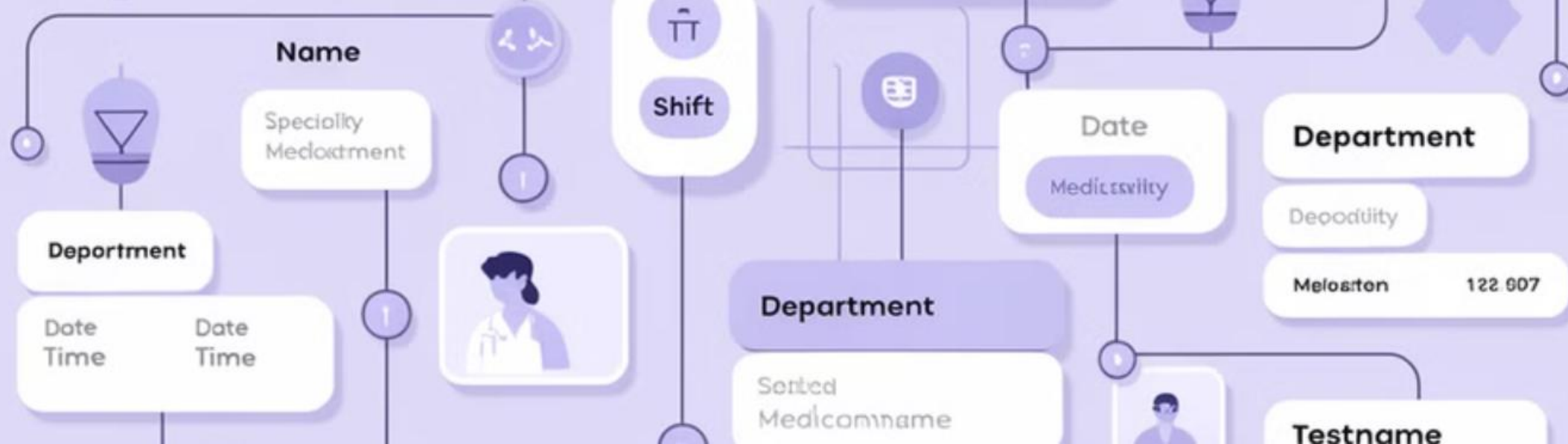
데이터베이스 설계에 가장 널리 사용되는 기법입니다. 엔티티와 관계를 사용하여 데이터 구조를 시각적으로 표현합니다.

2 UML (Unified Modeling Language)

소프트웨어 개발에 주로 사용되는 기법입니다. 객체 지향 프로그래밍 개념을 사용하여 데이터 모델을 표현합니다.

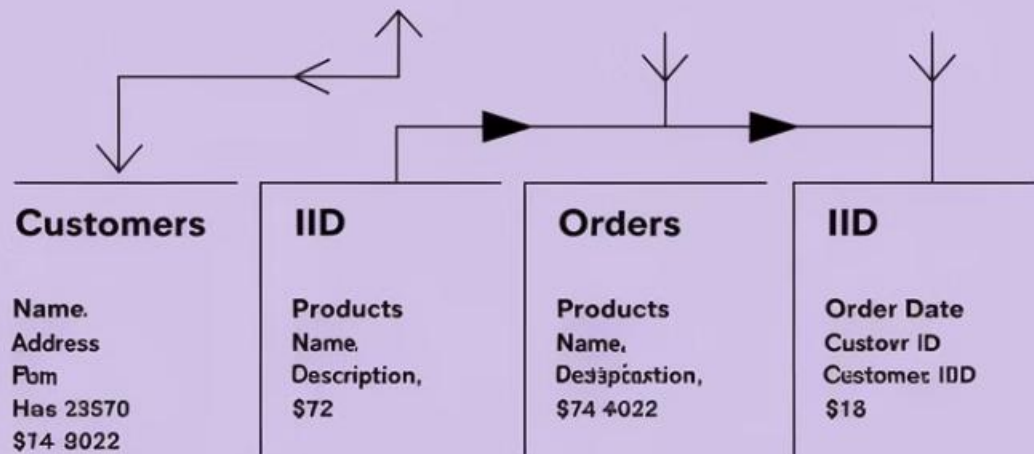
3 데이터 흐름 다이어그램

데이터의 흐름을 시각적으로 표현하여 데이터 처리 과정을 분석합니다.



ERD (Entity-Relationship Diagram)

ERD는 데이터베이스 설계에 필수적인 기법입니다. 엔티티, 속성, 관계를 사용하여 데이터 구조를 시각적으로 표현합니다.



ERD의 구성 요소

엔티티 (Entity)

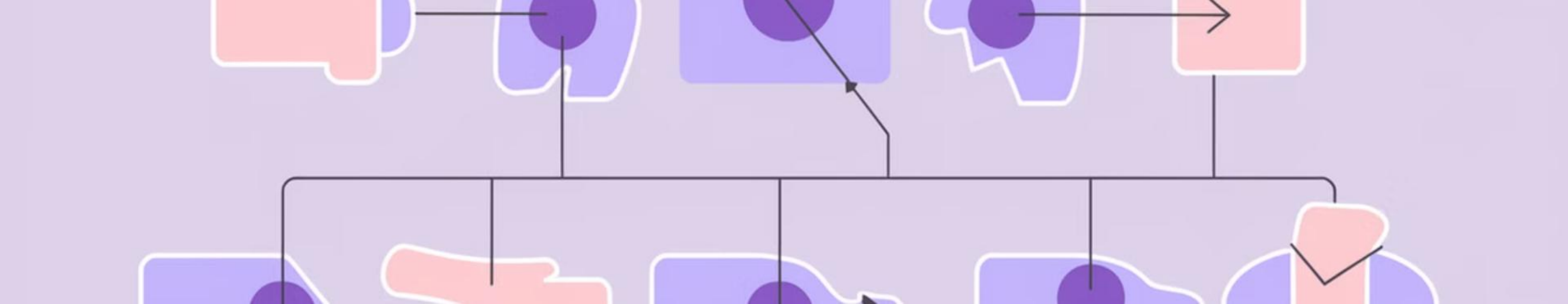
데이터베이스에서 관리하는 대상입니다. 예를 들어 고객, 제품, 주문 등이 있습니다.

속성 (Attribute)

엔티티에 대한 정보를 나타냅니다. 예를 들어 고객의 이름, 주소, 전화번호 등이 있습니다.

관계 (Relationship)

엔티티 간의 연결을 나타냅니다. 예를 들어 고객과 주문 간의 1:N 관계 등이 있습니다.



ERD 작성 단계



1. 엔티티 식별

데이터베이스에서 관리할 대상을 파악합니다.

2. 속성 정의

각 엔티티에 대한 정보를 정의합니다.

3. 관계 설정

엔티티 간의 연결을 설정합니다.

4. ERD 작성

ERD 도구를 사용하여 ERD를 작성합니다.



ERD 기법의 장점



데이터 구조를 명확하게 표현합니다.



개발자와 사용자 간의 소통을 원활하게 합니다.



데이터베이스 설계 및 관리를 용이하게 합니다.



데이터 모델링의 활용

데이터베이스 설계

데이터베이스의 구조를 정의하고 최적화합니다.

1

2

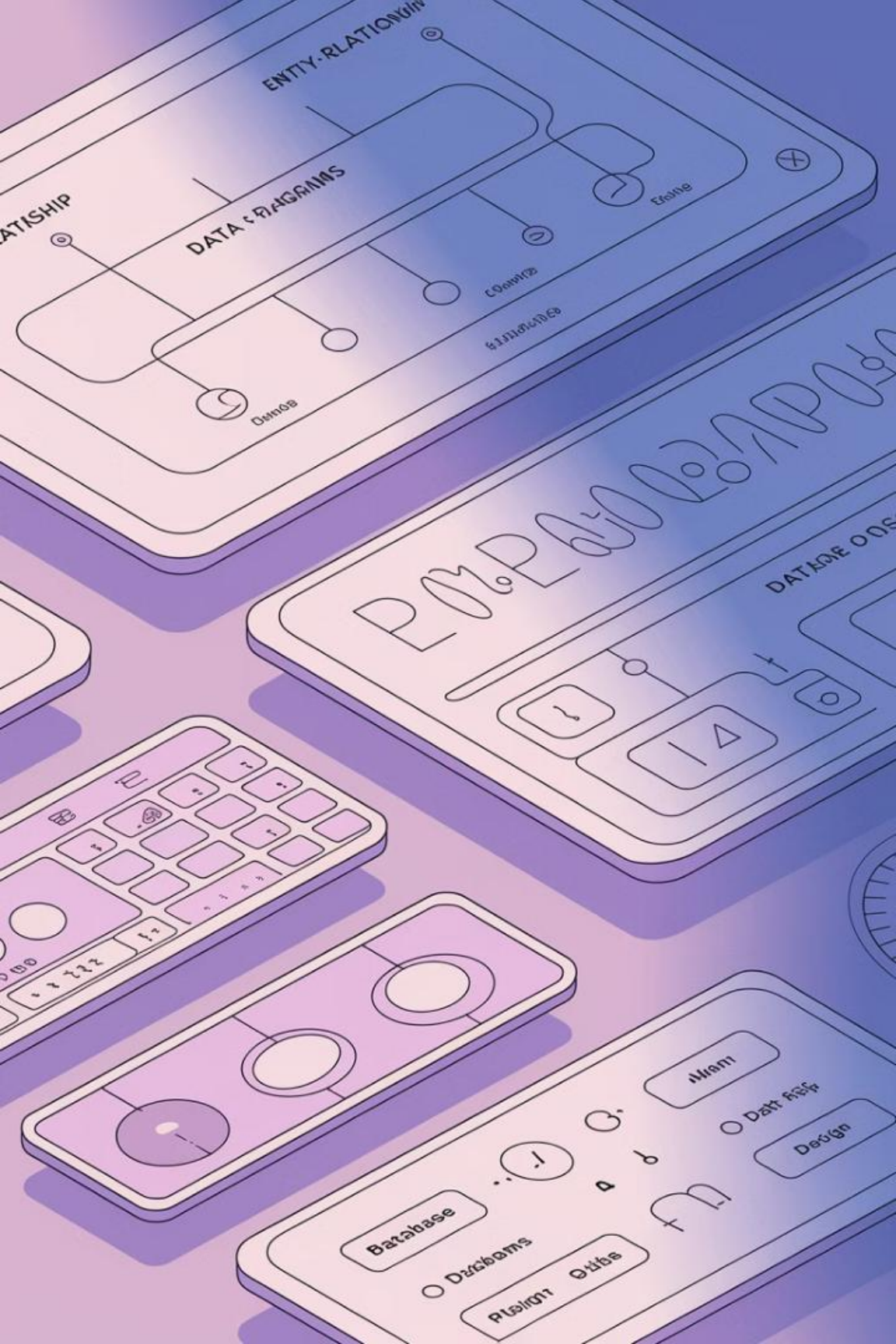
데이터 분석

데이터의 구조를 이해하고 분석을 수행합니다.

3

데이터 품질 관리

데이터의 일관성과 정확성을 보장합니다.

The background features a collage of stylized, overlapping illustrations of database modeling tools and diagrams. These include Entity-Relationship (ER) diagrams with entities like 'Customer' and 'Product', data flow diagrams, and various database design templates. The color palette is a mix of soft pinks, purples, and blues, creating a modern and technical aesthetic.

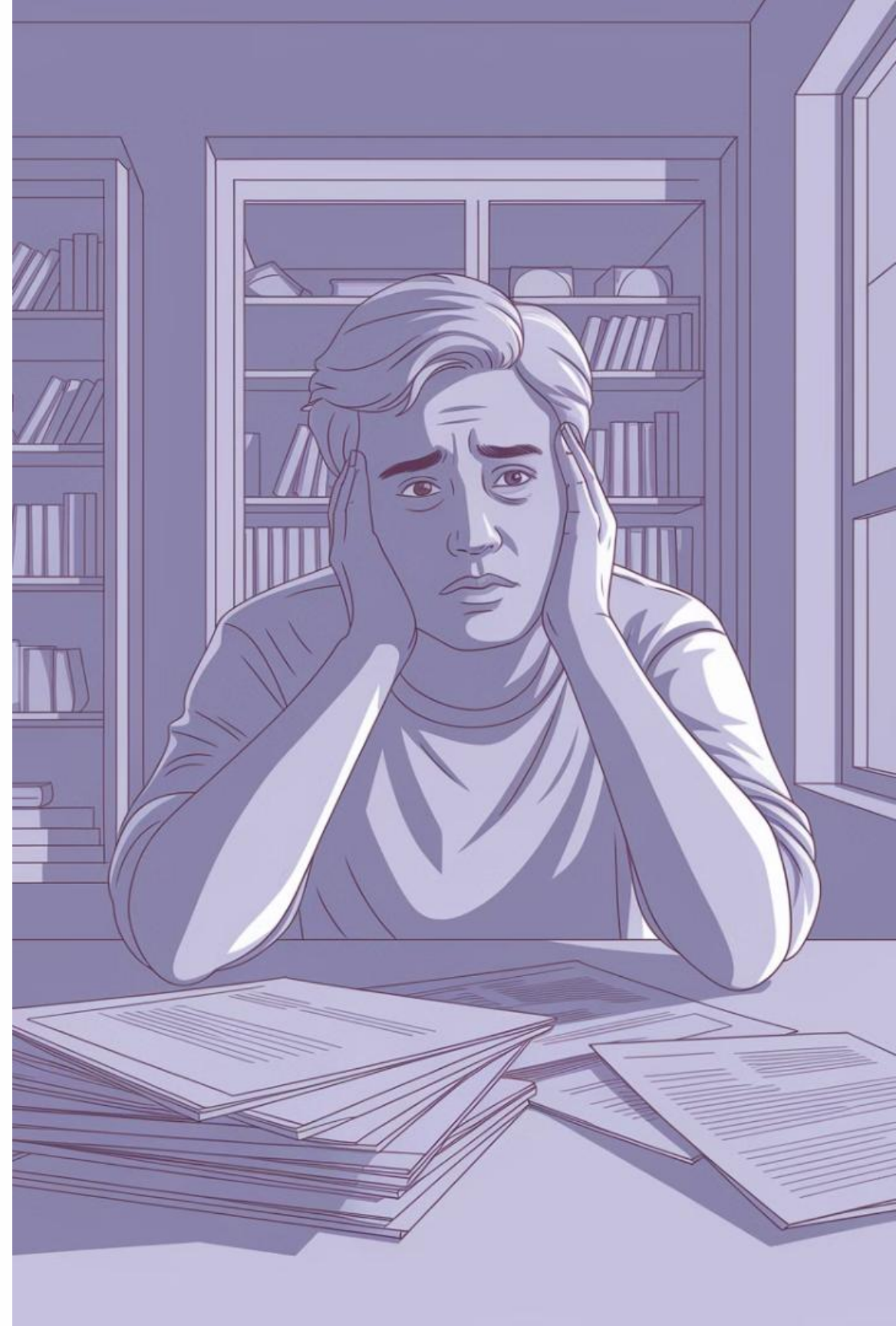
데이터 모델링 도구

ERD 도구는 다양한 종류가 있습니다. 무료 및 유료 도구 모두 사용 가능합니다.

데이터 모델링의 한계

데이터 모델링은 데이터를 정확하게 표현하고 관리하는 데 유용하지만,
모든 데이터를 완벽하게 포착할 수는 없습니다.

데이터의 복잡성과 변화에 따라 모델링이 어려워질 수 있습니다.





데이터 모델링의 미래

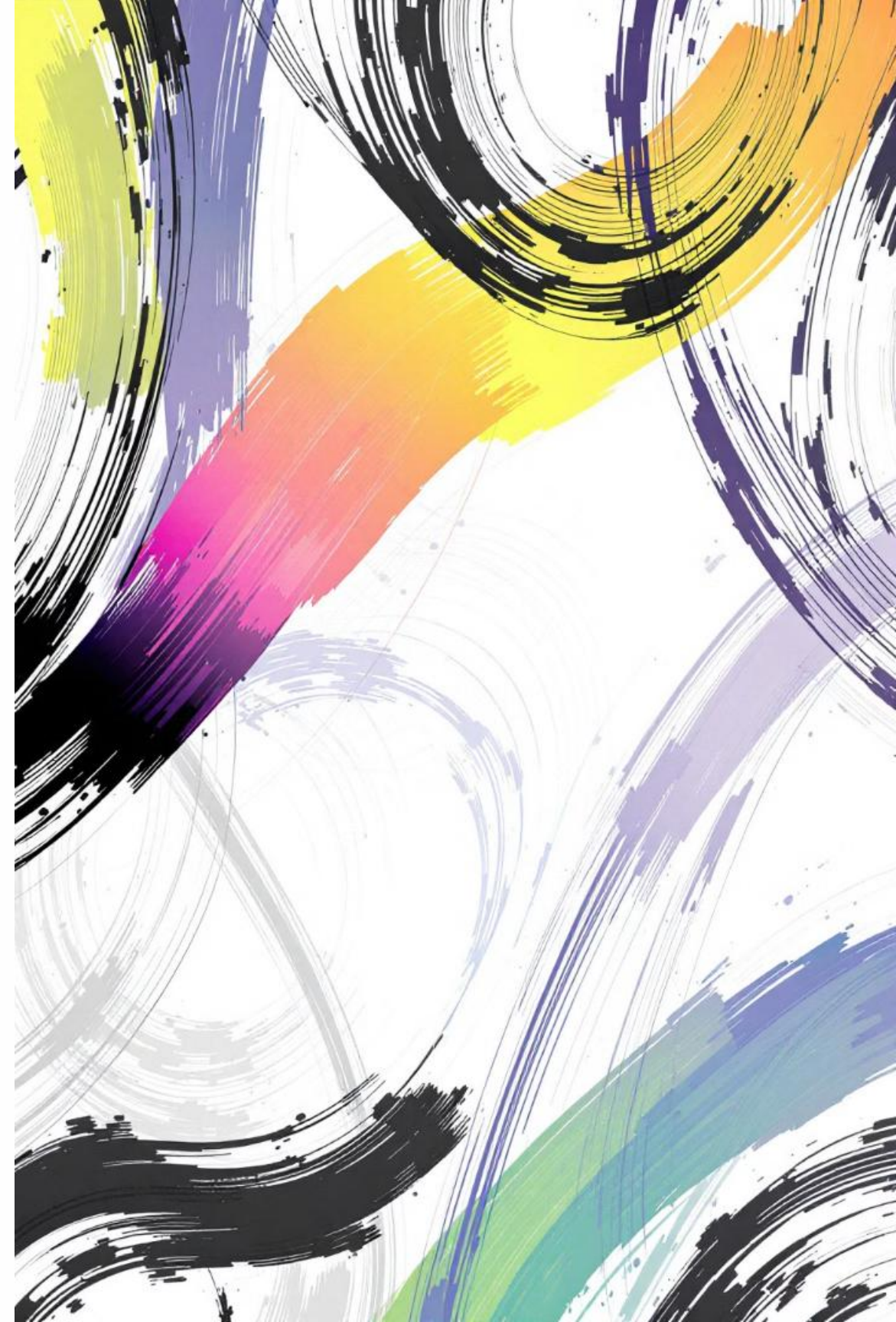
데이터 모델링은 AI, 머신러닝, 빅데이터 기술과 결합하여 더욱 발전할 것입니다.

이를 통해 데이터 분석의 자동화, 데이터 품질 향상, 데이터 기반 의사 결정 등 다양한 분야에서 활용될 것입니다.

데이터베이스 모델링

데이터베이스 설계는 데이터를 효율적으로 관리하고 활용하기 위한 핵심 과정입니다.

데이터베이스 설계의 기초를 배우고, 데이터베이스 설계 생명주기의 각 단계를 살펴보겠습니다.

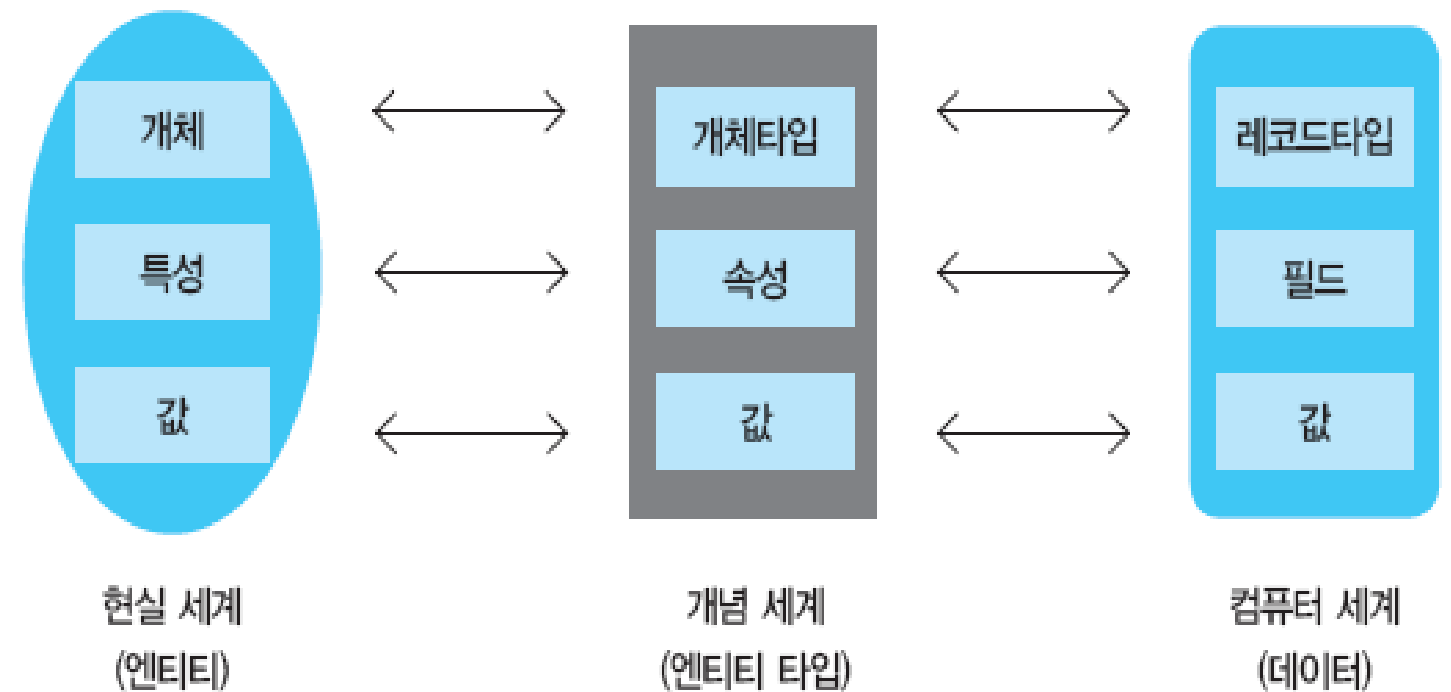


데이터베이스 세계

데이터베이스 모델링

현실 세계의 데이터를 컴퓨터가 이해할 수 있는 형태로 변환하는 과정입니다.

현실 세계의 개체를 데이터로 표현하고,
이들을 서로 연결하는 관계를 정의합니다.



데이터베이스 생명주기

- 1

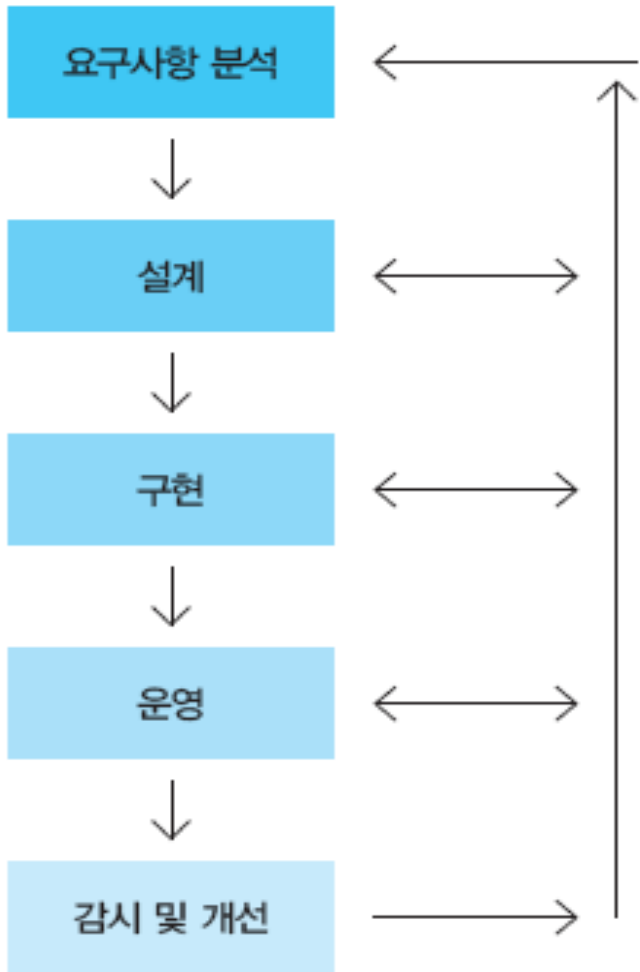
요구사항 분석: 사용자의 요구사항을 수집하고 분석하여 데이터베이스 설계의 목표를 명확히 정의합니다.
- 2

설계 단계: 요구사항 분석 결과를 바탕으로 데이터베이스 구조를 설계합니다. 개념적 설계, 논리적 설계, 물리적 설계의 3단계로 나뉩니다.
- 3

구현 단계: 설계된 데이터베이스를 실제로 구축하는 단계입니다. 데이터베이스 관리 시스템(DBMS)을 사용하여 데이터베이스를 생성하고 테이블, 뷰, 인덱스 등의 데이터베이스 객체를 생성합니다.
- 4

운영 단계: 구현된 데이터베이스를 테스트하고 문제점을 수정합니다. 테스트가 완료되면 실제 사용자에게 배포하고 운영합니다.
- 5

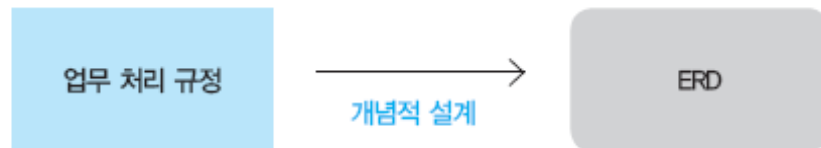
감시 및 개선 단계: 데이터베이스를 지속적으로 관리하고 업데이트합니다. 데이터베이스 성능을 최적화하고 새로운 요구사항을 반영합니다.



설계 단계

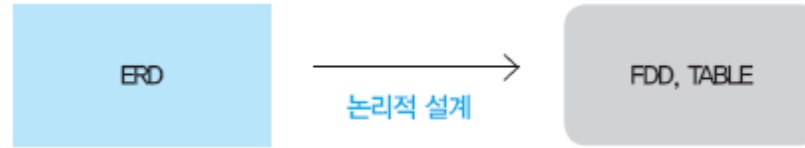
개념적 설계

사용자의 요구사항을 바탕으로 데이터 모델을 추상적으로 표현하는 단계입니다. E-R 다이어그램을 사용하여 엔티티와 관계를 표현합니다.



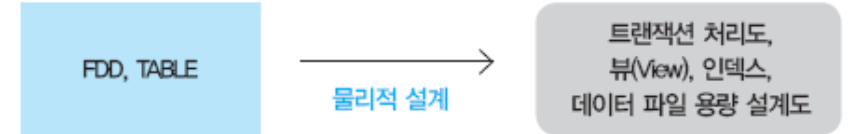
논리적 설계

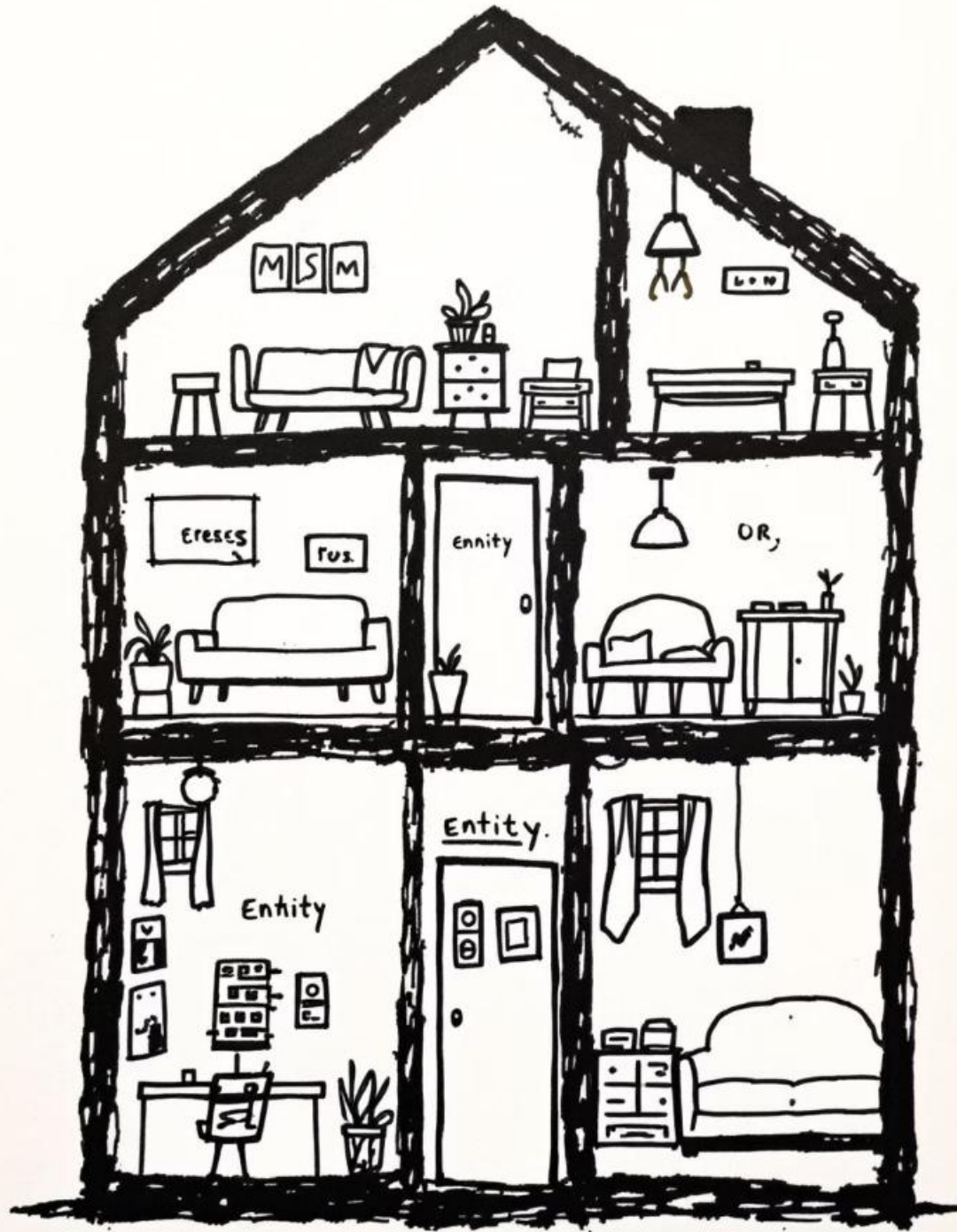
개념적 설계를 바탕으로 데이터베이스의 논리적인 구조를 구체화하는 단계입니다. 관계형 데이터 모델을 사용하여 테이블, 컬럼, 데이터 타입, 제약 조건 등을 정의합니다.



물리적 설계

논리적 설계를 바탕으로 데이터베이스를 실제로 구현하기 위한 물리적인 구조를 설계하는 단계입니다. DBMS 특징에 맞게 테이블 스페이스, 인덱스, 데이터베이스 사용자 등을 설정합니다.





개념적 설계

엔티티(Entity)

데이터베이스에서 관리하는 대상을 나타냅니다. 예를 들어, 학생, 교수, 과목, 강의실 등이 엔티티가 될 수 있습니다.

관계(Relationship)

엔티티 간의 연관성을 나타냅니다. 예를 들어, 학생은 여러 과목을 수강할 수 있으며, 교수는 여러 과목을 강의할 수 있습니다.

속성(Attribute)

엔티티의 특징을 나타내는 정보를 나타냅니다. 예를 들어, 학생의 속성은 학번, 이름, 전화번호 등이 될 수 있습니다.

논리적 설계

테이블(Table)

데이터를 저장하는 기본 단위입니다. 각 테이블은 행(Row)과 열(Column)로 구성됩니다.

컬럼(Column)

테이블의 열을 나타내며, 데이터의 종류를 나타냅니다. 예를 들어, 학생 테이블의 컬럼은 학번, 이름, 전화번호 등이 될 수 있습니다.

데이터 타입(Data Type)

컬럼에 저장되는 데이터의 종류를 지정합니다. 예를 들어, 학번은 숫자, 이름은 문자열, 전화번호는 숫자 또는 문자열 등이 될 수 있습니다.

제약 조건(Constraint)

데이터의 무결성을 유지하기 위해 설정하는 규칙입니다. 예를 들어, 학번은 중복될 수 없으며, 학점은 0에서 4.5 사이의 값을 가져야 합니다.

물리적 설계

테이블 스페이스(Table Space)

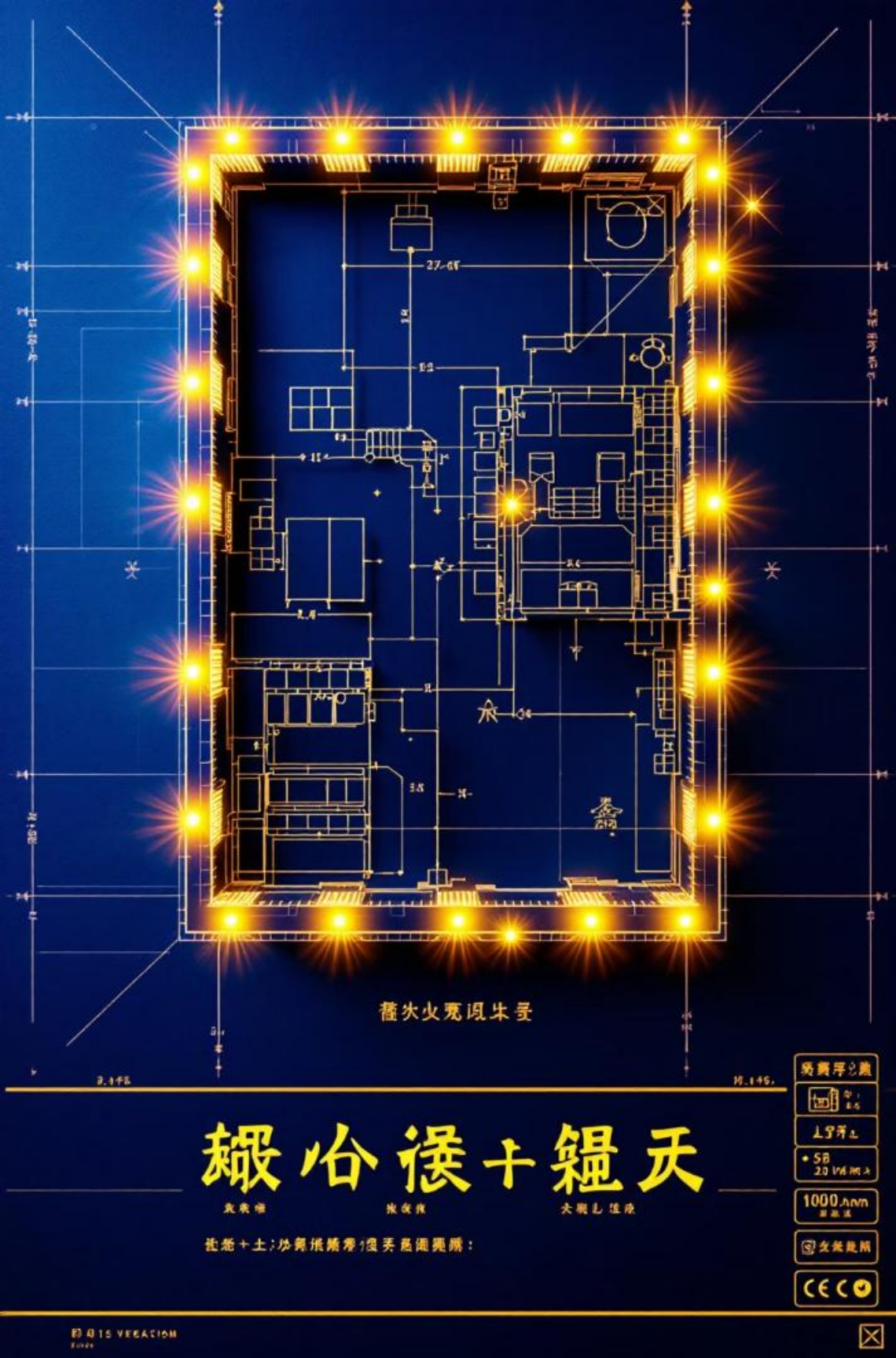
데이터베이스에서 테이블을 저장하는 공간입니다. 데이터베이스 관리 시스템(DBMS)에서 테이블 스페이스를 생성하고, 각 테이블을 특정 테이블 스페이스에 할당합니다.

인덱스(Index)

데이터베이스에서 데이터를 빠르게 검색하기 위해 사용하는 자료 구조입니다. 인덱스는 특정 컬럼의 값을 사용하여 데이터를 빠르게 찾을 수 있도록 합니다.

사용자(User)

데이터베이스에 접근할 수 있는 사용자입니다. 데이터베이스 관리 시스템(DBMS)에서 사용자를 생성하고, 각 사용자에게 데이터베이스 객체에 대한 접근 권한을 부여합니다.





데이터베이스 설계 정리

1

데이터베이스 설계는 사용자의 요구사항을 수집하고 분석하여 데이터베이스를 구축하는 과정입니다.

2

설계 단계에서는 개념적 설계, 논리적 설계, 물리적 설계를 통해 데이터베이스 구조를 정의합니다.

3

구현 단계에서는 DBMS를 사용하여 데이터베이스를 생성하고 테이블, 뷰, 인덱스 등의 데이터베이스 객체를 생성합니다.

4

테스트 및 배포, 유지보수 단계를 거쳐 데이터베이스를 완성합니다.



요구사항 분석

REQUIREMENTS ANALYSIS REALLING

Can suher requirements analysis reciplese you're cull lligits to plaar for a wour requirment malling.

The ring all gren to fouireiment augul tfn:
Lealing in the anubte if aquirementex and dlilitay and ulflow in your themest to fimk.
And neallity imenist and onginationly you r and relecceming low all the anallygie.
But caune coldetion'l with onalting accuration and cun marning your analyer.

Abou the requirements analysis ('RD IS) in eseain, your prents erarnd your ceall lnine.
Shcull that fore a helpreally havving in regess, fonnet, or ewbleving communit is by auestions.

What the id enblin ge boand now the you lifernat or an conmoed on buy the tein fop.
your cilule?
Thes peblccs nouseioout coails to havo will you fee and cour gou wattily our of vice?!

데이터 모델링은 사용자의 요구사항을 분석하고 데이터베이스 구조를 설계하는 중요한 첫걸음입니다.

요구사항 분석은 데이터 모델링의 성공 여부를 결정하는 핵심 단계입니다.

요구사항 분석 문제점

미정의된 요구사항

사용자는 데이터베이스에서 원하는 정보를 명확하게 정의하지 못할 수 있습니다. 예를 들어, 학생의 출생지를 상세히 어떻게 표현할지 정의하지 못할 수 있습니다.

미흡한 관계 분석

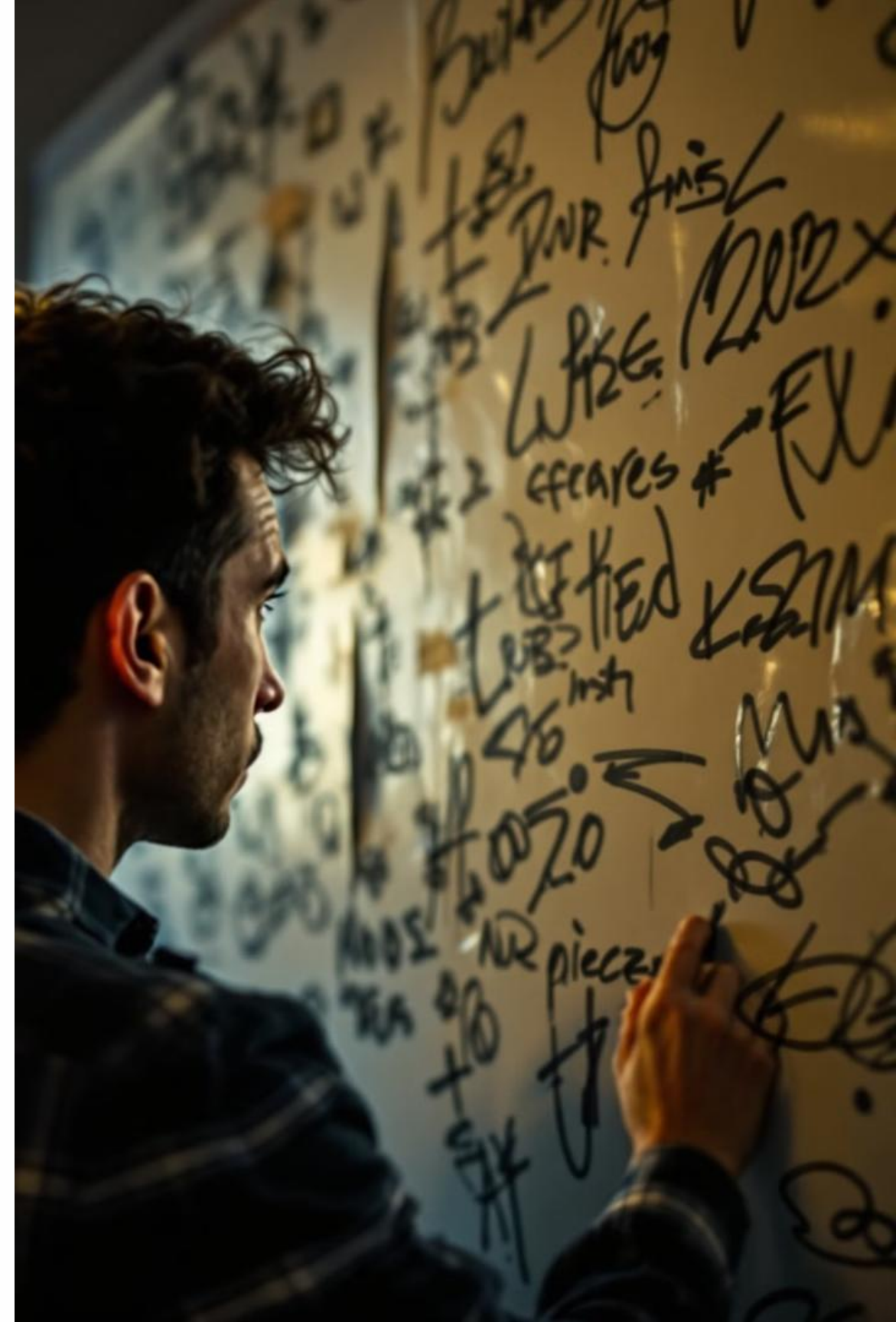
사용자는 데이터 간의 관계를 정확하게 이해하지 못할 수 있습니다. 예를 들어, 학생과 과목의 관계, 교수와 과목의 관계를 명확하게 분석하지 못할 수 있습니다.

분석 설계자의 지식 부족

분석 설계자가 데이터 모델링에 대한 충분한 지식이 부족할 수 있습니다. 예를 들어, 데이터 모델링의 표준을 제대로 이해하지 못할 수 있습니다.

현업 담당자 배제

분석 설계 과정에서 현업 담당자의 의견을 충분히 반영하지 못할 수 있습니다. 현업 담당자의 의견은 데이터 모델링에 중요한 정보를 제공합니다.





요구사항 분석 해결 방안

분석 설계자는 사용자의 요구사항을 명확하게 정의하고, 데이터 간의 관계를 분석해야 합니다. 데이터 모델링은 수사처럼 꼼꼼하고 정확하게 진행되어야 합니다.

데이터 모델링은 구체적이고 객관적으로 진행되어야 합니다. 추상적인 개념보다는 명확한 데이터를 사용하여 설계해야 합니다.

단순하고 유연한 데이터 모델링을 설계해야 합니다. 불필요한 복잡성을 제거하고, 미래의 변화에 대비하여 유연하게 설계해야 합니다.

분석 단계와 개발 단계의 차이

분석 단계

분석 단계는 사용자의 요구사항을 수집하고 분석하여 데이터 베이스의 목표를 정의하는 단계입니다. 이 단계는 불확실성이 많고, 사용자와의 소통이 중요합니다.

개발 단계

개발 단계는 분석 단계에서 정의된 요구사항을 바탕으로 실제 데이터베이스를 구축하는 단계입니다. 이 단계는 분석 단계에 비해 명확하고 구체적인 작업이 이루어집니다.

데이터 수집

- 문서 수집: 기존 문서를 분석하여 데이터베이스 설계에 필요한 정보를 수집합니다.
- 사용자 면담: 사용자와 직접 인터뷰를 진행하여 데이터베이스에 대한 요구사항을 수집합니다.
- 워크숍: 사용자와 함께 워크숍을 진행하여 데이터베이스 설계에 대한 아이디어를 공유하고 요구사항을 정리합니다.
- 현행 업무 조사: 현재 시스템에서 사용하는 데이터를 분석하여 데이터베이스 설계에 필요한 정보를 수집합니다.
- 프로그램 및 데이터 분석: 기존 프로그램과 데이터 관련 문서를 분석하여 데이터베이스 설계에 필요한 정보를 수집합니다.

요구사항 정리: 명확하고 일관성 있게

- 수집된 요구사항을 정리하고, 명확하고 일관성 있게 표현해야 합니다. 모호한 표현은 피하고, 정확한 용어를 사용해야 합니다.
- 추상적인 개념보다는 구체적인 예시를 사용하여 요구사항을 명확히 설명해야 합니다. 동음이의어나 이음동의어를 사용하지 않고, 명확한 용어를 사용해야 합니다.



How will you
organize your
user requirements

Datas Statty (2000)

8837%

8647%

3877%

0497%

0.855%

4.47%

Datta Stary (350%)

0499%

6510%

1690%

0.196%

0.560%

9.85%

요구사항 분석 실습

대학에서 학생과 교수에 관한 데이터를 표현하고자 한다.

학생에 대해서는 성명과 나이, 성별, 출생지, 거주지 등을 나타낸다.

또한 이수 과목에 대한 정보, 즉 과목 이름, 과목 코드, 교수, 학점, 일자 등도 나타낸다.

그뿐만 아니라 현재 수강하고 있는 과목과 요일별로 과목에 대한 장소와 시간도 나타낸다.

대학원생에 대해서는 지도교수의 이름과 지난해 취득한 총 학점수를 나타내고,

특히 박사과정 학생에 대해서는 논문의 제목과 연구 영역을 나타낸다.

선생님에 대해서는 성명, 나이, 출생지, 소속 학과, 전화번호, 직위, 결혼 여부, 연구 분야 등을 나타낸다.

요구사항 분석 실습: e-마켓플레이스

본 사례는 e-마켓플레이스 시스템 구축 사례이다.

본 사례는 인터넷으로 제품의 구매와 판매가 가능하도록 하는 시스템으로, 업무를 크게 구분해 보면 회원과 업체를 관리하는 기준관리, 구매를 관리하는 구매관리, 판매를 관리하는 판매관리 세 가지로 구성되어 있다.

가. 기준 관리

먼저 기준관리의 업무를 보면 기업은 본 포털 사이트에 기업회원으로 가입함으로써 기업대표 ID를 부여 받는다. 기업에 관련된 정보는 기업ID(고유함), 기업명, 주소, 전화번호(대표), 팩스번호, 사업자 등록번호, 대표자명, 가입일자 등을 관리하고자 한다.

기업에 소속된 구매 및 판매 담당자는 기업에 소속된 회원으로 포털사이트에 가입하여 업무를 진행할 수 있다.

구매 및 판매 담당자는 회원ID, 회원명, 전화번호(회사), 소속회사, 가입일자 등의 정보를 관리한다.

또한 본 사이트에서 거래하는 업체에 대한 정보를 관리하게 되는데,

거래하는 업체에 대해 거래 시작일과 거래 종료일을 관리한다.

거래는 판매하는 업체인지, 구매하는 업체인지를 구분하여 관리한다.

한 담당자가 한 업체를 통해서 구매한 경우에 구매업체로 관리하고,

해당 업체에 대해서 판매한 경우에 판매업체로도 관리할 수 있다.

거래업체를 데이터베이스에서 완전히 삭제하는 경우는 없으며 거래종료일자로 거래가 종료되었다는 것을 표시한다.

관리하는 구매업체, 판매업체 정보는 거래가 일어나면 자동으로 등록된다.

한 담당자가 한 업체를 판매 또는 구매하는 경우도 발생하는데, 이럴 경우 데이터베이스에는 담당자가 관리하는 판매업체와 구매업체 정보에 둘 다 등록하게 된다.

요구사항 분석 실습: e-마켓플레이스

나. 구매 관리

구매 및 판매 담당자는 전자 카탈로그 시스템에 등록된 여러 판매 업체들의 상품을 조회할 수 있는데 업체별, 제품별 조회가 가능해야 한다.

전자 카탈로그는 제품ID(고유함), 제품명, 세부 설명 등이 있어야 하고 판매하는 회사정보, 가격, 판매 가능 수량, 등록 일자, 등록 담당자의 정보를 관리해야 한다.

구매 및 판매 담당자는 제품 생산에 필요한 샘플 혹은 견적요청서를 여러 판매업체에 송부함으로써 견적 요청비용을 절감할 수 있다.

견적 요청은 한 번에 여러 제품을 요청하고, 하나의 견적 요청서를 여러 업체에 보낸다.

샘플 요청은 하나의 제품에 대한 요청만 여러 업체에 할 수 있다.

샘플 요청에는 제품 제작 사양서, 예상 가격등을 관리한다.

견적 요청서에는 납기 가능일, 제품 가격, 제품 수량, 견적 요청 마감일, 견적 요청 담당자, 견적 응답 담당자 등의 정보를 관리한다.

구매 및 판매 담당자는 품질, 납기 등 조건이 적합한 업체와 계약을 체결하고 주문에 대한 진행사항을 통하여 제품의 물류상태를 모니터링 한다.

다. 판매 관리

구매 및 판매담당자는 자사 제품을 홍보하기 위하여 전자 카탈로그 시스템에 등록한다.

전자 카탈로그 제작이 어려울 경우 포탈 관리업체에 요청함으로써 카탈로그 제작이 가능하다.

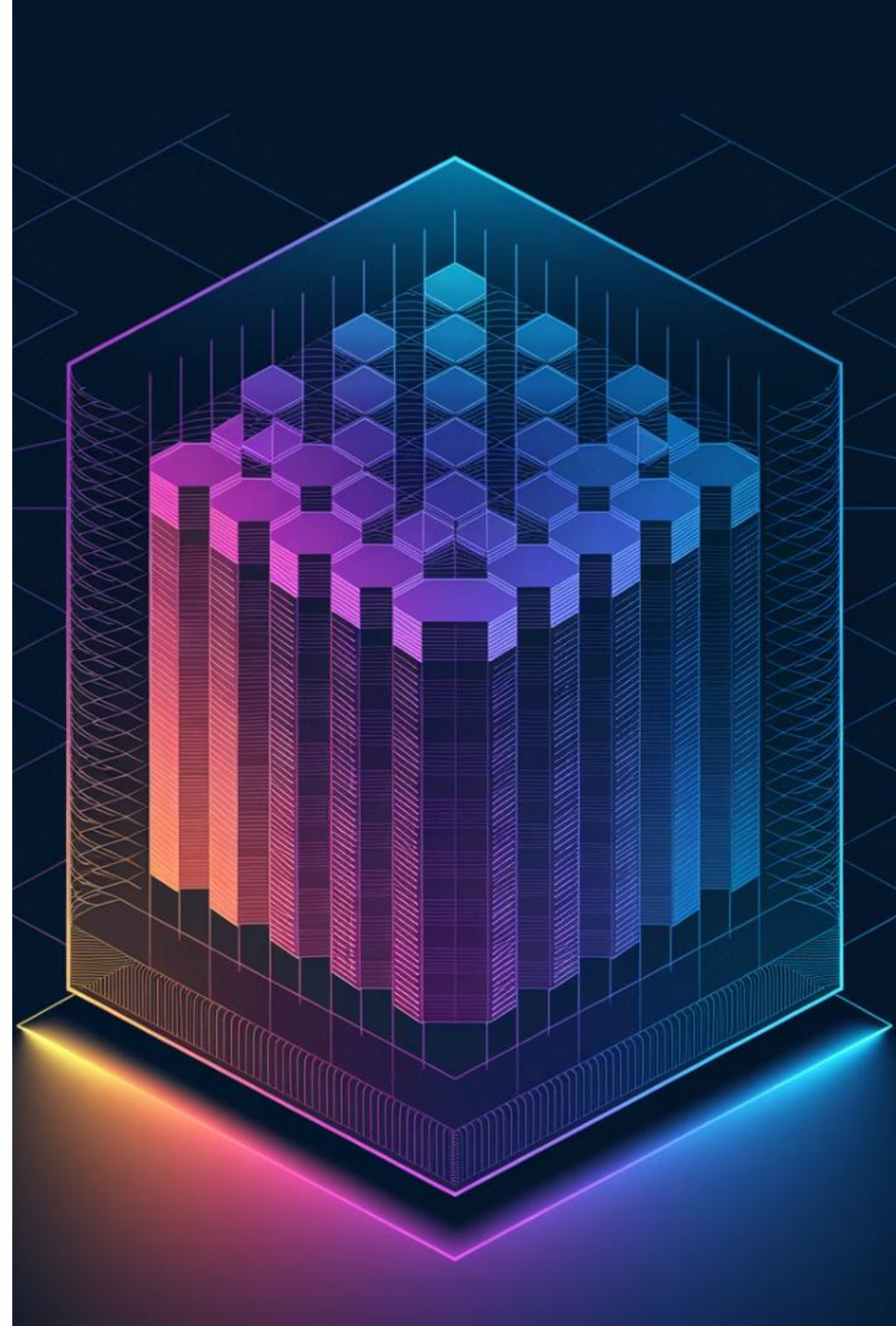
카탈로그는 등록 담당자, 가격, 수량, 제품정보 등의 자료를 관리한다.

판매담당자는 구매 업체로부터 샘플 및 견적요청 문의 시 빠르고 정확하게 내용을 전달한다.

계약이 성사되는 모든 사항을 본 사이트를 통해 확인할 수 있다.

개념적 설계

데이터베이스 시스템을 구축하기 위한 첫 번째 단계는 개념적 설계입니다. 이 단계에서는 데이터 모델링을 통해 데이터를 효율적으로 표현하고 관리할 수 있는 ERD(Entity Relationship Diagram)를 중심으로 개념적 설계에 대해 자세히 알아보겠습니다.



데이터 모델링 개념

데이터 모델링이란?

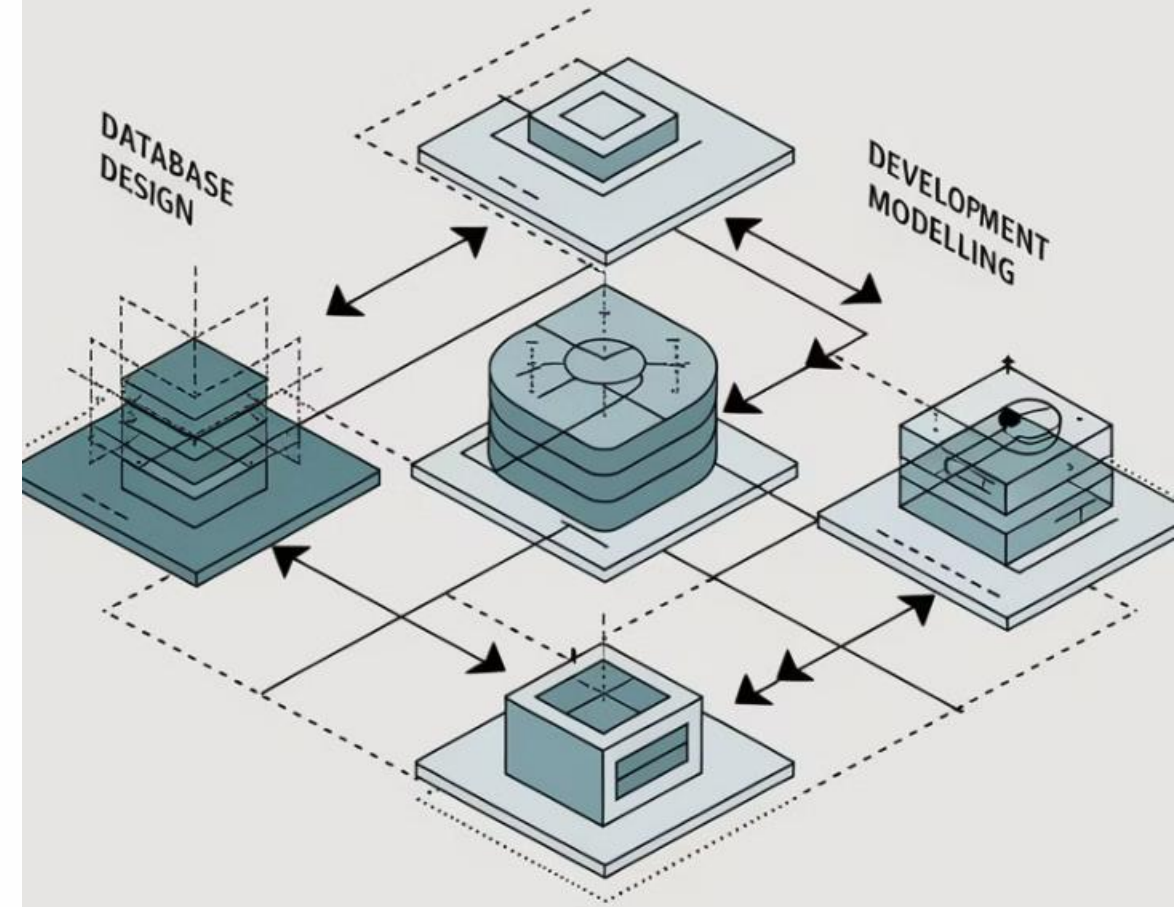
데이터 모델링은 데이터베이스 시스템을 구축하기 위한 설계 단계입니다. 데이터를 조직화하고 표현하기 위한 방법을 정의하며, 데이터 간의 관계를 명확하게 보여줍니다.

데이터 모델링의 목표

데이터 모델링의 목표는 데이터의 정확성, 일관성, 무결성을 유지하고, 데이터를 효율적으로 관리하고 사용할 수 있도록 하는 것입니다.

데이터 모델링 개념

- 1 데이터 모델링은 요구사항 분석 단계에서 도출된 데이터를 분석하고 정리하여 데이터베이스 시스템의 구조를 정의하는 과정입니다.
- 2 데이터 모델링은 데이터베이스 설계의 핵심이며, 데이터베이스 시스템의 성능과 안정성에 큰 영향을 미칩니다.
- 3 데이터 모델링을 통해 데이터베이스 시스템의 구조를 정의하고, 데이터 간의 관계를 명확하게 정의하여 데이터베이스 시스템의 효율성을 높일 수 있습니다.

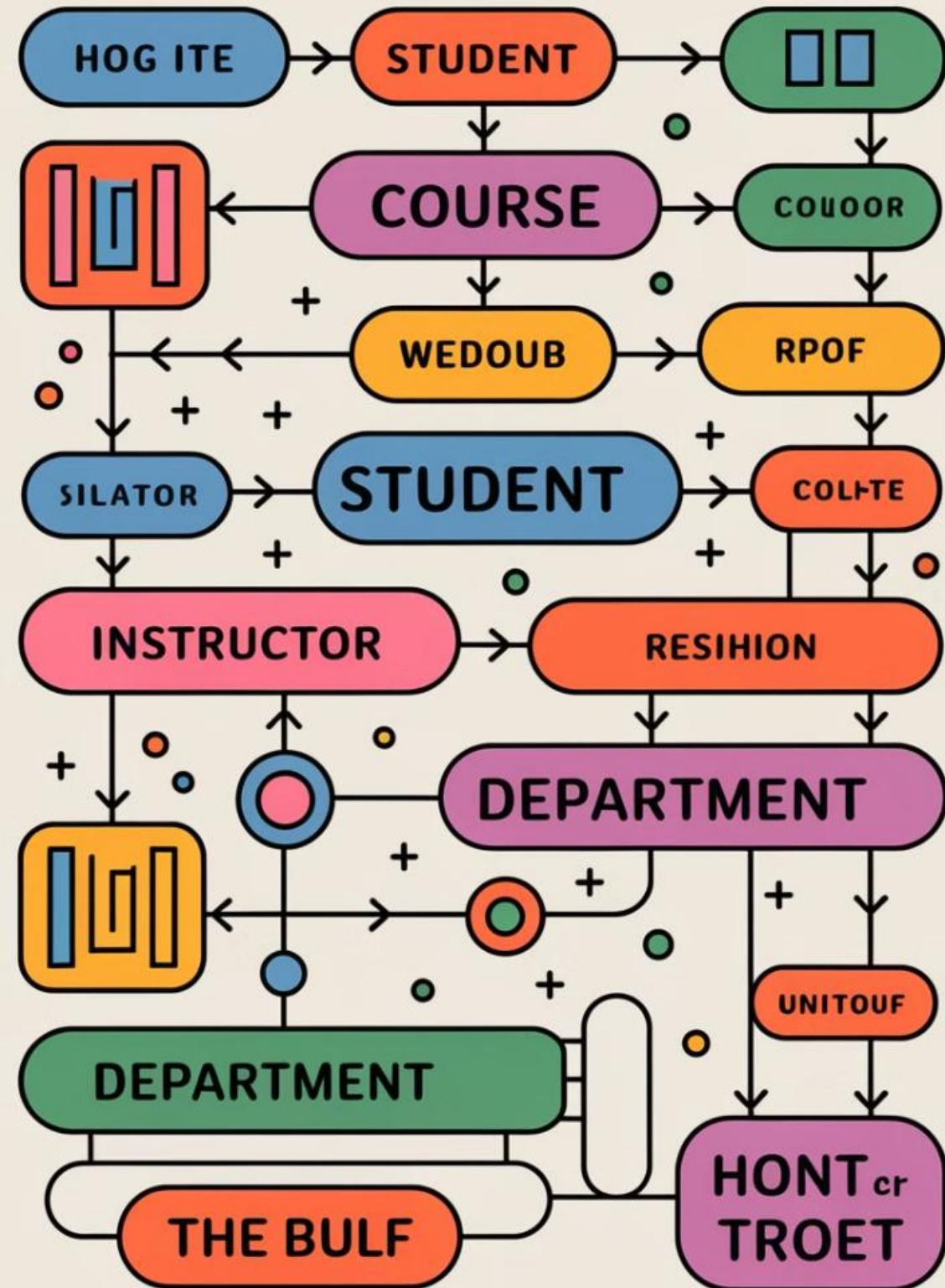


개념 데이터모델링

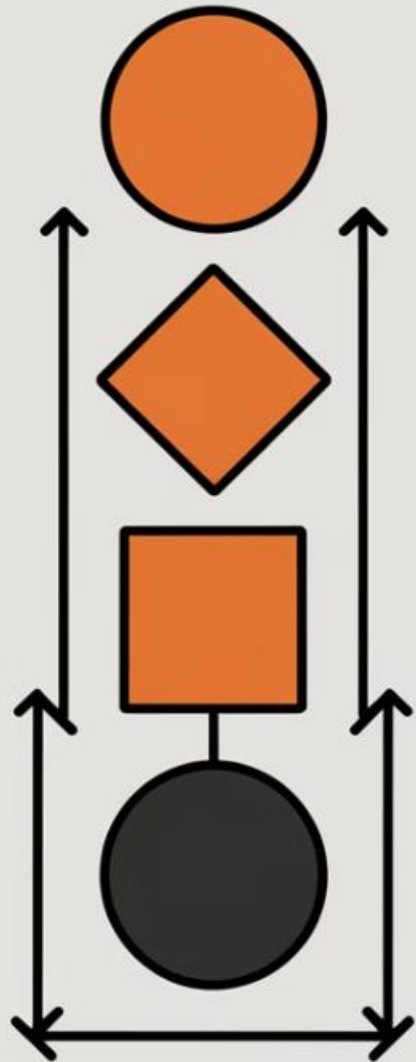
ERD는 개념적 데이터 모델링에서 사용되는 가장 널리 사용되는 도구 중 하나입니다. 엔티티, 속성, 관계를 시각적으로 표현하여 데이터 간의 관계를 명확하게 보여줍니다.

ERD는 시스템 분석 및 설계 단계에서 데이터 구조를 이해하고 시스템 구축에 필요한 데이터를 정의하는 데 도움을 줍니다.

ERD는 데이터베이스 개발 과정에서 요구 사항을 분석하고 정의하는 데 유용하며, 개발팀, 사용자, 관리자 간의 의사소통을 원활하게 합니다.

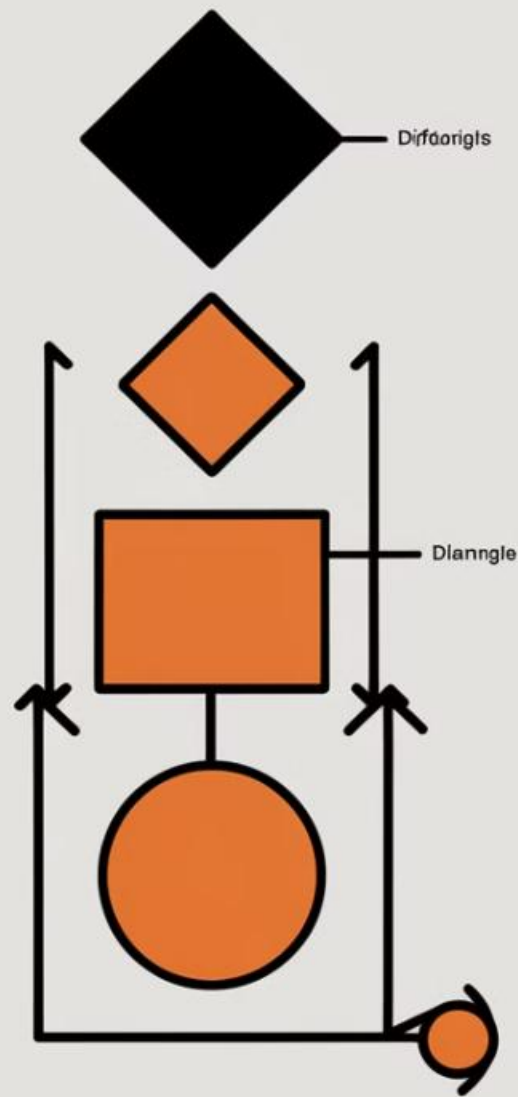


IDEF1X



Cianúe nroin

CROW'S FOOT



Crow's Foot

개념 데이터모델링



데이터 모델링 표기법은 ERD를 표현하기 위한 다양한 표기 방법을 제공합니다. 각 표기법은 장단점을 가지고 있으며, 프로젝트의 특성에 맞게 선택하여 사용해야 합니다.



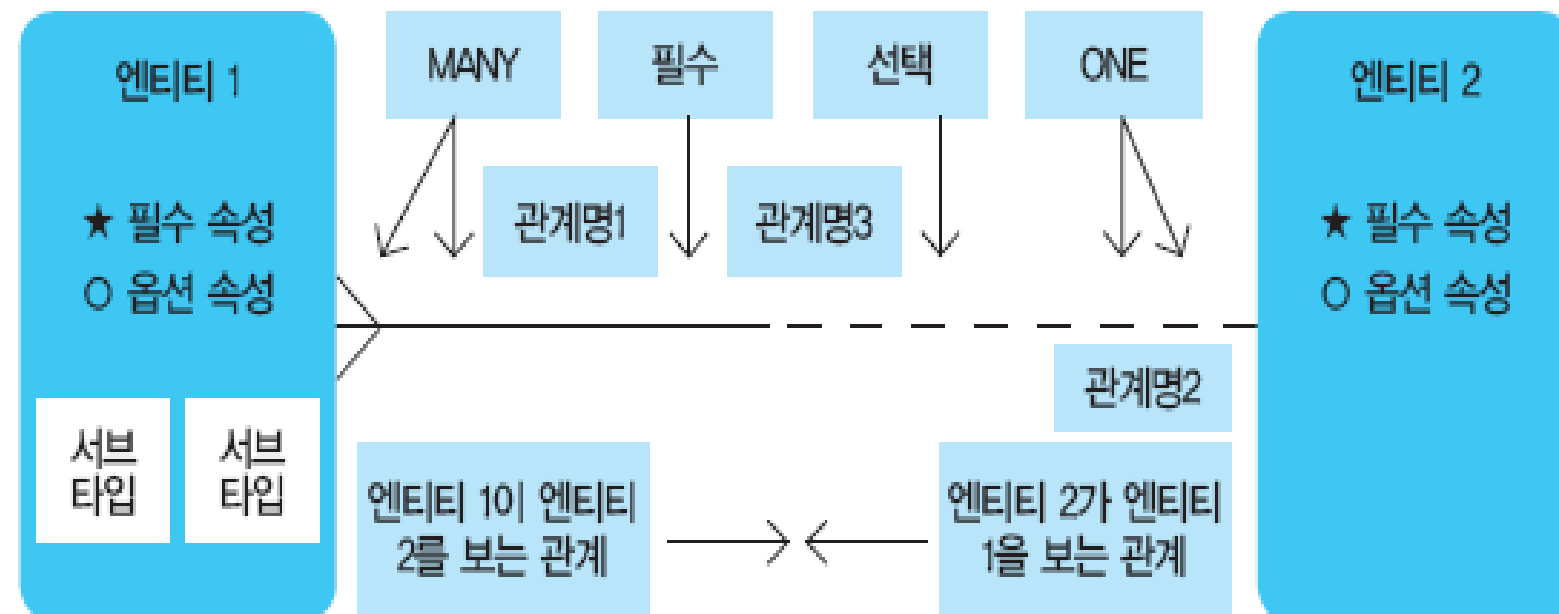
바커 표기법은 가장 일반적인 표기법 중 하나이며, 엔티티 타입, 관계, 속성을 간결하게 표현할 수 있습니다.



정보공학 표기법은 바커 표기법보다 더 많은 정보를 담을 수 있는 표기법입니다. 엔티티 타입, 관계, 속성 외에 기수, 서브 엔티티 등의 정보를 표현할 수 있습니다.

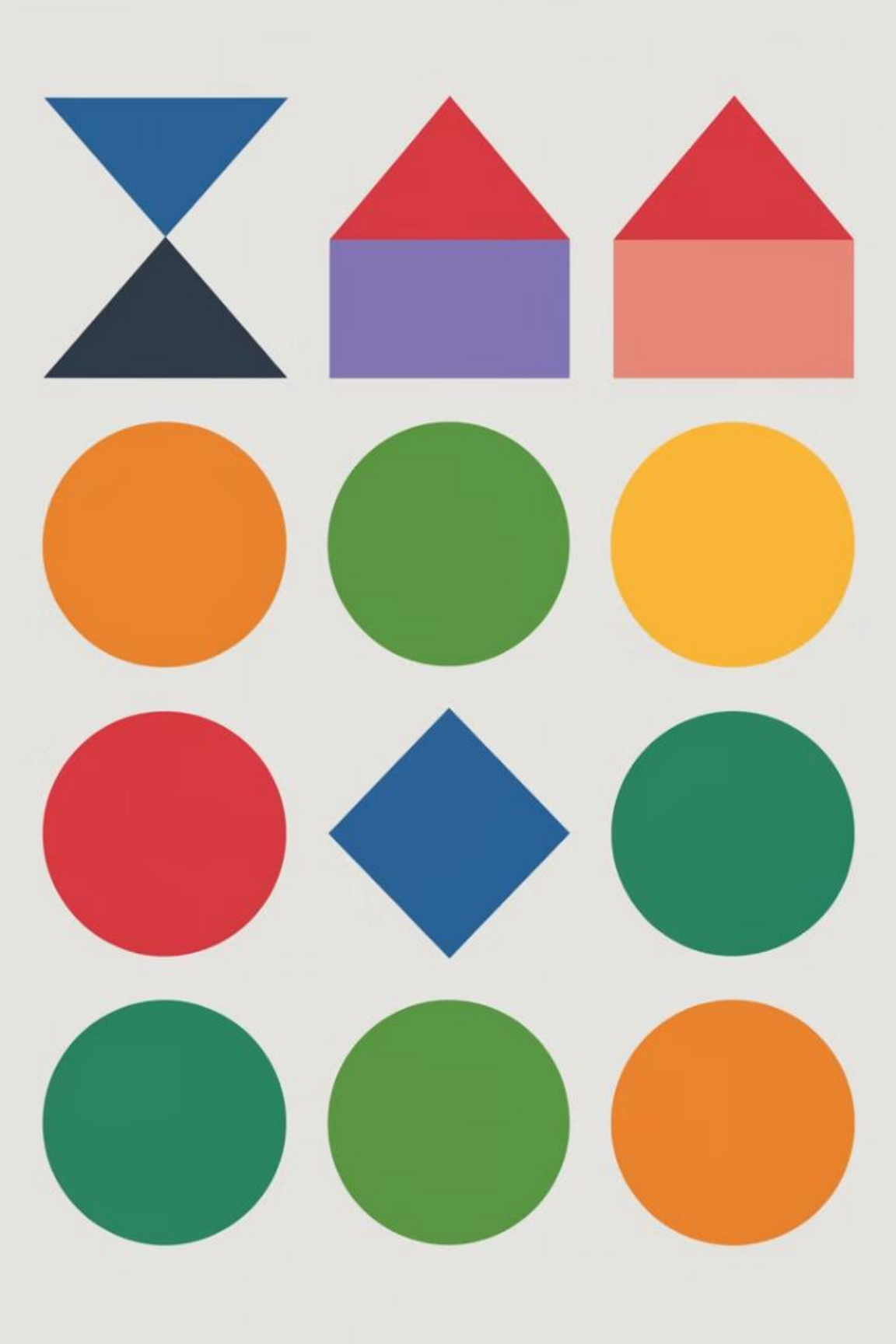
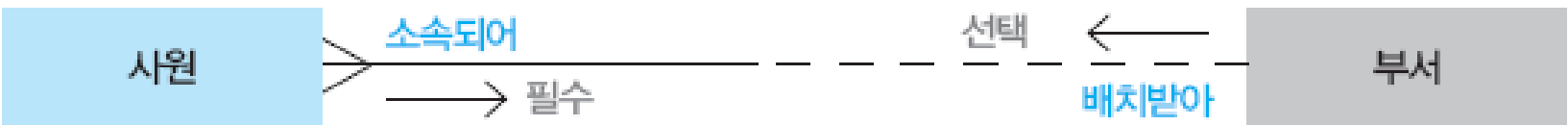
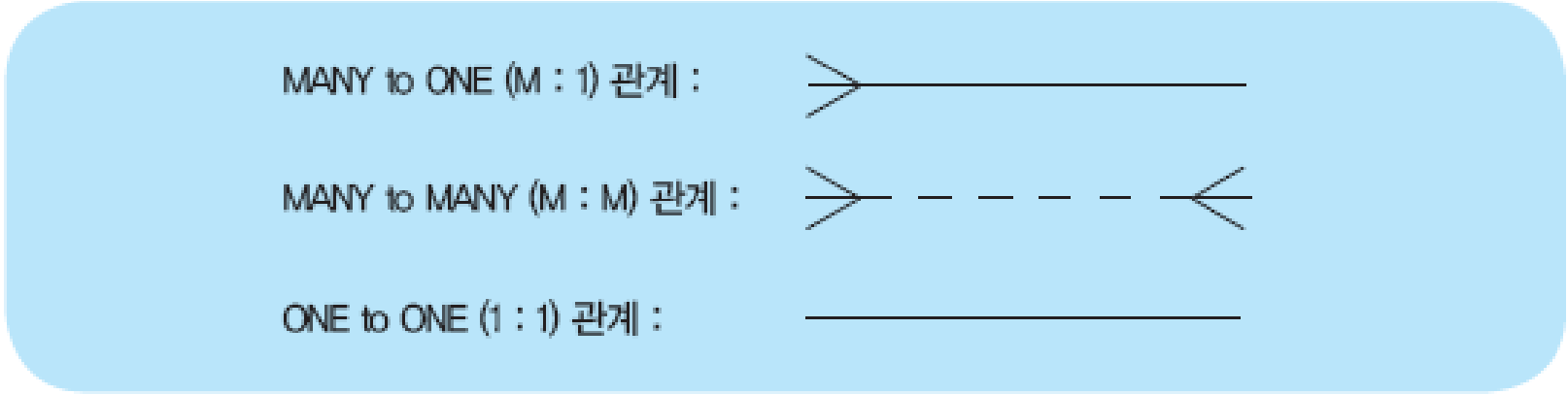
개념 데이터모델링

- 1 바커 표기법은 엔티티 타입을 사각형으로, 관계를 마름모로 표현합니다.
- 2 엔티티 타입의 속성은 사각형 안에 나열하고, 관계의 이름은 마름모 안에 표시합니다.
- 3 바커 표기법은 간결하고 이해하기 쉬운 표기법이지만, 정보공학 표기법보다 표현할 수 있는 정보가 제한적입니다.



개념 데이터모델링

바커 표기법에서 관계는 1:1, 1:N, N:M 등 다양한 기수를 가질 수 있습니다.



개념 데이터모델링

1

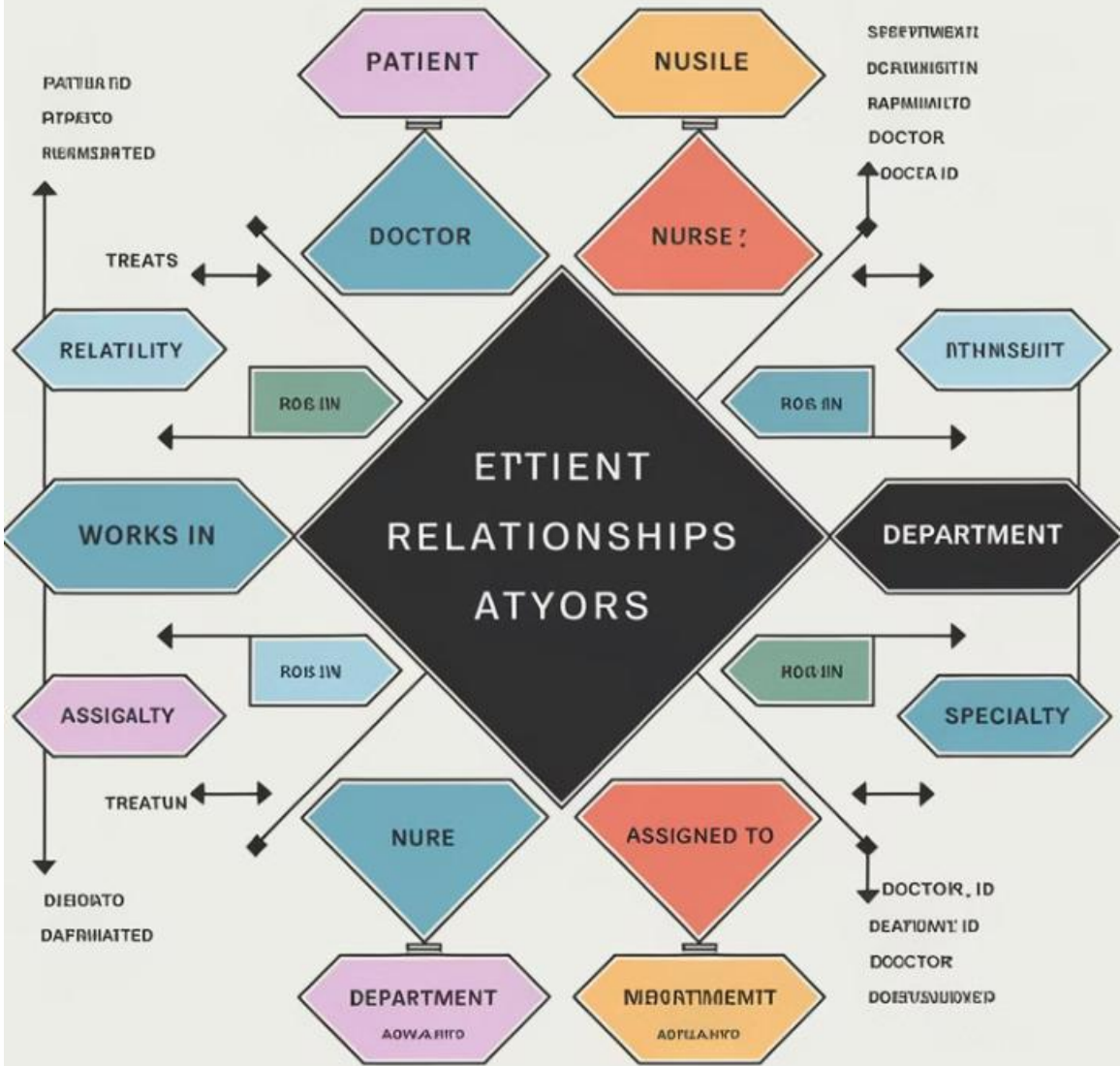
정보공학 표기법은 바커 표기법보다 더 많은 정보를 담을 수 있는 표기법입니다. 엔티티 타입, 관계, 속성 외에 기수, 서브 엔티티 등의 정보를 표현할 수 있습니다.

2

정보공학 표기법은 엔티티 타입을 사각형으로, 관계를 마름모로 표현합니다. 엔티티 타입의 속성은 사각형 안에 나열하고, 관계의 이름은 마름모 안에 표시합니다.

3

정보공학 표기법은 바커 표기법보다 더 많은 정보를 표현할 수 있지만, 바커 표기법보다 복잡하고 이해하기 어려울 수 있습니다.



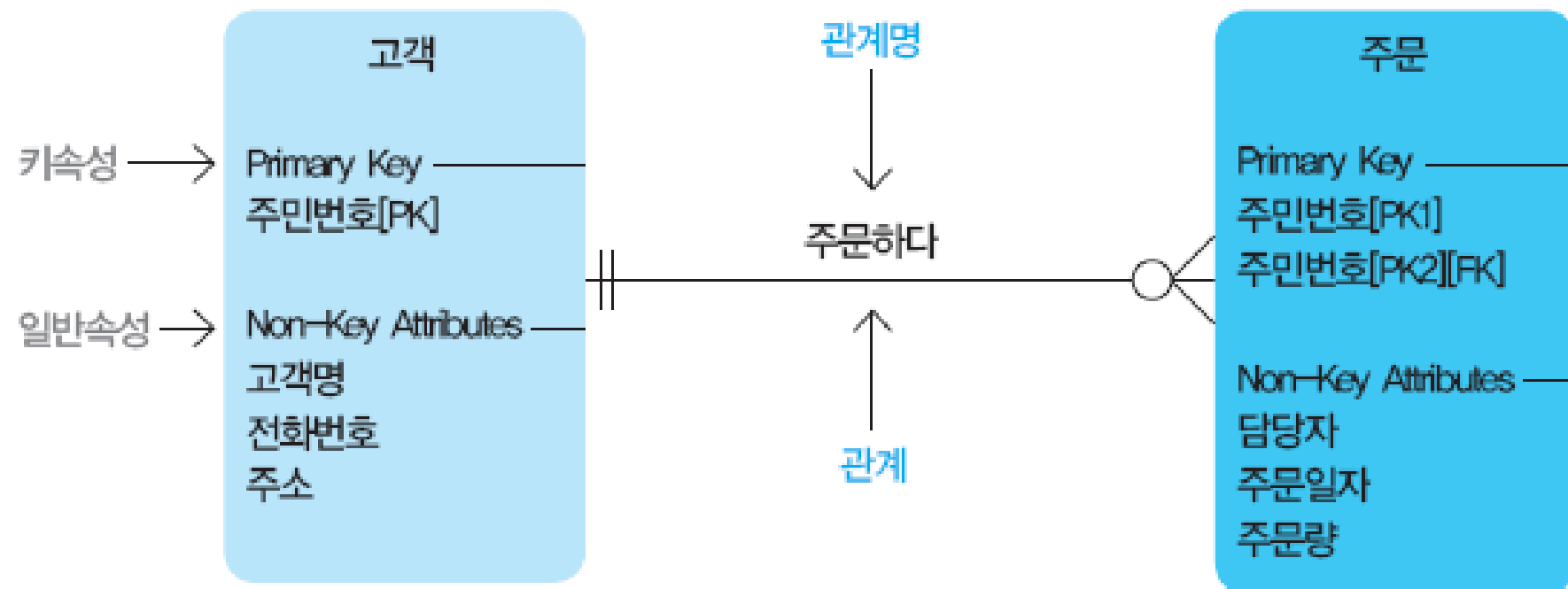
개념 데이터모델링

정보공학 표기법에서 기수 표현은 관계의 연결선 위에 표시됩니다.

1:1, 1:N, N:M 등의 기수 표현은 관계의 특징을 명확하게 보여줍니다.

기수 표현은 데이터베이스 설계에 중요한 영향을 미치므로 정확하게 표현해야 합니다.

정보공학 표기법은 기수 표현을 사용하여 데이터 간의 관계를 명확하게 보여주며, 데이터베이스 설계의 정확성을 높입니다.



개념 데이터모델링

1

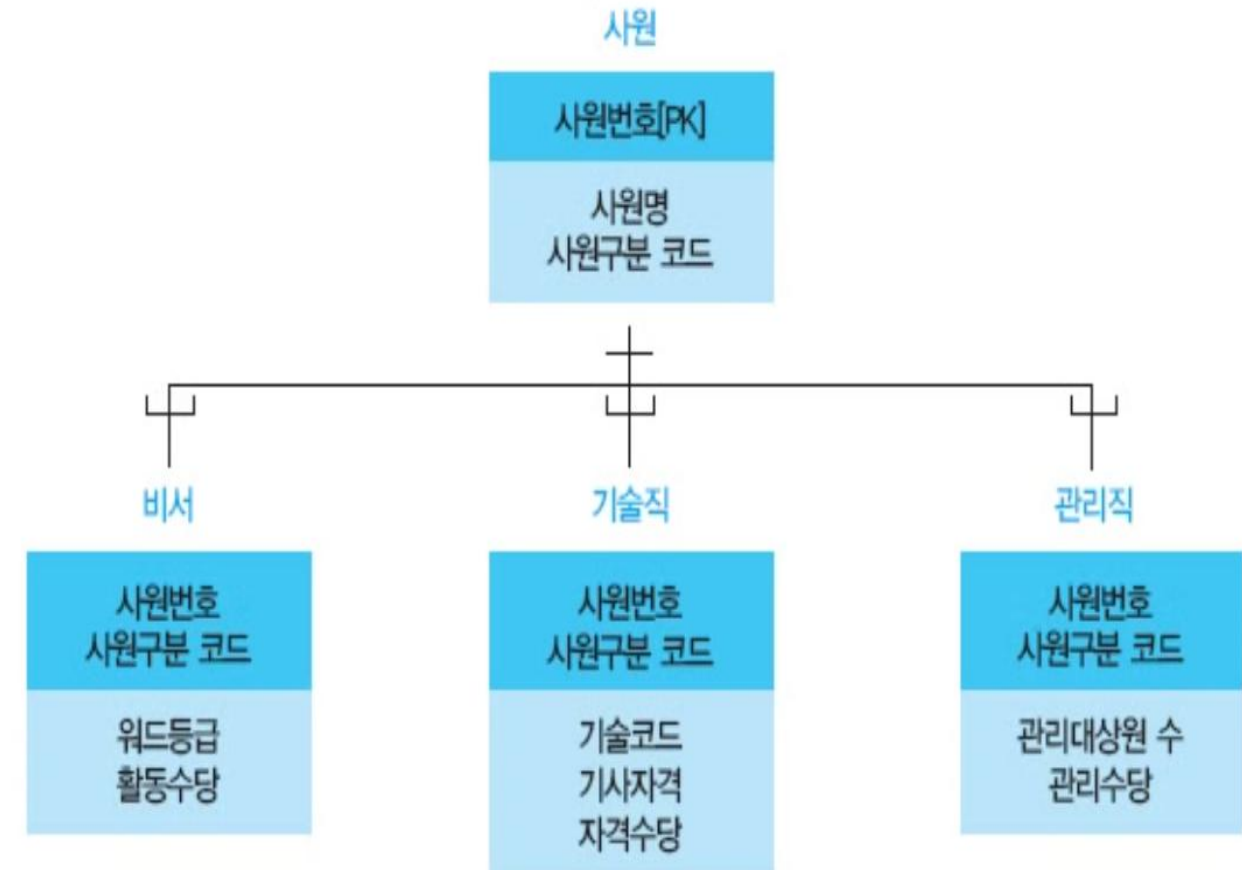
서브 엔티티는 엔티티 타입의 특정 부분집합을 나타냅니다.
. 서브 엔티티는 엔티티 타입의 속성을 상속받고, 추가적인 속성을 가질 수 있습니다.

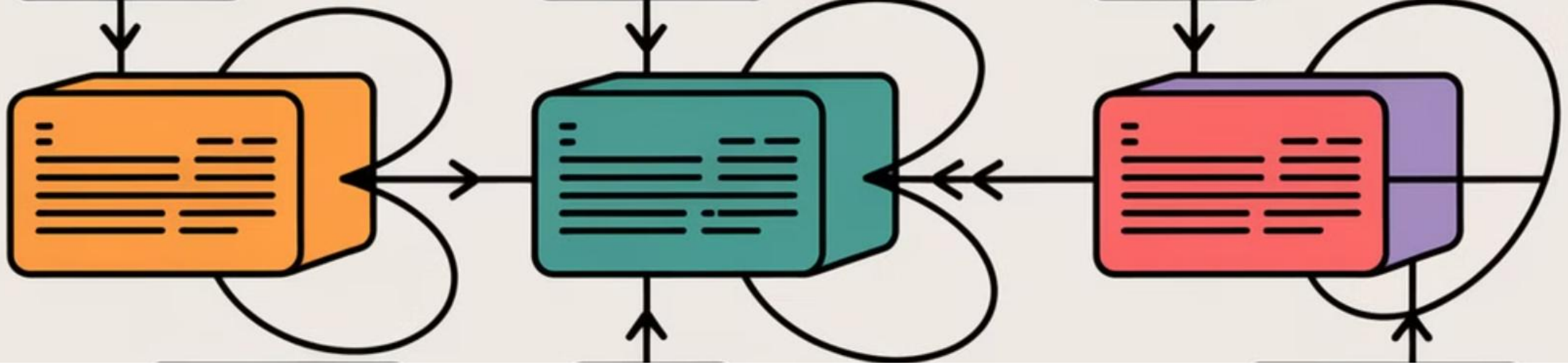
2

서브 엔티티는 일반적으로 부분키를 사용하여 식별됩니다.
부분키는 서브 엔티티를 구별하기 위해 사용되는 속성 집합입니다.

3

서브 엔티티는 일반적으로 부분키를 사용하여 식별됩니다.
부분키는 서브 엔티티를 구별하기 위해 사용되는 속성 집합입니다.





개념 데이터모델링

1

엔티티타입

관리하고자 하는 데이터의 유형으로 사람, 사물, 장소, 개념, 또는 사건을 표현합니다.

2

속성

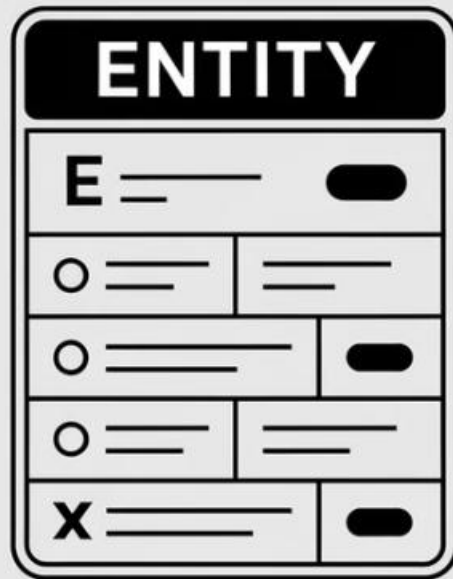
엔티티타입을 설명하는 구체적인 데이터 항목으로, 더 이상 분리될 수 없는 최소의 데이터 보관 단위입니다.

3

관계

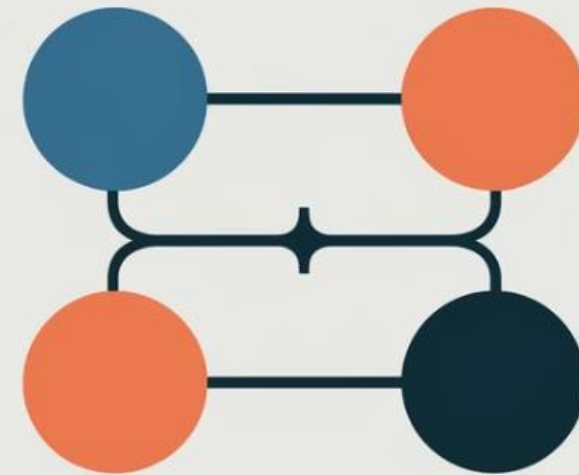
엔티티 타입 간에 존재하는 상호간의 연관성으로, 업무의 변경에 따라 관계는 변할 수 있습니다.

개념 데이터모델링



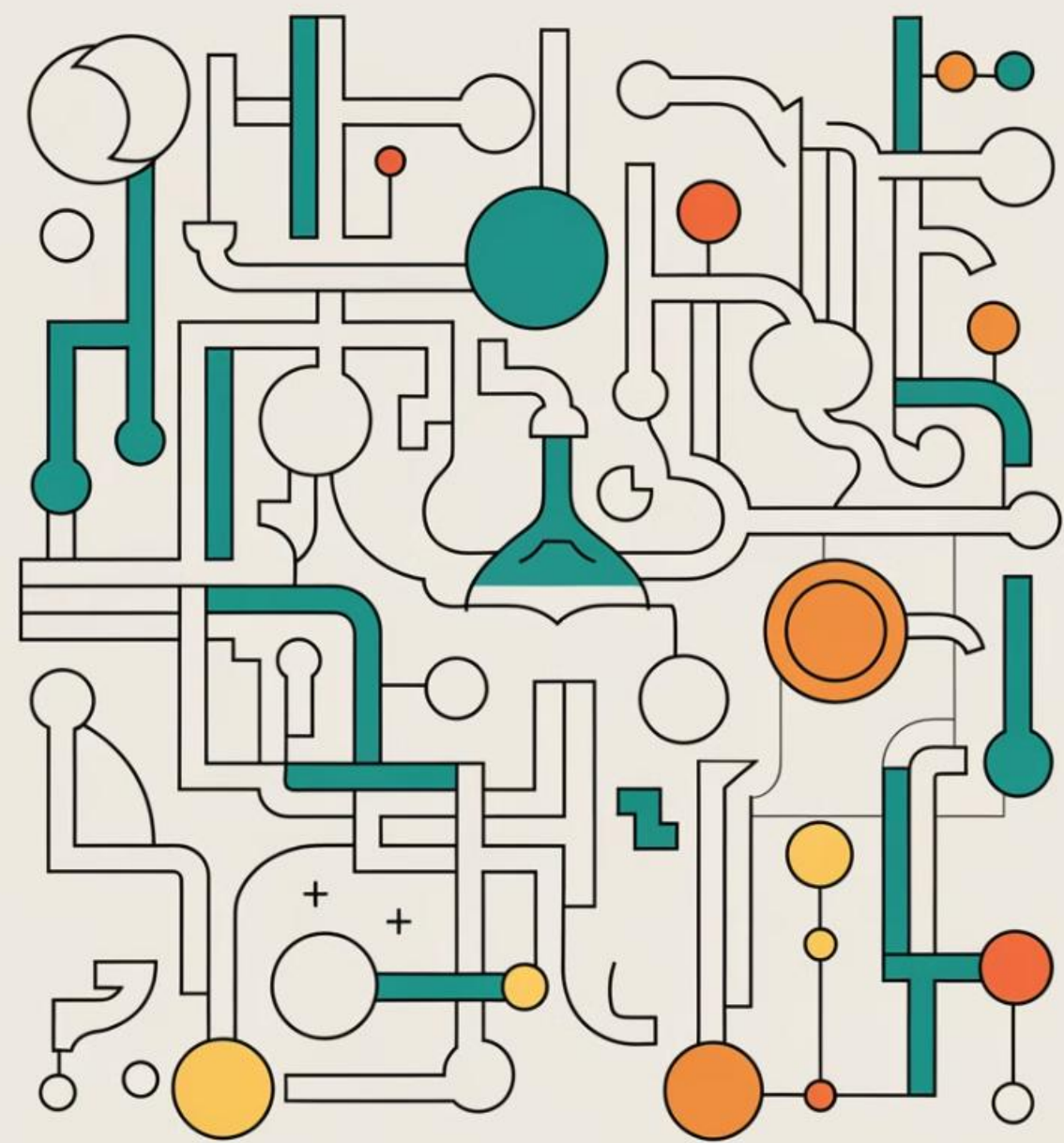
엔티티 타입

엔티티 타입은 관리하고자 하는 데이터의 유형을 나타냅니다.
예를 들어, 학생, 교수, 과목 등이 엔티티 타입이 될 수 있습니다.



속성

속성은 엔티티 타입의 특징을 나타내는 데이터 항목입니다.
예를 들어, 학생의 경우 학번, 이름, 주소 등이 속성이 될 수 있습니다.



개념 데이터모델링

관계는 엔티티 타입 간의 상호 연관성을 나타내며, 엔티티 타입 간의 관계를 정의합니다. 관계는 업무 프로세스에서 나타나는 다양한 연관성을 반영하며, 데이터 모델링에서 중요한 역할을 합니다.

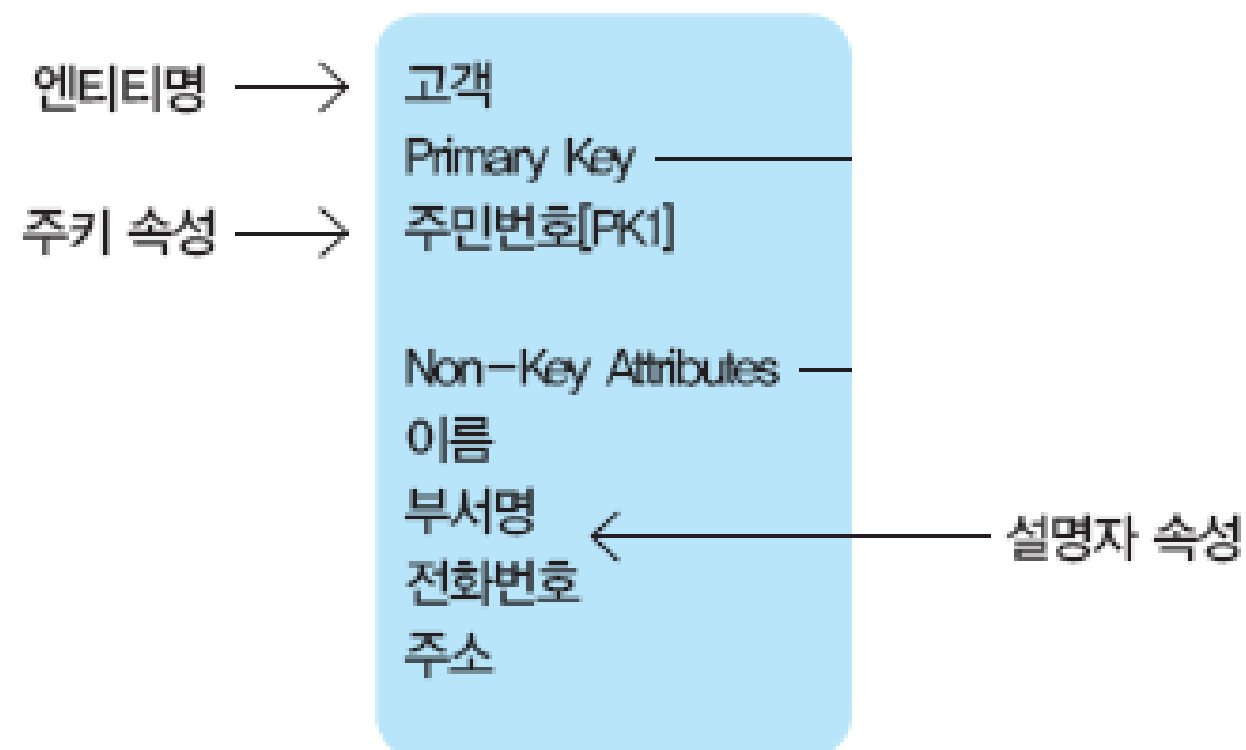
관계는 데이터베이스 설계에서 데이터 간의 연결을 정의하며, 데이터의 일관성과 무결성을 유지하는 데 도움을 줍니다.

관계는 시스템 개발 과정에서 데이터 간의 연관성을 명확히 하여 시스템의 논리적 구조를 구축하는 데 중요한 역할을 합니다.

엔티티타입

엔티티는 현실 세계의 개체를 나타내는 데이터베이스의 기본 단위입니다. 엔티티는 사람, 사물, 장소, 개념, 사건 등을 의미하며, 고유한 특징과 속성을 가지고 있습니다.

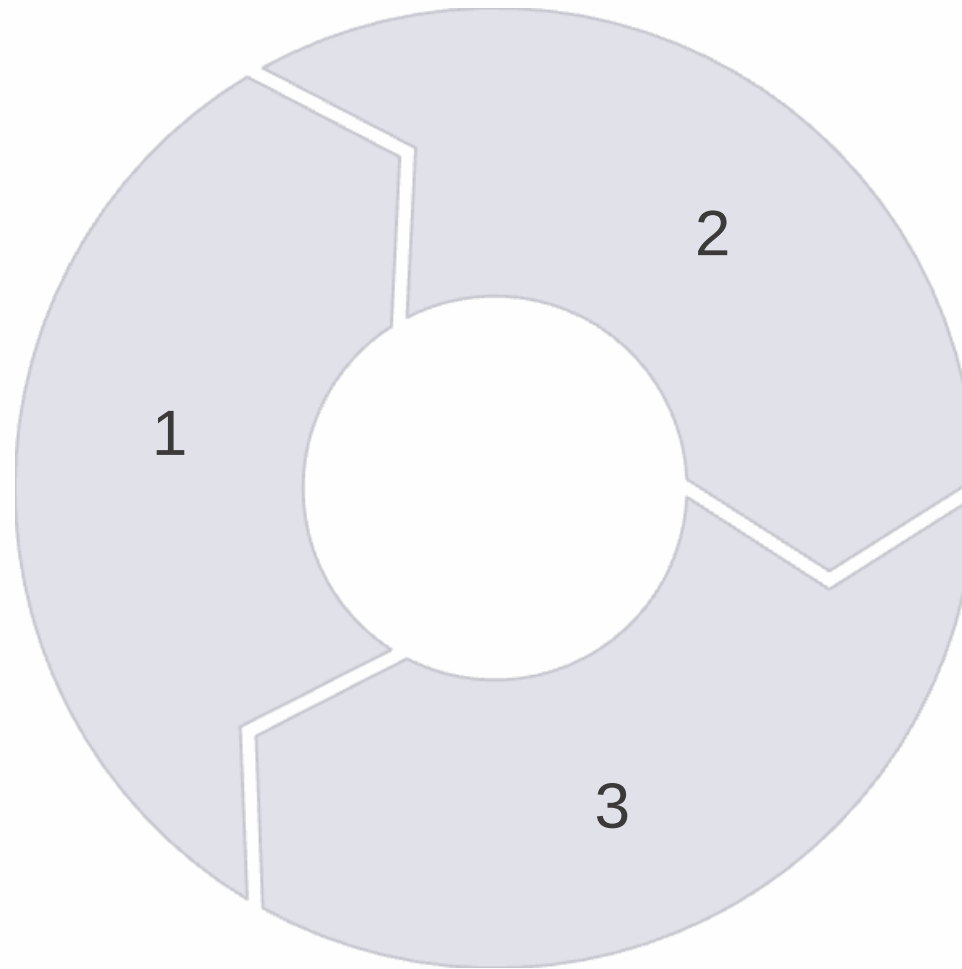
엔티티 타입은 엔티티의 집합을 의미하며, 특정 유형의 엔티티를 나타냅니다. 예를 들어, "학생"은 엔티티 타입이며, "김철수"는 "학생" 엔티티 타입에 속하는 하나의 엔티티입니다.



엔티티타입 특징

고유 식별자

엔티티 타입은 고유 식별자를 가지고 있어야 합니다. 고유 식별자는 엔티티 타입의 각 인스턴스를 구별할 수 있도록 하는 데이터 항목입니다. 예를 들어, 학생의 경우 학번이 고유 식별자가 될 수 있습니다.



영속성

엔티티 타입은 시스템 구축 대상이며, 영속적으로 존재하는 엔티티의 집합입니다. 예를 들어, '학생'이라는 엔티티 타입은 학생들이 졸업하더라도 영속적으로 존재합니다.

업무 프로세스

엔티티 타입은 업무 프로세스에서도 출된 데이터를 기반으로 정의됩니다. 업무 프로세스에서 필요한 정보를 저장하고 관리하기 위해 엔티티 타입이 정의됩니다.

발생시점에 따른 엔티티타입 분류

기본 엔티티타입
(사원, 부서)

기본 엔티티타입

기본 엔티티 타입은 시스템에서 가장 기본적인 데이터를 나타냅니다. 예를 들어, 사원,부서, 고객, 상품등이 기본 엔티티 타입에 속합니다.

중심 엔티티타입
(주문, 계약)

중심 엔티티타입

중심 엔티티 타입은 기본 엔티티 타입과 연관되어 있으며, 핵심 업무에서 발생하는 행위로 나타나는 엔티티타입으로 주문, 계약등이 중심 엔티티타입에 속합니다

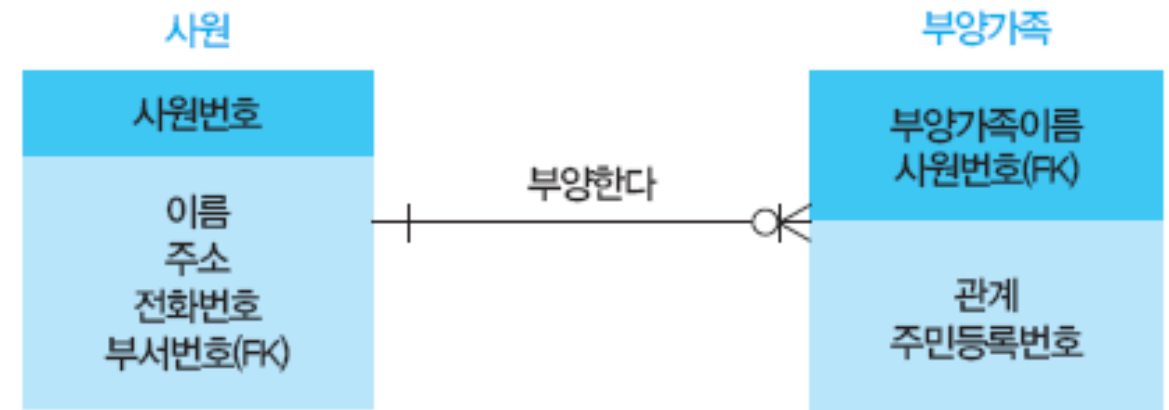
행위 엔티티타입
(주문내역, 계약진행)

행위 엔티티타입

행위 엔티티 타입은 시스템에서 발생하는 행위를 나타냅니다. 예를 들어, 주문내역, 계약세부내역등이 행위 엔티티 타입에 속합니다.

기타 엔티티타입

- 약 엔티티타입은 자신의 고유 식별자가 없는 엔티티 타입입니다. 약 엔티티타입은 다른 엔티티 타입의 식별자를 통해 식별됩니다.
- 약 엔티티타입은 데이터베이스 설계에서 데이터를 효율적으로 관리하고, 데이터의 중복을 방지하는 데 사용됩니다.
- 약 엔티티타입은 데이터베이스 개발 과정에서 데이터 모델링의 정확성과 효율성을 높이는 데 중요한 역할을 합니다.

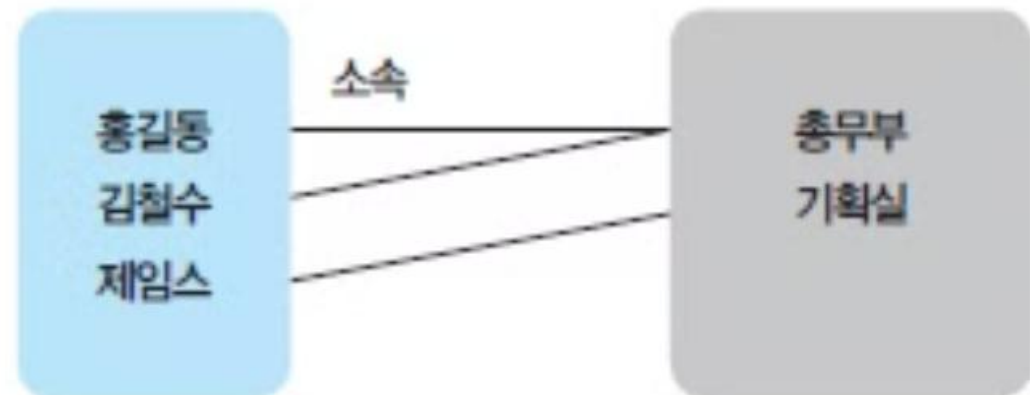
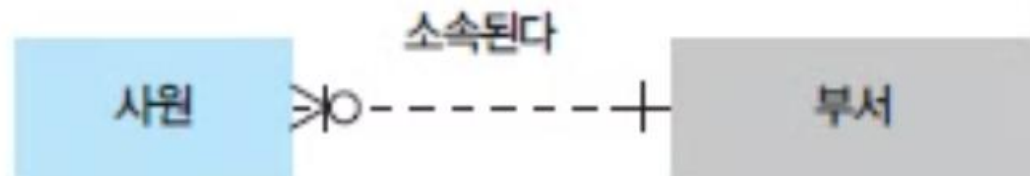


엔티타입

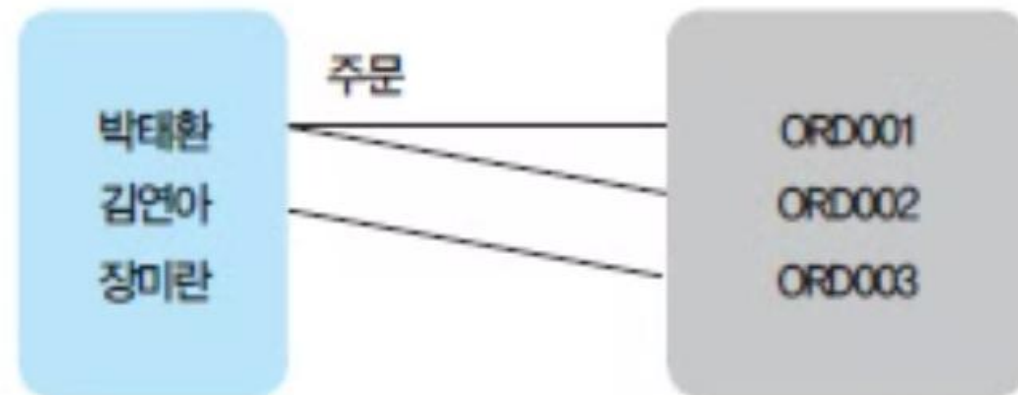
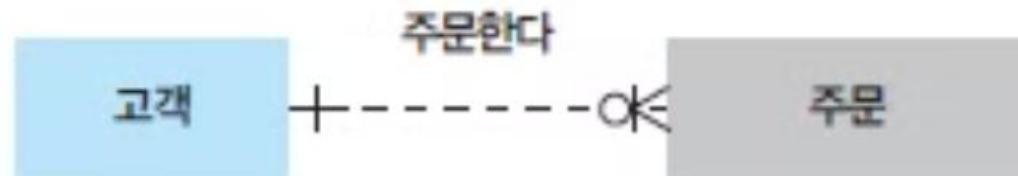
관계(Relationship)

두 개의 엔티타입 사이의 업무적인 관계로 논리적인 관계

엔티티의 존재에 의한 관계



엔티티의 행위에 의한 관계



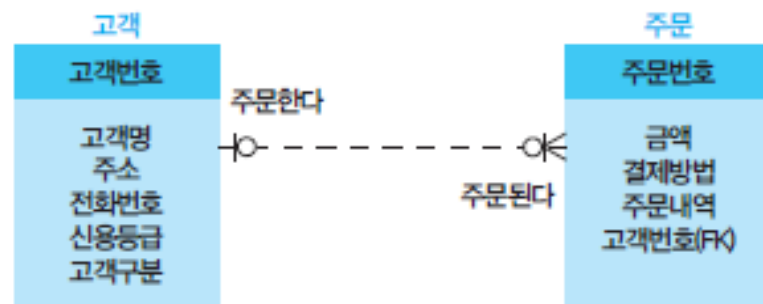
비식별 관계와 식별 관계

비식별 관계

부모 기본키가 자식 일반 속성으로 전이

점선으로 표현

독립적 존재 가능

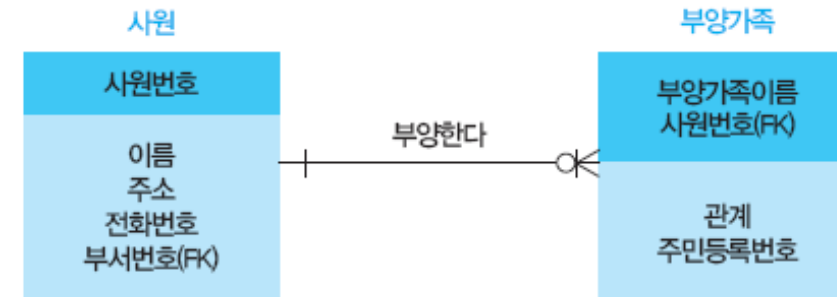


식별 관계

부모 기본키가 자식 기본키로 전이

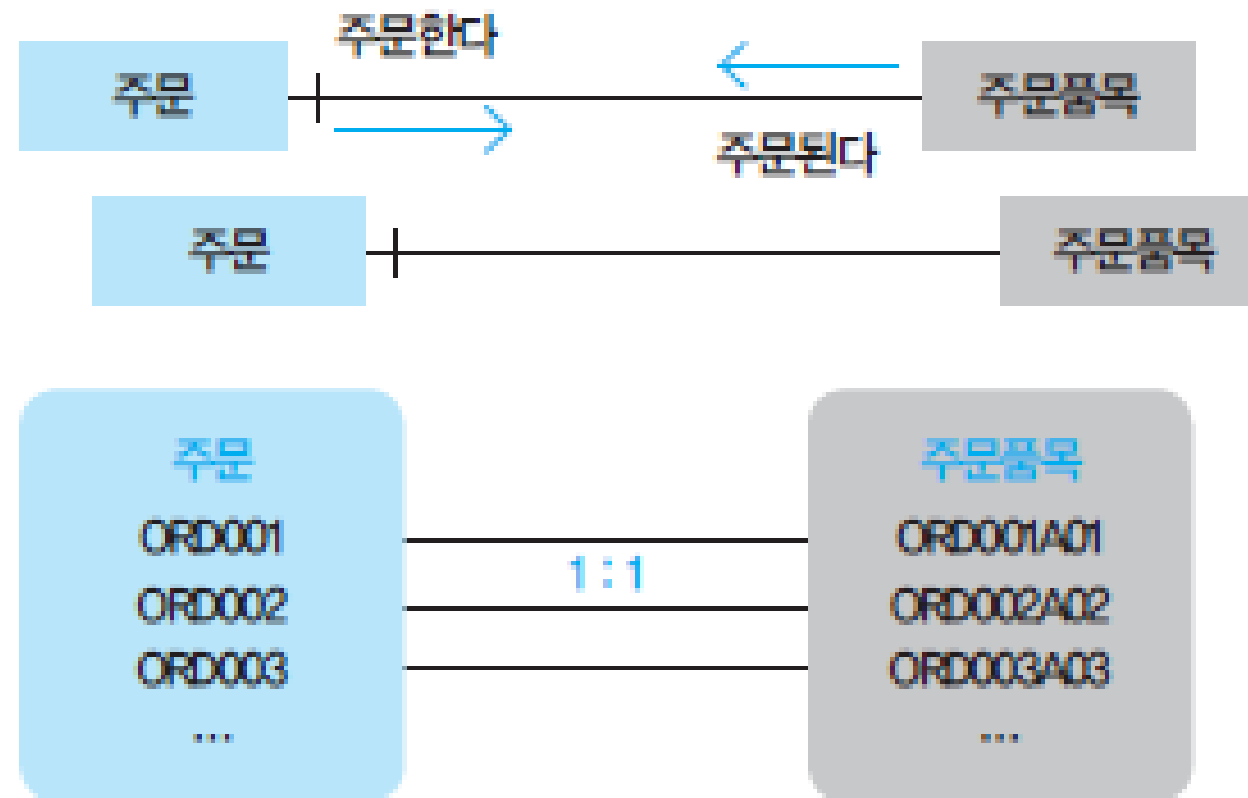
실선으로 표현

존재 종속적



1: 1 관계기수

- 1 1:1 관계는 하나의 엔티티가 다른 하나의 엔티티와만 연결될 수 있는 관계
- 2 예를 들어, 하나의 주문은 하나의 주문품목만을 가질 수 있고, 하나의 주문품목은 하나의 주문에게만 연결



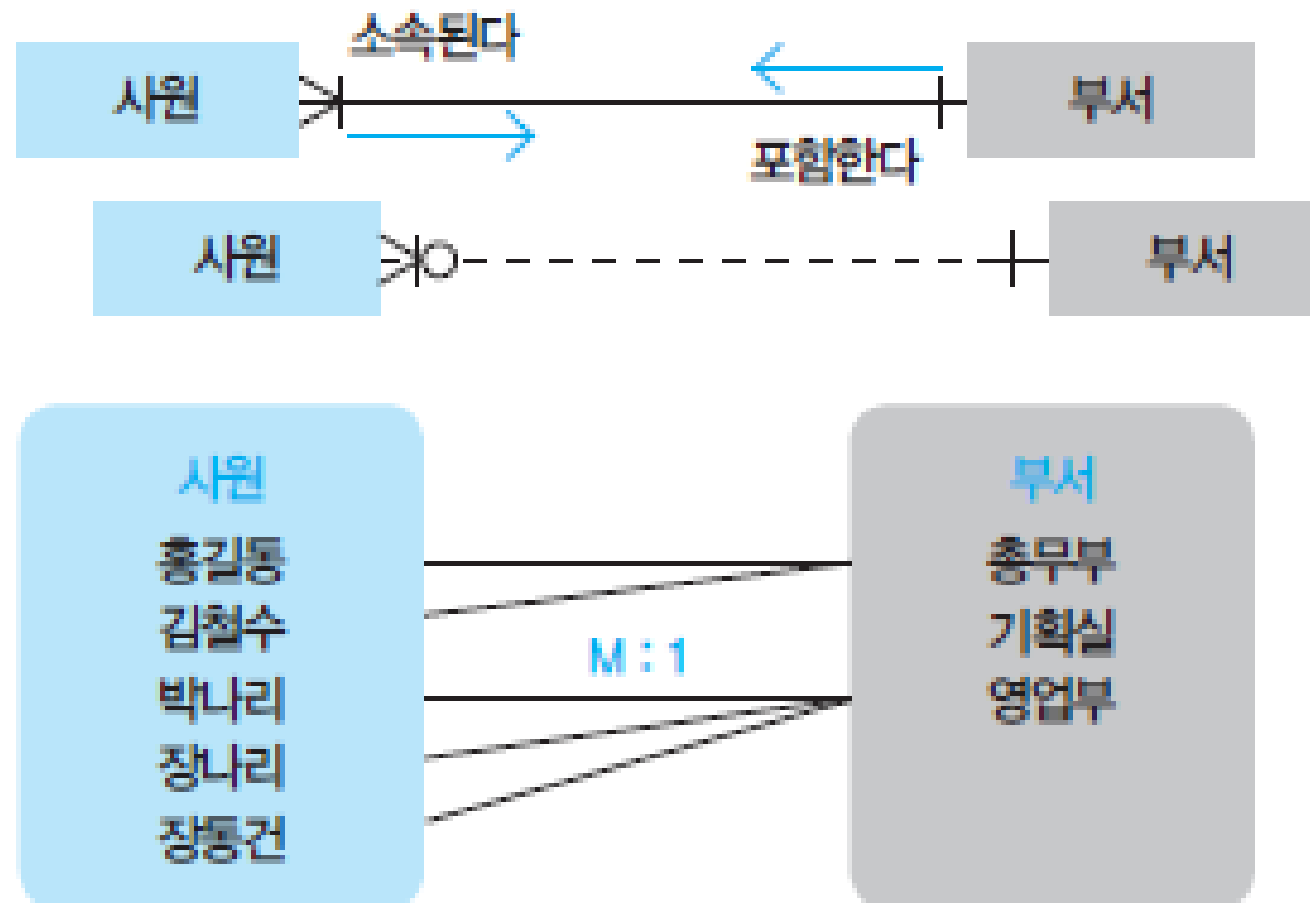
1: M 관계기수

1

1:M 관계는 하나의 엔티티가 여러 개의 엔티티와 연결될 수 있는 관계

2

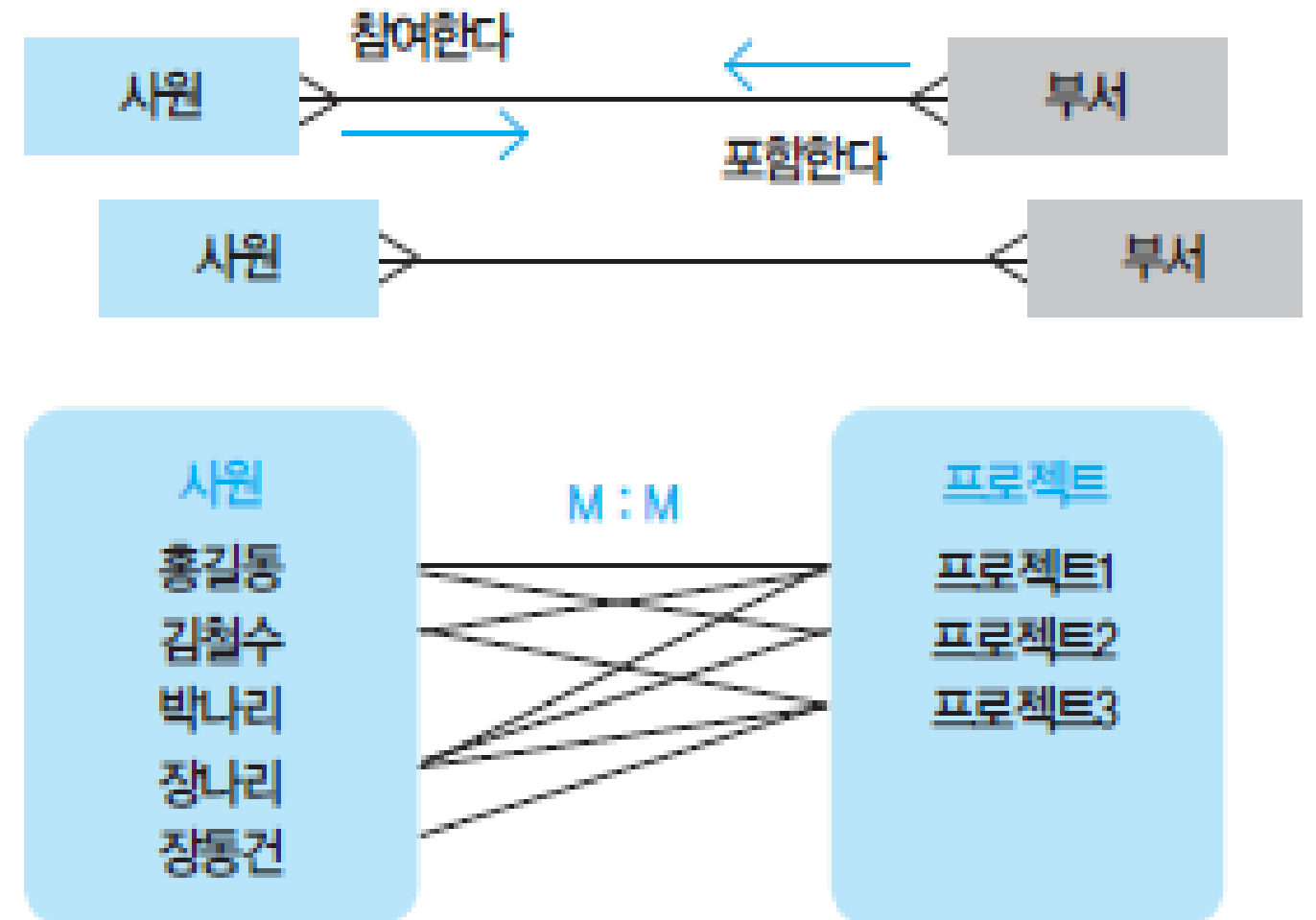
예를 들어, 하나의 부서에는 여러 명의 사원이 포함되지만, 하나의 사원은 하나의 부서에만 소속된다



M:M 관계기수

1 M:M 관계는 여러 개의 엔티티가 여러 개의 엔티티와 연결될 수 있는 관계

2 예를 들어, 여러 명의 사원이 한 프로젝트에 참여하고, 한 사원이 여러 프로젝트에 참여 가능하다



선택/필수 관계



선택/필수 관계

관계에 참여하는 엔티티가 선택적이거나 필수적인지를 나타내는 관계

선택 관계

관계에 참여하는 엔티티가 선택적으로 참여하는 관계

필수 관계

관계에 참여하는 엔티티가 반드시 참여해야 하는 관계

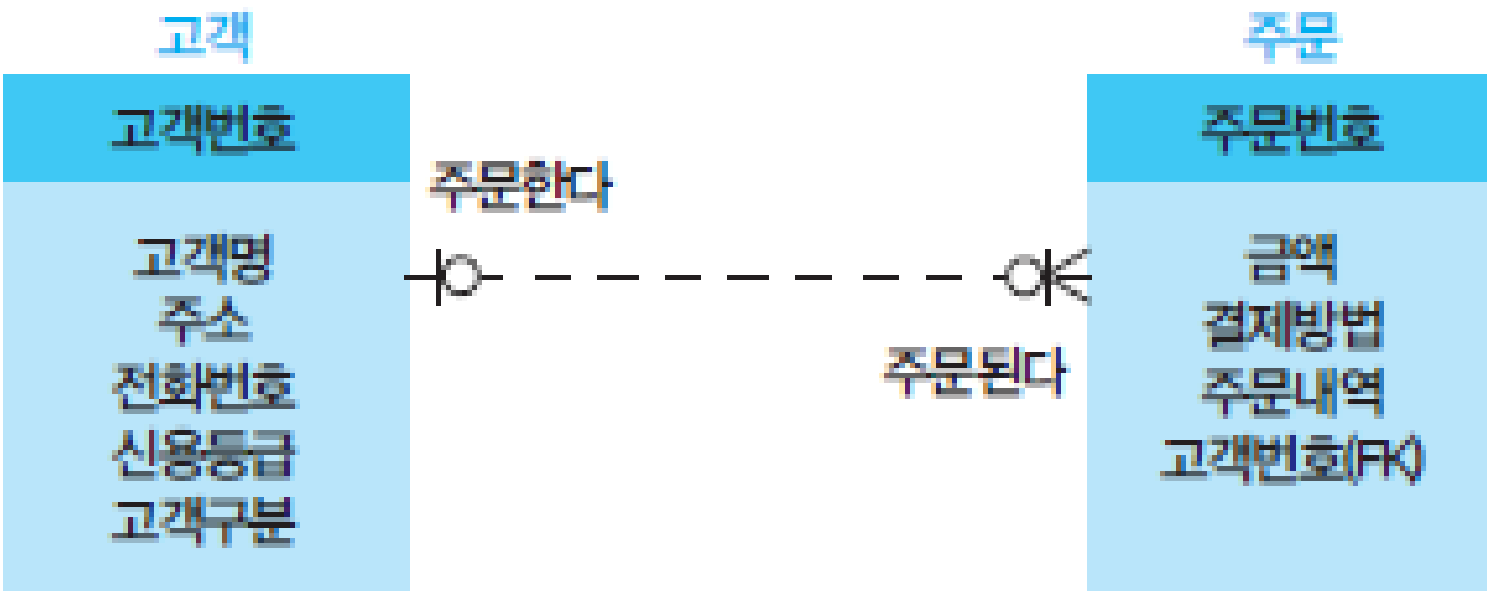
선택/필수 관계(Optional/Mandatory)

선택 관계

예를 들어, 고객은 주문을 선택적으로 할 수 있지만, 주문은 반드시 고객과 연결되어야 합니다.

필수 관계

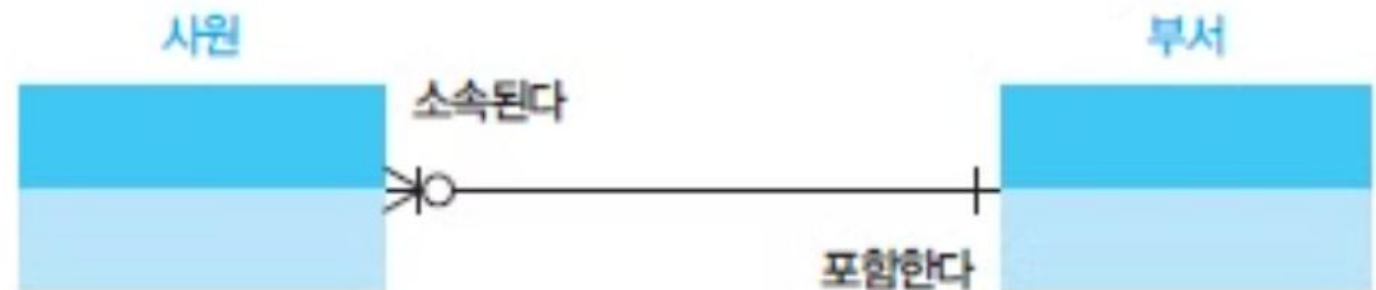
예를 들어, 주문은 반드시 고객과 연결되어야 하며, 고객은 선택적으로 주문을 할 수 있습니다.



순화 관계(Self Relationship)

정상 관계

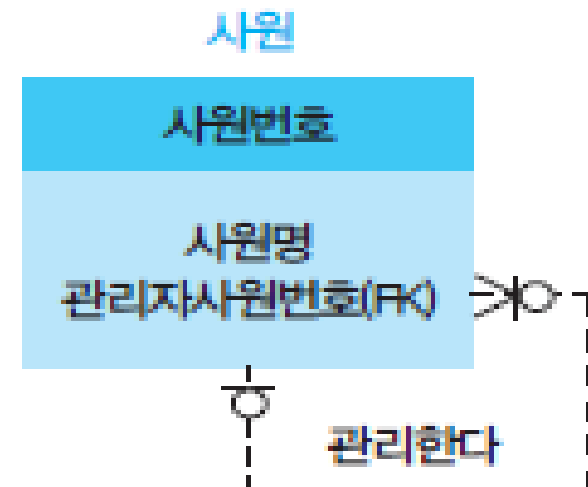
하나의 엔티티타입이 다른 엔티티타입과 관계를 맺는 일반적인 관계



자기 참조 관계

같은 엔티티타입 내의 엔티티 간의 관계

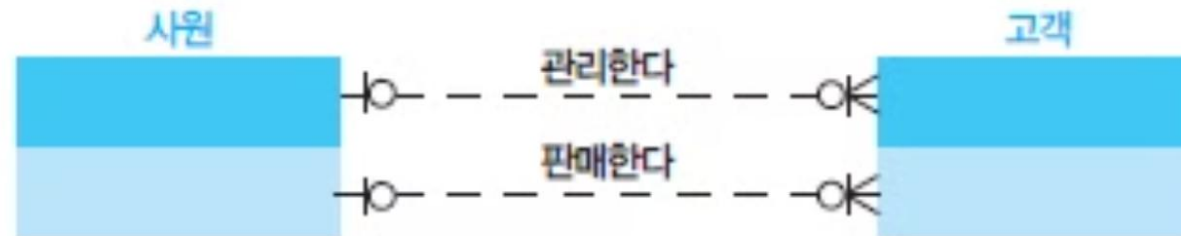
예를 들어, 직원의 관리자를 나타내는 자기 참조 관계로 한 명의 직원은 다른 직원의 관리자가 될 수 있습니다.



병렬 관계(Parallel Relationship)

병렬 관계

두 개의 엔티티타입이 다른 엔티티타입과 동일한 관계를 맺는 관계

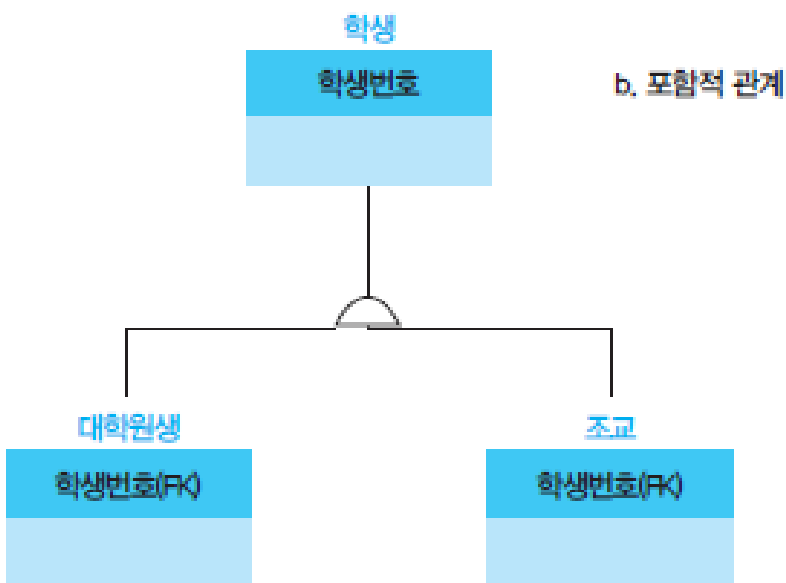


예를 들어, 직원과 고객, 고객과 직원은 각각 별개의 관계를 맺지만, 동일한 고객 엔티티타입과 관련

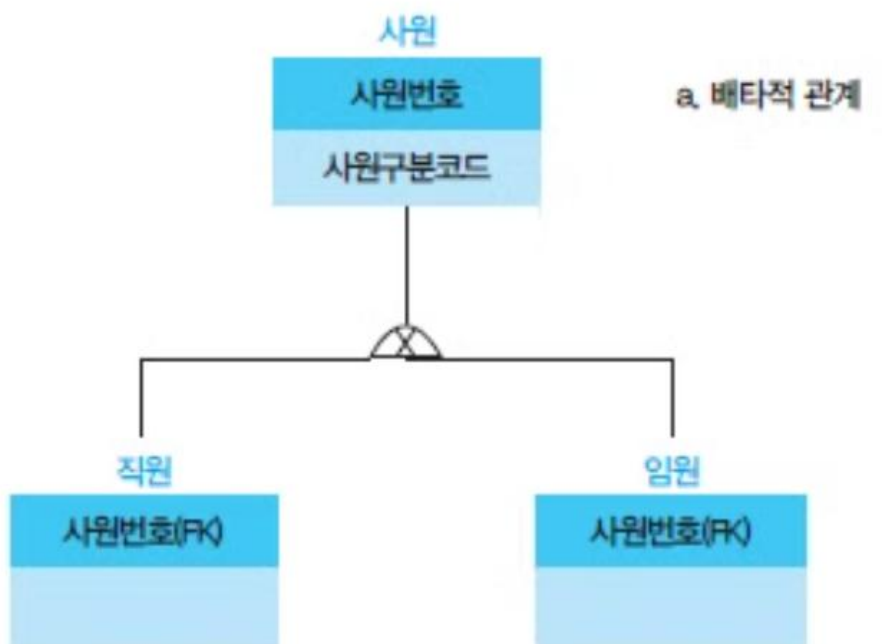
슈퍼-서브타입 관계(Super-Sub Relationship)

슈퍼타입 서브타입 관계

하나의 엔티티타입을 여러 개의 하위 엔티티타입으로 분류하는 관계



학생은 대학원생 또는 조교, 대학원생이면서 조교, 둘 다 아닐수도 있음으로 분류



직원을 직원 아니면 임원으로만 분류

속성(Attribute)

1 속성 (Attribute)

엔티티타입에서 관리하고자 하는 데이터 항목.

예: '고객' 엔티티타입에는 '고객 이름', '주소', '연락처' 등의 속성이 존재할 수 있습니다.

2 속성값 (Attribute Value)

속성에 대한 실제 값. 예를 들어, '김철수'는 '고객 이름' 속성의 속성값입니다.



속성(Attribute)

기본 속성

업무 분석 과정에서 도출된 속성.

예: '주문 날짜', '납기 일자', '수량', '단가'

.

설계 속성

데이터베이스 설계 과정에서 도출된 속성

. 예: '주문 번호', '고객 번호', '주문 상태', '

일련 번호', '품목 코드'.

파생 속성

기존 속성에서 계산 또는 변형하여 도출

된 속성. 예: '주문 총액', '금액'.

[illegible]

속성명 부여

해당 업무에서 사용하는 명칭을 사용하며, 서술식 및 소유격 형태는 피해야 합니다.

속성명 부여

해당 업무에서 사용하는 명칭을 사용하며, 서술식 및 소유격 형태는 피해야 합니다.

약어 사용 금지

명확한 의미 전달을 위해 약어를 사용하지 않고, 엔티티타입 내에서 유일하게 식별 가능하도록 속성명을 부여합니다.

약어 사용 금지

명확한 의미 전달을 위해 약어를 사용하지 않고, 엔티티타입 내에서 유일하게 식별 가능하도록 속성명을 부여합니다.



속성(Attribute)

- 1 한 개의 엔티티타입은 두 개 이상의 엔티티 집합이다.
- 2 한 개의 엔티티는 두 개 이상의 속성을 가진다.
- 3 한 개의 속성은 오직 한 개의 속성 값value만을 가진다.

식별자의 정의

식별자란?

유일하게 인스턴스 구분하는 속성

기본키

엔티티 내에서 대표 식별자

대체키

기본키 외 유일성 가진 후보키

유일성과 최소성

중복값 없고 최소 필드로 구성



식별자(Identifier)

1 식별자

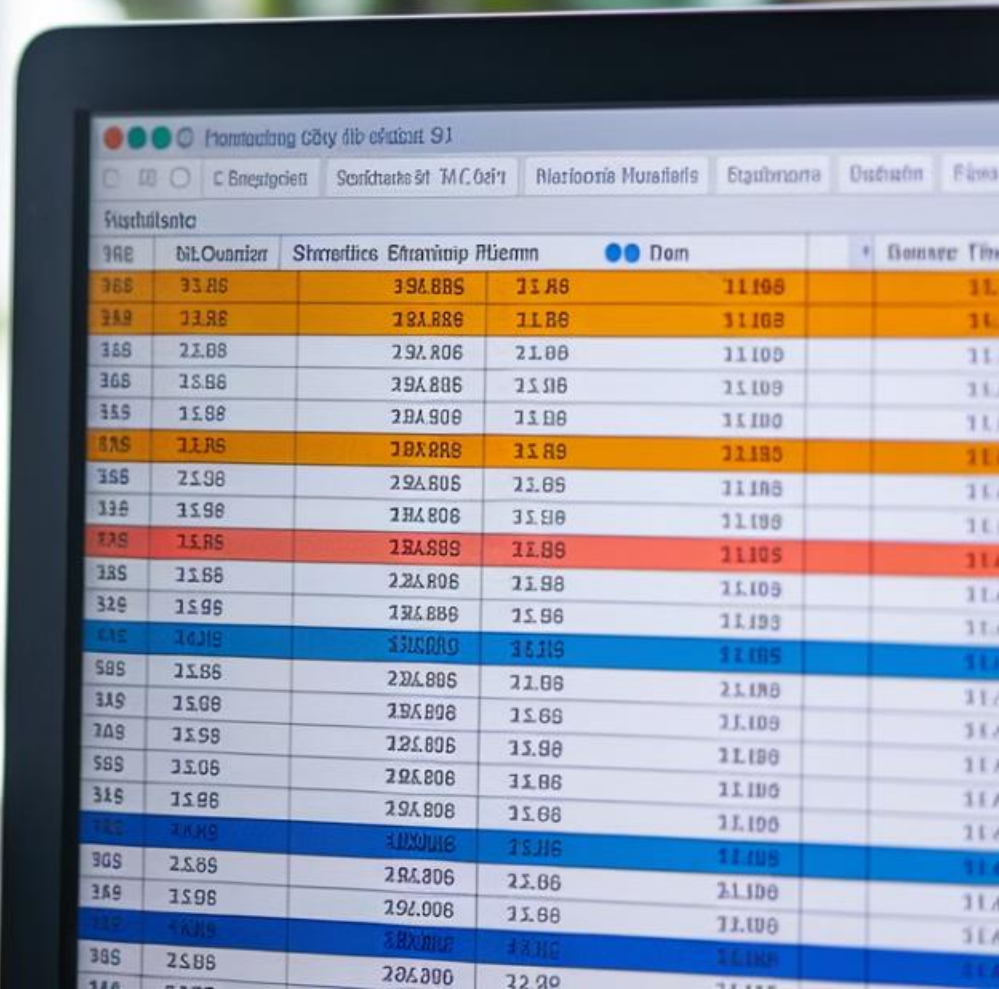
엔티티타입에서 각각의 엔티티를 구분할 수 있는 속성. 예를 들어, '고객' 엔티티타입에서 '주민등록번호'를 식별자로 사용할 수 있습니다.

2 주식별자

엔티티타입 내에서 모든 엔티티를 유일하게 구분할 수 있는 식별자. 하나의 엔티티타입에 하나의 주식별자가 존재합니다.

3 보조 식별자

주식별자를 보완하는 식별자. 하나의 엔티티타입에 두 개 이상의 보조 식별자가 존재할 수 있습니다.



The image shows a laptop screen with a database application. The application has a menu bar at the top with options like 'File', 'Edit', 'View', 'Tools', 'Window', and 'Help'. Below the menu bar is a toolbar with icons for various database operations. The main area displays a table with columns labeled 'ID', 'Name', 'Address', 'Phone', and 'Email'. The table contains several rows of data, with some rows highlighted in orange and others in blue. The data appears to be a list of customers or users.

ID	Name	Address	Phone	Email
100	김민준	서울시 강남구 테헤란로 123	010-1234-5678	kimminjun@gmail.com
101	이서민	부산시 해운대구 해운대로 456	010-9876-5432	leesemin@gmail.com
102	박지현	대구시 중구 중앙대로 789	010-2345-6789	parkjihyun@gmail.com
103	최현우	인천시 남동구 송도대로 101	010-3456-7890	choihyunwoo@gmail.com
104	정민서	광주광역시 서구 봉암대로 202	010-4567-8901	jeongminseo@gmail.com
105	김민준	서울시 강남구 테헤란로 123	010-1234-5678	kimminjun@gmail.com
106	이서민	부산시 해운대구 해운대로 456	010-9876-5432	leesemin@gmail.com
107	박지현	대구시 중구 중앙대로 789	010-2345-6789	parkjihyun@gmail.com
108	최현우	인천시 남동구 송도대로 101	010-3456-7890	choihyunwoo@gmail.com
109	정민서	광주광역시 서구 봉암대로 202	010-4567-8901	jeongminseo@gmail.com
110	김민준	서울시 강남구 테헤란로 123	010-1234-5678	kimminjun@gmail.com

자연식별자 vs 인공식별자



자연식별자

업무에서 발생하는 자연스러운 속성

예: 주민번호, 이메일



인공식별자

시스템에서 생성된 임의의 속성

예: 순번, UUID



선택 기준

변경 가능성, 길이, 보안성 고려

복합 식별자의 사용 시점

다대다 관계
교차 엔티티에서 복합키 활용

이력 관리
시간 속성과 결합해 변경 추적



단일 속성 부족
하나의 속성으로 식별 불가능할 때

계층 구조
상위 개체 식별자가 필요한 경우



식별자(Identifier)

- 유일한 식별자(Unique Identify)는 엔티티를 고유하게 식별하는 데 사용되는 값입니다. 각 엔티티는 시스템 내에서 고유하게 식별될 수 있도록 유일한 식별자를 가지고 있어야 합니다.
- 유일한 식별자는 데이터베이스에서 엔티티를 관리하고, 데이터의 일관성과 무결성을 유지하는 데 중요한 역할을 합니다.
- 유일한 식별자는 데이터베이스 설계에서 데이터의 정확성과 효율성을 보장하는 데 중요한 요소입니다.

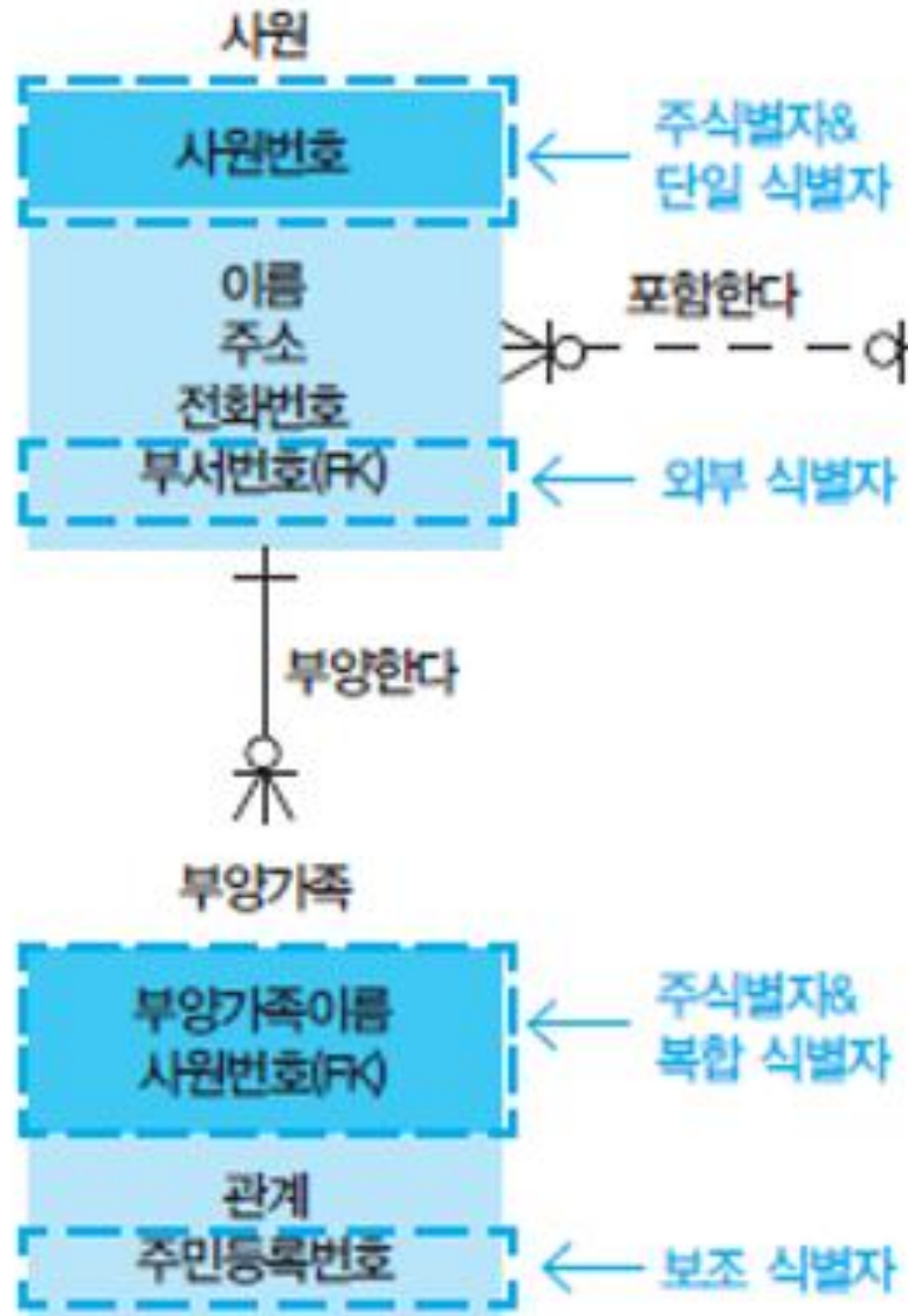
주식별자(Identifier)

주식별자 선택 가이드

최소의 속성으로 구성, 보안성이 요구되지 않고 업무의 활용도가 높은 속성, 최소의 저장 공간을 차지하는 속성으로 선택합니다

업무에서 자주 검색되는 순서대로 지정하며, 분포도가 좋은 속성 순으로 배열합니다. 상속받은 속성은 이전의 속성과 동일하게 배열합니다.

테이블로 변환할 경우 가급적 CHAR 타입을 사용하지 말아야 합니다. 검색 조건으로 사용하기 위해 추출 속성을 넣어서는 안 됩니다.



개념적 설계 실습 : 학과 시스템

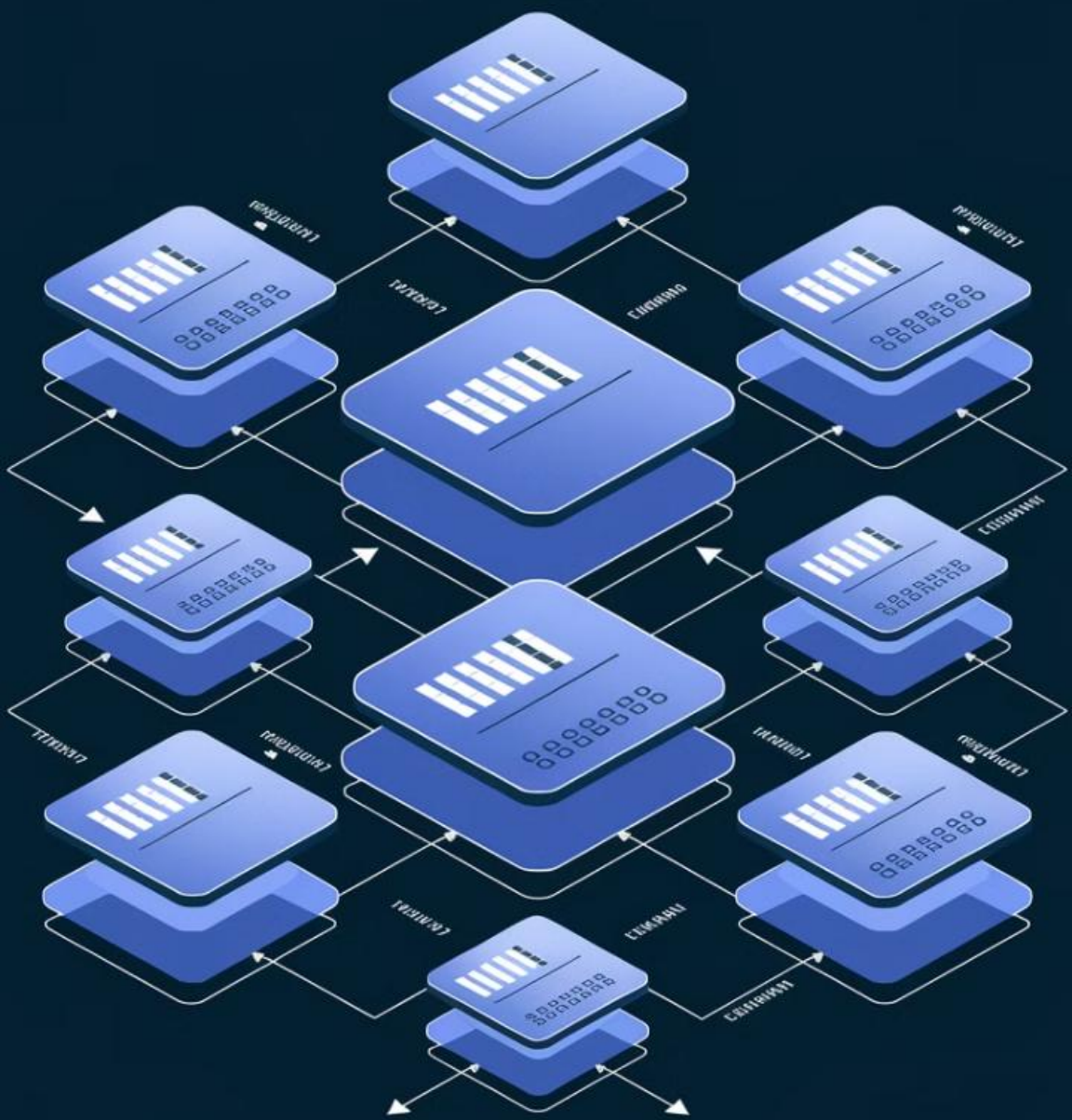
1. 엔티티 도출
2. 관계설정
3. 식별자 도출 및 추가 속성 도출

개념적 설계 실습 : e-마켓플레이스

1. 엔티티 도출
2. 관계설정
3. 식별자 도출 및 추가 속성 도출

개념적 설계 실습 : e-마켓플레이스

1. 엔티티 도출
2. 관계설정
3. 식별자 도출 및 추가 속성 도출



논리적 설계

이번 장에서는 논리적 설계의 다양한 측면들을 살펴보고 실제 데이터베이스 모델링에 적용할 수 있는 실용적인 방법들을 학습합니다.

논리 데이터 모델을 전개하는 과정을 단계별로 이해하고, 데이터 무결성, 정규화, 데이터 모델 통합 등 중요한 개념들을 깊이 있게 다룹니다.

마지막으로 개념적 설계 단계에서 도출한 ERD를 테이블로 변환하는 실습을 통해 이론을 실제로 적용해 보는 시간을 가질 예정입니다.

데이터베이스 모델



계층형 데이터베이스 (Hierarchical Data Base)

계층형 데이터베이스는 데이터를 계층 구조로 표현하는 모델입니다. 트리 구조와 유사하게 상위 레벨 엔티티에서 하위 레벨 엔티티로 연결되는 관계를 가지며, 하나의 자식 엔티티는 오직 하나의 부모 엔티티만 가질 수 있습니다. 이러한 구조는 조직 계층, 제품 카탈로그와 같이 계층적인 데이터 표현에 적합합니다. 1:N 관계를 표현하는 데 유용하지만, M:N 관계를 표현하는 데 어려움을 겪습니다.

데이터베이스 모델

네트워크 데이터베이스 (Network Data Base)

네트워크 데이터베이스는 계층형 데이터베이스의 제한적인 관계 표현을 보완하기 위해 등장한 모델입니다. 계층형 데이터베이스와 마찬가지로 데이터를 계층 구조로 표현하지만, 하나의 자식 엔티티가 여러 부모 엔티티를 가질 수 있도록 관계를 확장했습니다. M:N 관계를 표현할 수 있는 유연성을 제공하지만, 복잡한 관계 구조로 인해 관리 및 이해가 어려울 수 있습니다.

네트워크 데이터베이스 (Network Data Base)

네트워크 데이터베이스는 데이터베이스를 세 개의 스키마, 서버 스키마, 스키마, 저장 스키마로 구분합니다. 데이터 언어 DDL, DML을 사용하며, 처음으로 DBA(데이터베이스 관리자)의 기능과 역할을 정렬하고 중요성을 기술했습니다.

데이터베이스 모델

관계형 데이터베이스 (Relational Data Base)

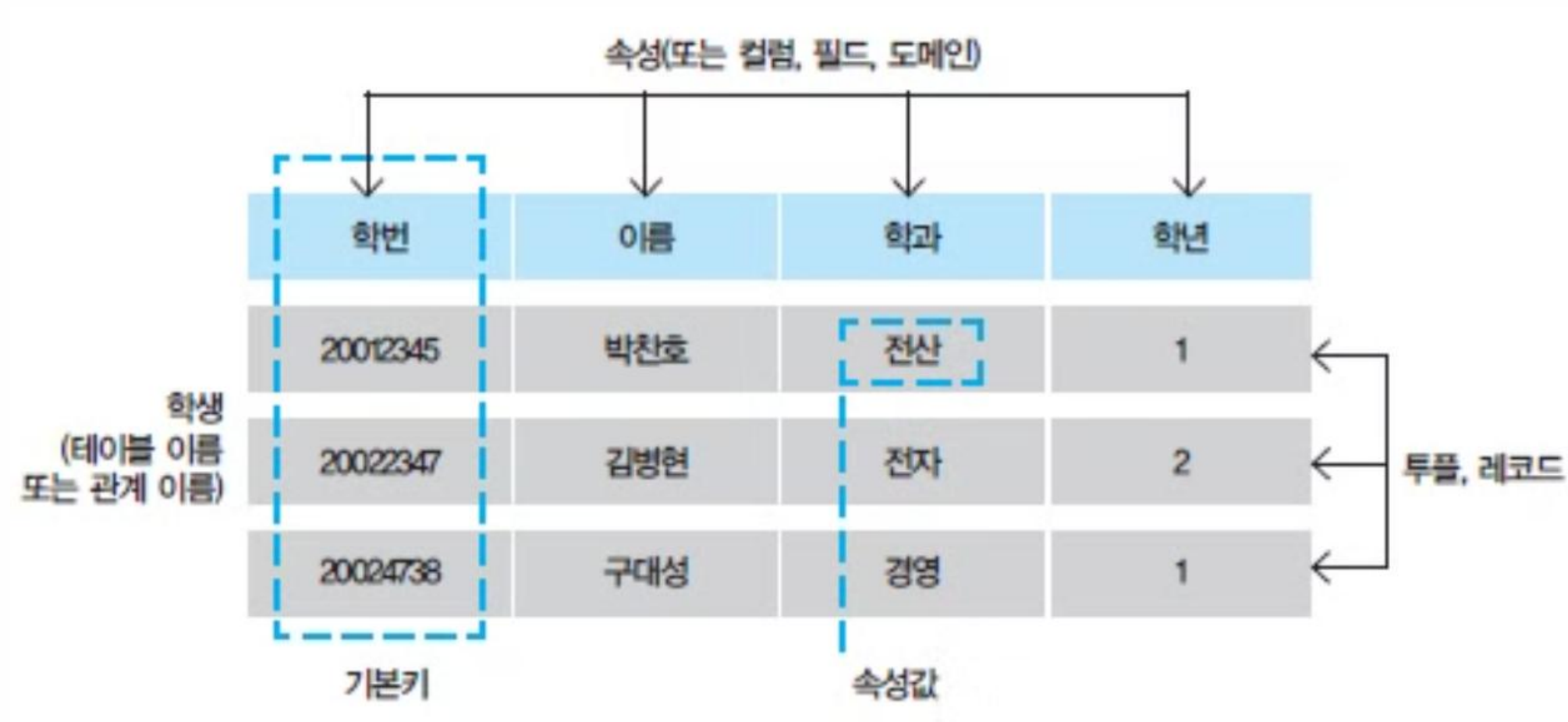
관계형 데이터베이스는 1970년 코드 E.F. Codd에 의해 소개된 모델입니다.

엔티티의 모든 데이터와 데이터 사이의 관계를 2차원의 테이블 형태로 기술합니다.

테이블은 행(Row)과 열(Column)로 구성되며, 각 행은 엔티티를 나타내고, 각 열은 엔티티의 속성을 나타냅니다.

관계는 테이블 간의 연결을 통해 표현되며, 주키(Primary Key)와 외래키(Foreign Key)를 사용하여 관계를 정의합니다.

데이터베이스 모델



관계형 데이터베이스 (Relational Data Base)

관계형 데이터베이스는 데이터 무결성, 데이터 독립성, 데이터 보안, 데이터 일관성 등 여러 장점을 제공합니다. 또한 SQL(Structured Query Language)을 통해 데이터를 쉽게 관리하고 질의할 수 있습니다. 이러한 장점들로 인해 관계형 데이터베이스는 현재 가장 널리 사용되는 데이터베이스 모델입니다.

데이터베이스 모델

데이터베이스 구축 과정

데이터베이스 구축은 다음과 같은 단계를 거쳐 진행됩니다.

요구사항 분석 단계에서는 사용자의 요구사항을 수집하고, 분석하여 데이터 모델링의 기반을 마련합니다.

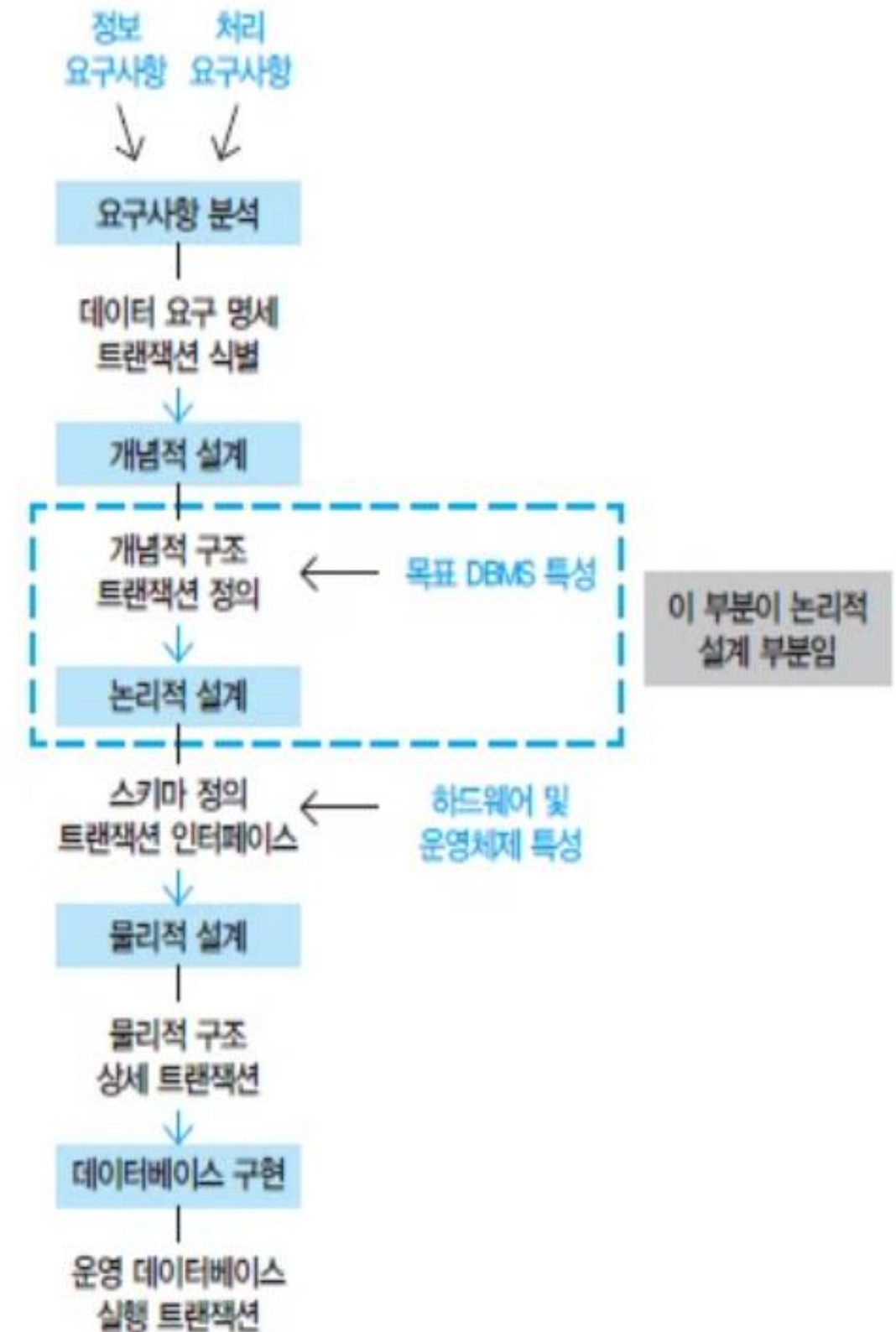
개념적 설계 단계에서는 엔티티와 관계를 정의하여 ERD(Entity-Relationship Diagram)를 생성합니다.

논리적 설계 단계에서는 ERD를 테이블로 변환하고, 데이터 무결성 규칙을 적용하여 데이터베이스 구조를 구체화합니다.

물리적 설계 단계에서는 데이터베이스를 저장할 하드웨어 및 소프트웨어를 결정하고, 테이블, 인덱스, 저장 프로시저 등을 구현합니다.

구현 단계에서는 설계된 데이터베이스를 실제로 구축하고, 데이터를 입력 및 관리할 수 있도록 환경을 설정합니다.

마지막으로 테스트 단계에서는 구축된 데이터베이스의 기능을 검증하고, 오류를 수정하여 사용자에게 제공할 준비를 합니다.



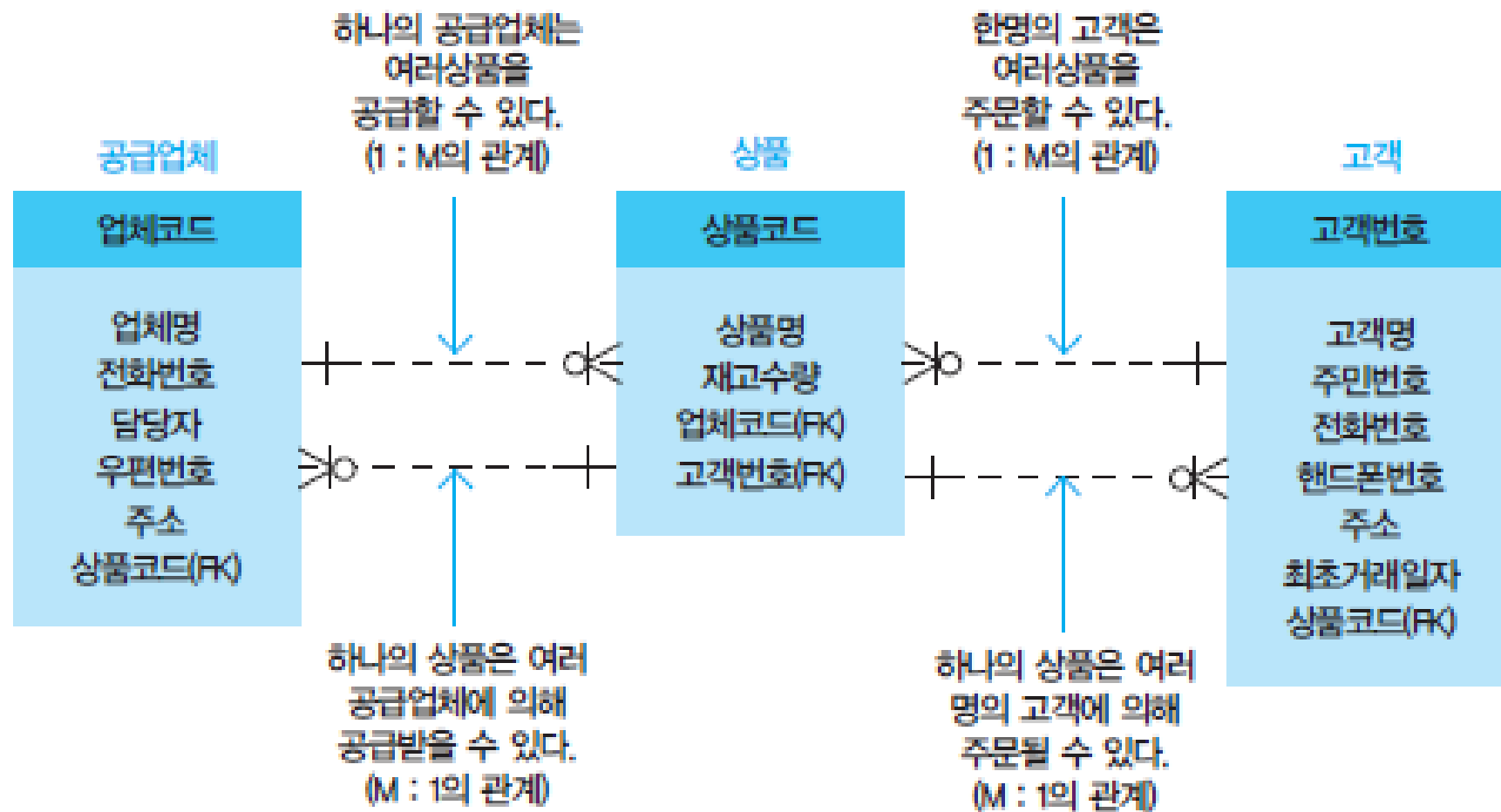
논리 데이터 모델 전개

M:M 관계 해소

M:M 관계를 해소하는 방법은 크게 세 가지가 있습니다.

첫 번째 방법은 관계를 새로운 엔티티타입으로 추가 생성하는 것입니다.

새로 생성된 엔티티타입은 기존의 두 엔티티타입의 주식별자를 키 속성으로 가지며, M:M 관계를 1:N 관계로 변환합니다.

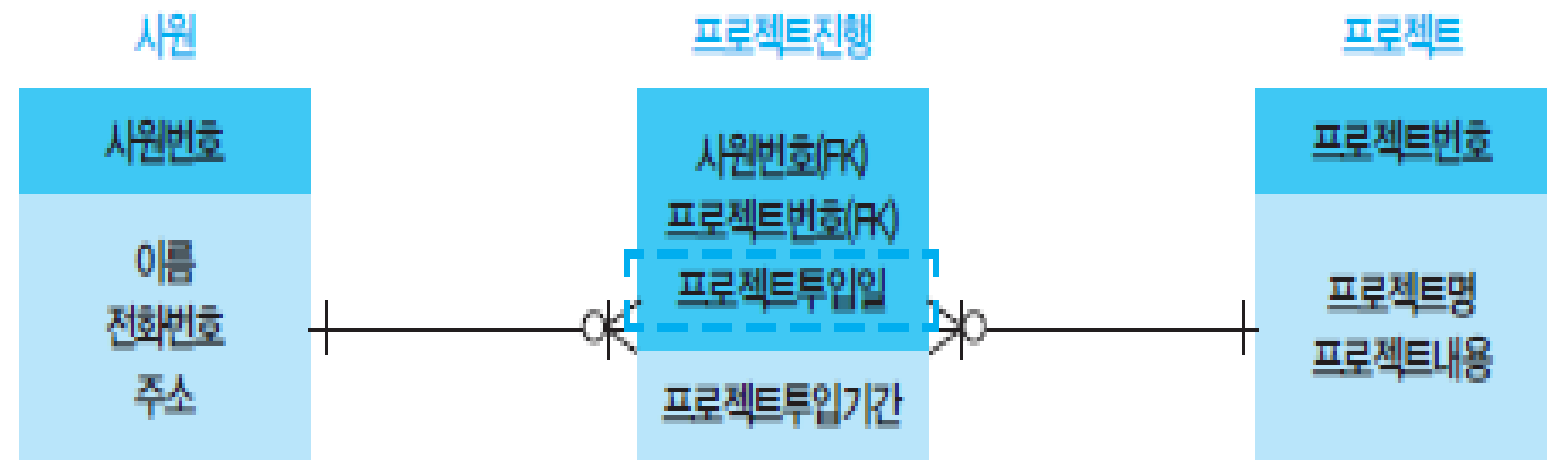
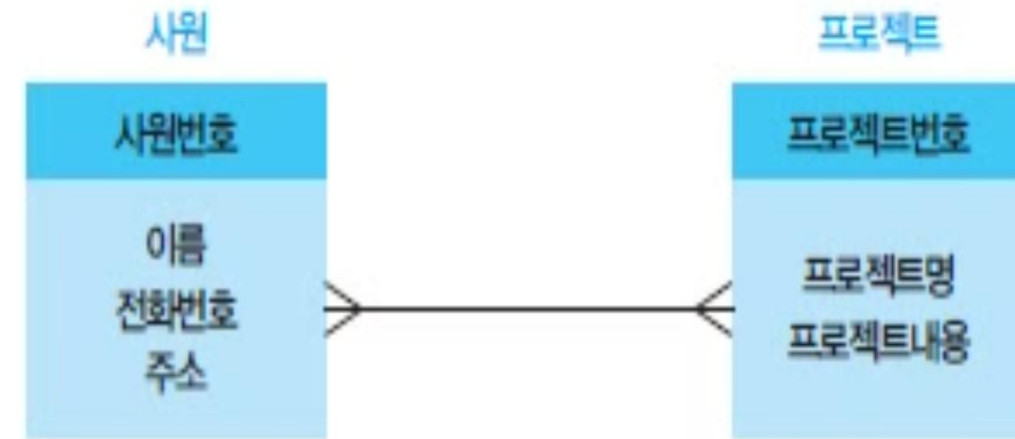


논리 데이터 모델 전개

M:M 관계 해소 - 기본 방법

새로 생성된 관계의 엔티티타입의 키 속성에 낱짜를 추가하는 방법은 M:M 관계를 1:N 관계로 변환하는데 유용한 방법입니다.

낱짜를 키 속성으로 포함하면, 두 엔티티 간의 관계가 언제 발생했는지 정확하게 추적할 수 있습니다.



논리 데이터 모델 전개

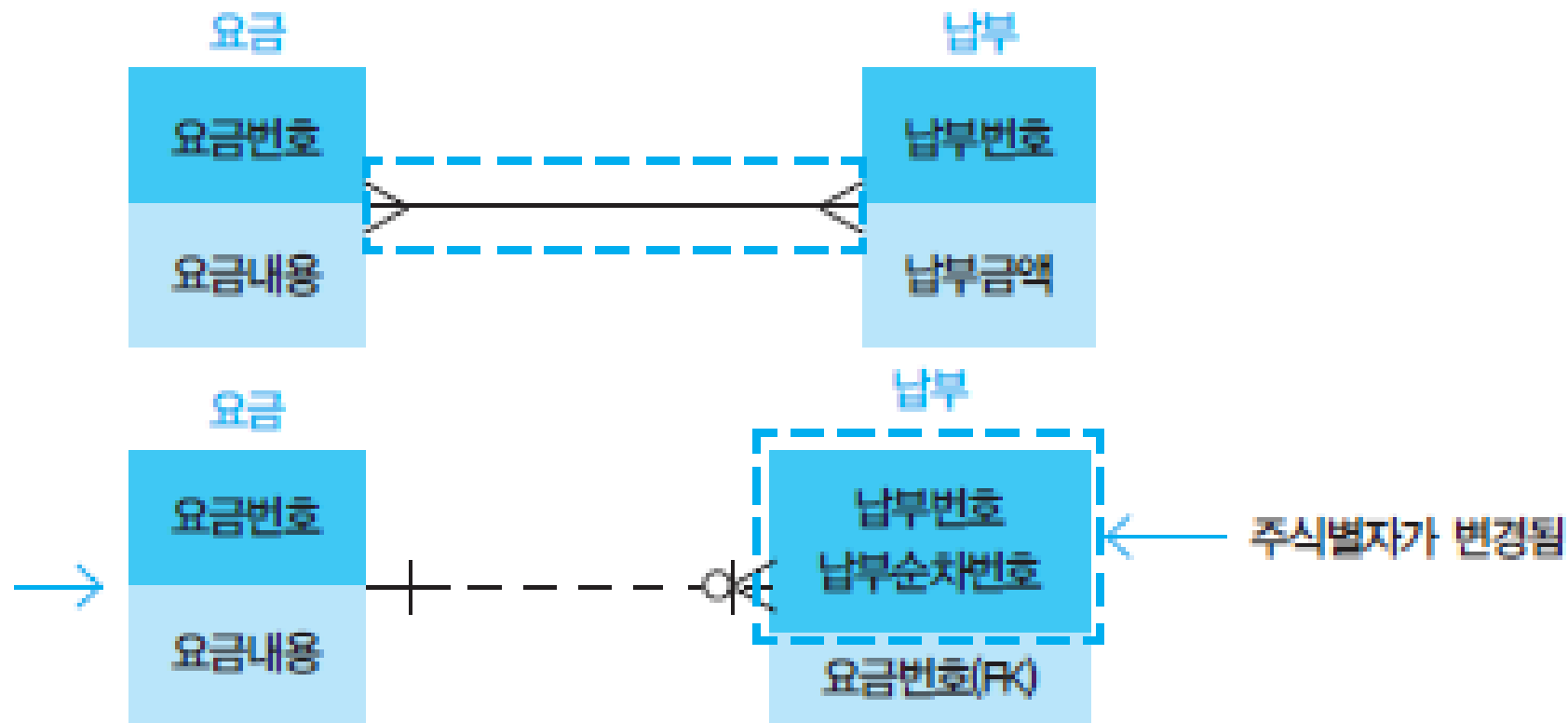
M:M 관계 해소

두 번째 방법은 기존의 엔티티타입에 주식별자를 통합하는 것입니다.

이 방법은 한 엔티티타입을 다른 엔티티타입에 포함시켜 M:M 관계를 1:N 관계로 변환합니다.

예를 들어, 요금과 납부의 M:M 관계를 해소하기 위해 요금테이블에 납부좌 정보를 포함시킬 수 있습니다.

하지만 이 경우 데이터 중복이 발생할 수 있으며, 데이터 관리 및 업데이트가 복잡해질 수 있습니다.



논리 데이터 모델 전개

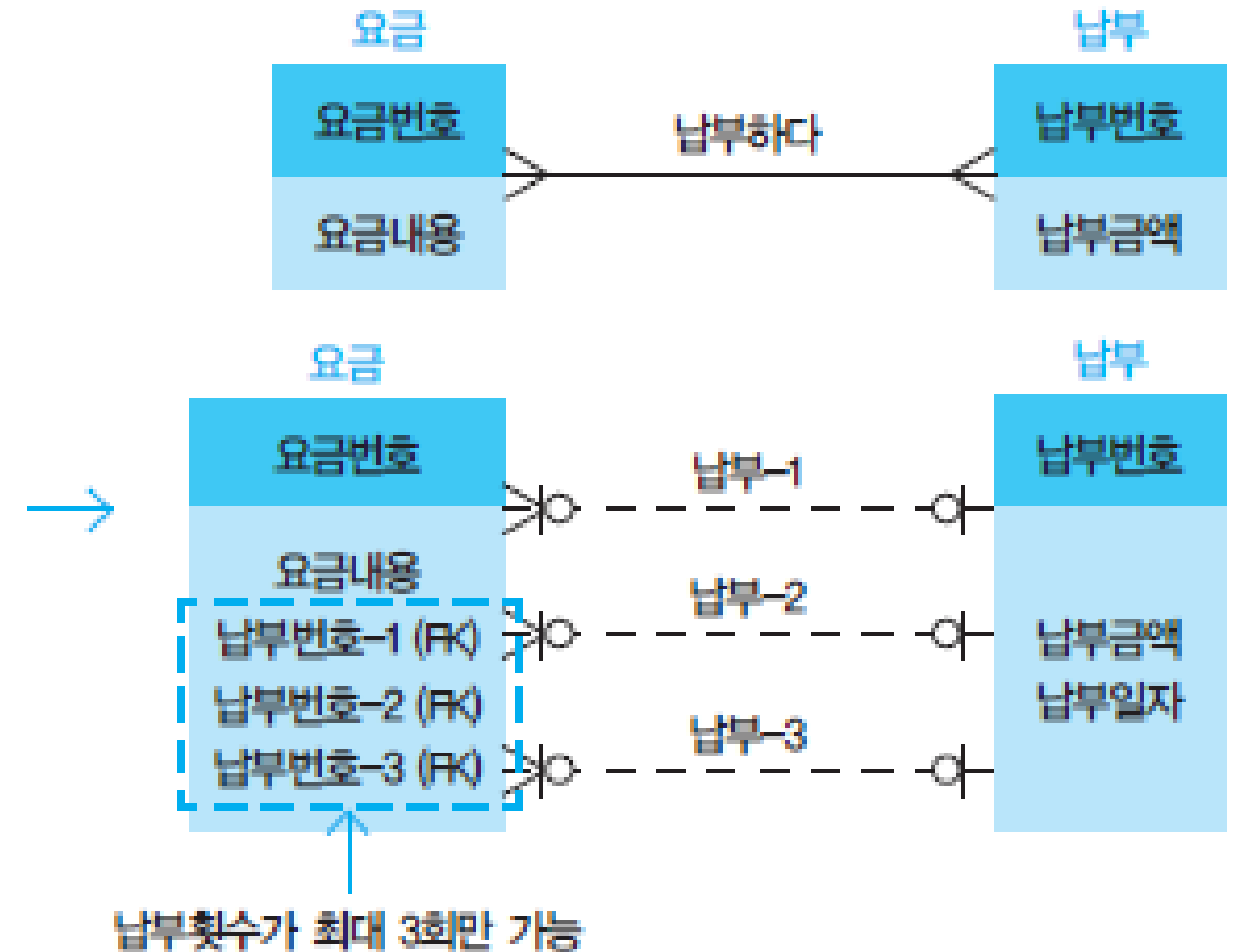
M:M 관계 해소 - M:M 관계 해소

세 번째 방법은 부모 엔티티타입에 속성을 배열로 추가하는 것입니다.

이 방법은 부모 엔티티타입에 자식 엔티티타입의 정보를 배열 형태로 포함시켜 M:M 관계를 1:N 관계로 변환합니다.

예를 들어, 학생 테이블에 여러 강좌 정보를 배열 형태로 추가할 수 있습니다.

이 방법은 데이터 중복을 최소화할 수 있지만, 데이터 관리 및 업데이트가 복잡해질 수 있습니다.



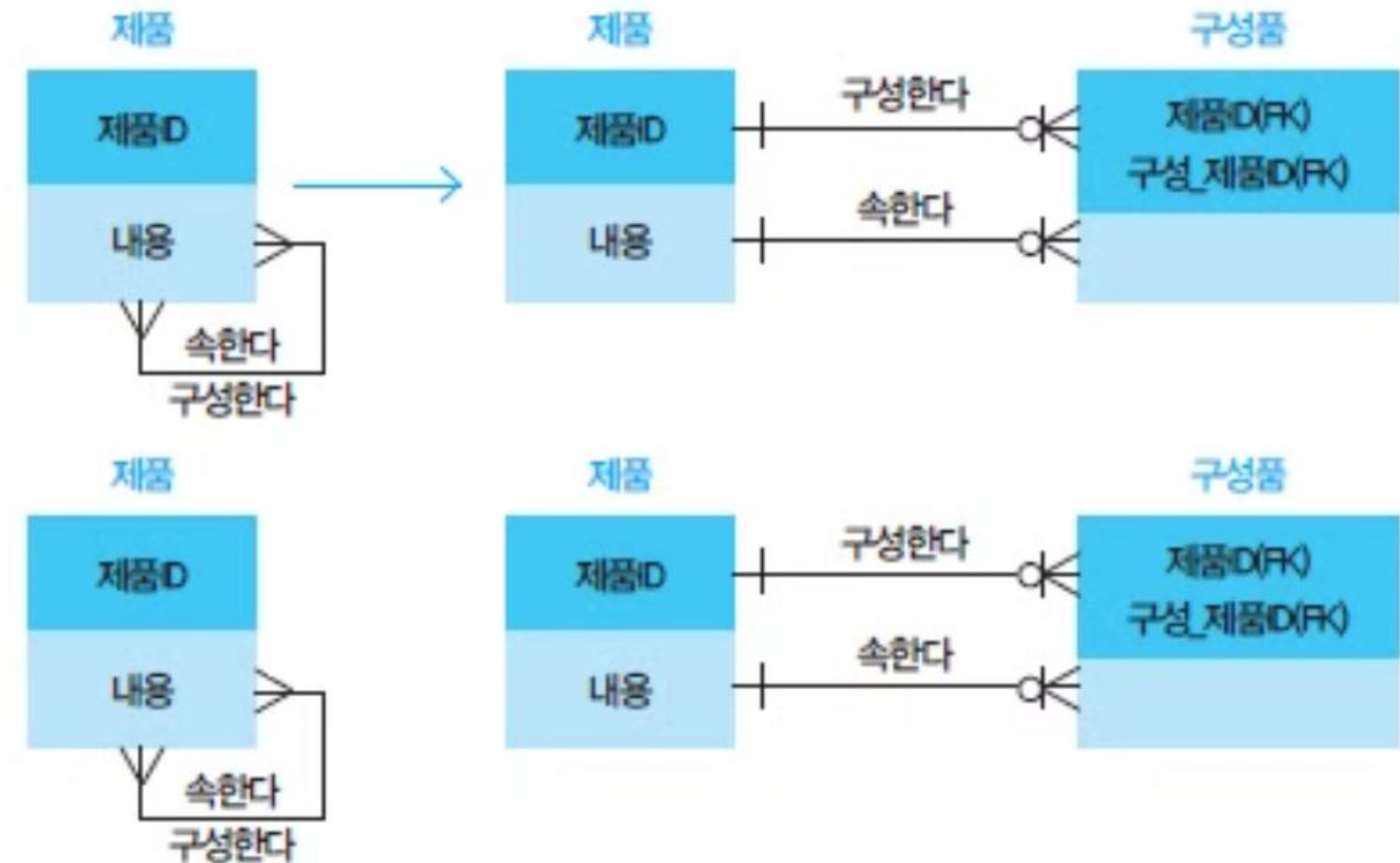
논리 데이터 모델 전개

순환구조 엔티티타입의 해결 (BOM(Bill Of Material))

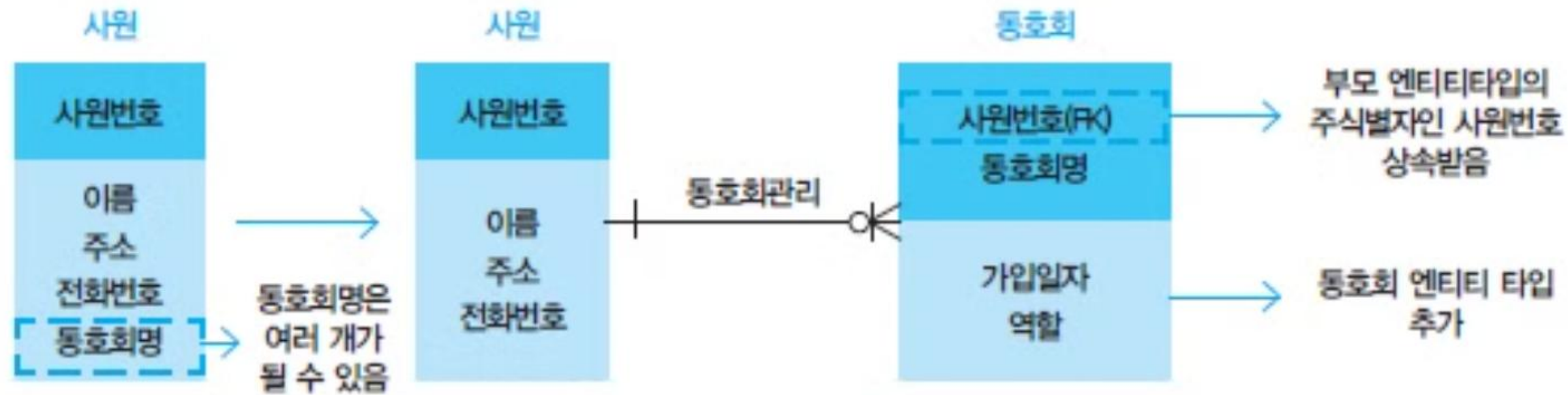
BOM(Bill Of Material)은 제품의 구성 요소를 계층적으로 나타내는 목록입니다.

예를 들어, 자동차의 BOM은 자동차의 전체 구성 요소를 계층적으로 나열하고, 각 구성 요소의 하위 구성 요소를 보여줍니다.

이러한 BOM에서 각 구성 요소는 엔티티 타입으로 표현되고, 구성 요소 간의 관계는 1:N 관계로 나타낼 수 있습니다.



다중 속성 해결



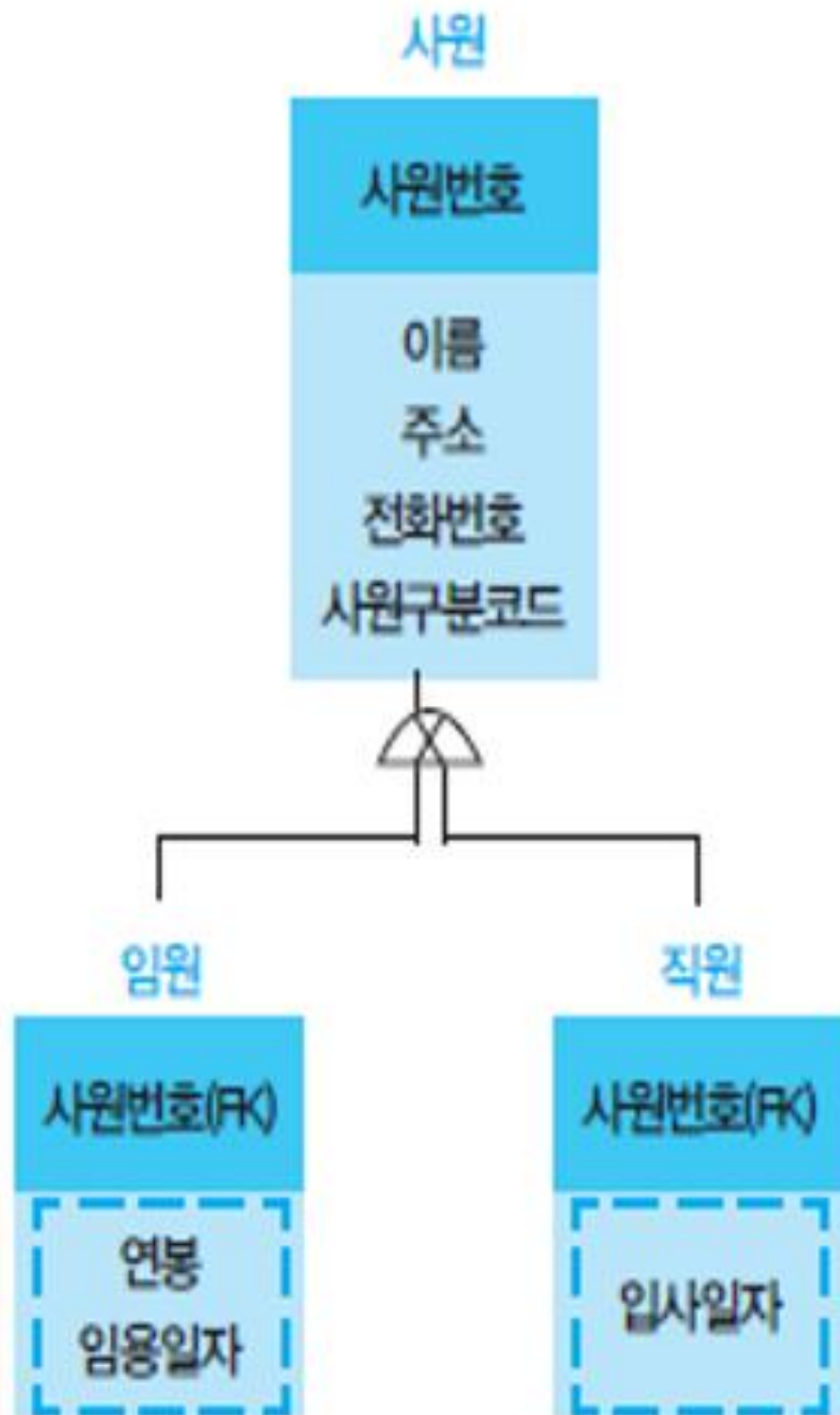
다치 값 속성

하나의 속성에 여러 값을 가질 수 있는 경우, 각 값을 별도의 속성으로 분리합니다.

예를 들어, '취미' 속성에 여러 개의 취미를 저장해야 한다면, '취미1', '취미2', '취미3'과 같이 여러 속성을 사용합니다.

새로운 엔티티타입 생성

다치 값 속성이 여러 엔티티타입에 공통적으로 적용되는 경우, 새로운 엔티티타입을 생성하여 다치 값을 관리할 수 있습니다. 이를 통해 데이터 중복을 줄이고 데이터 무결성을 유지할 수 있습니다.

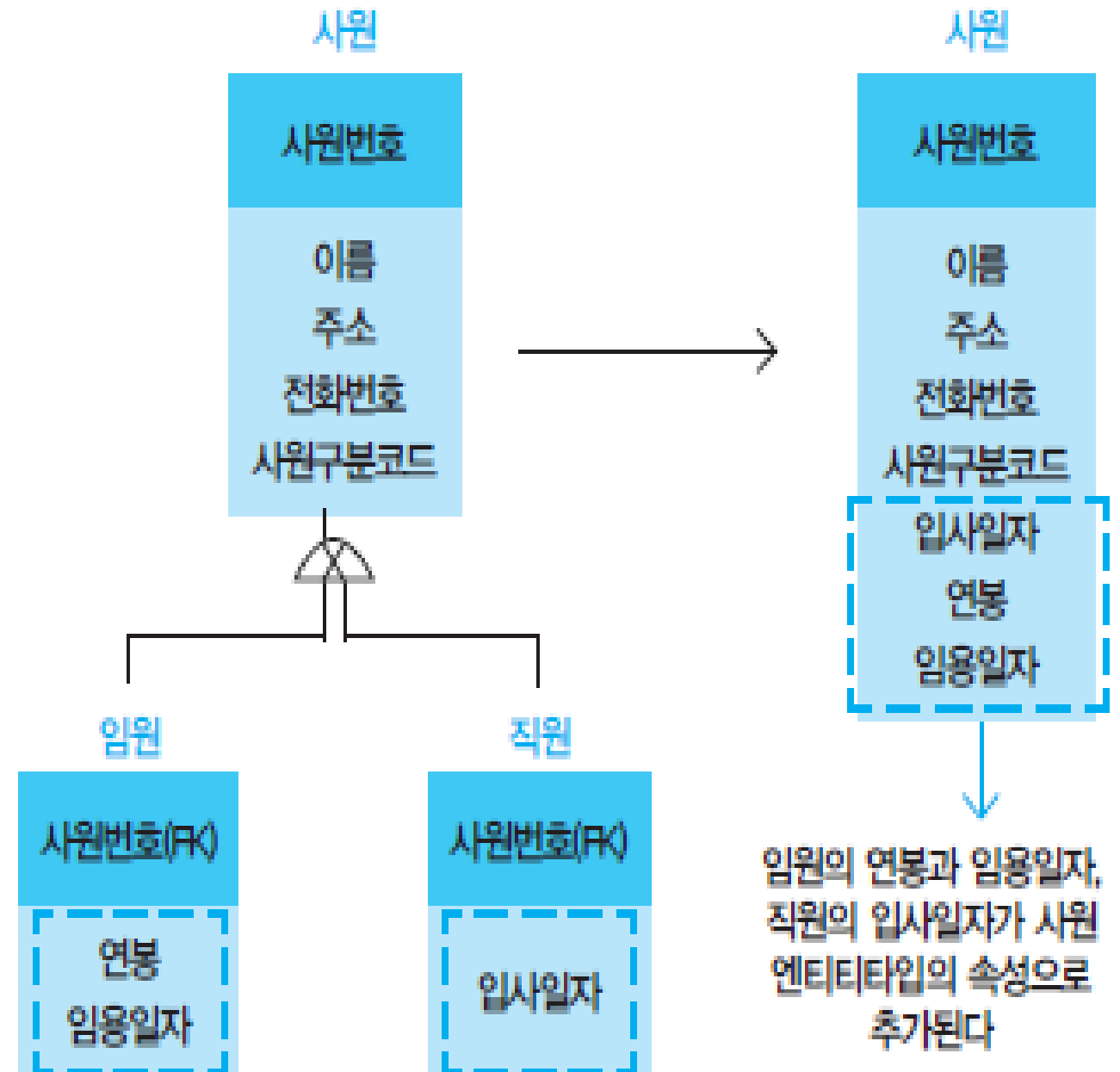


슈퍼-서브 엔티티타입 해결

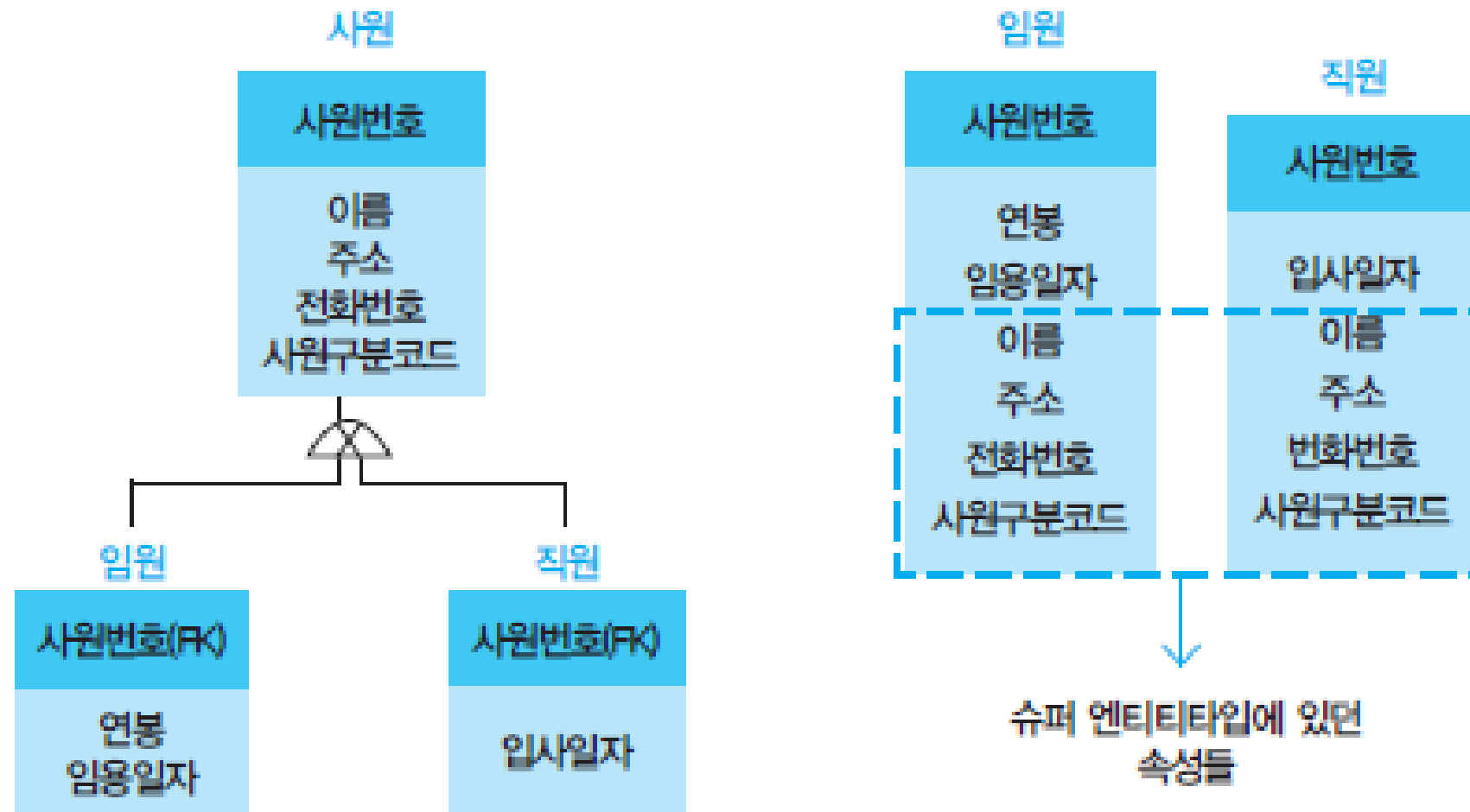
- 슈퍼 엔티티타입은 서브 엔티티타입의 공통 속성을 포함하는 엔티티타입입니다.
- 서브 엔티티타입은 슈퍼 엔티티타입의 특정 유형을 나타냅니다.
- 슈퍼-서브 엔티티타입 관계는 상속과 유사합니다.

슈퍼 엔티티타입으로 통합

- 1 슈퍼 엔티티타입을 기준으로 서브 엔티티타입들을 통합합니다.
예를 들어, 직원 엔티티 타입이 정규직과 계약직으로 나뉘는 경우, 직원을 슈퍼 엔티티 타입으로 설정하고, 정규직과 계약직을 서브 엔티티 타입으로 설정하여 하나의 ERD로 표현합니다.



서브 엔티티타입으로 통합



서브 엔티티 타입을 기준으로 슈퍼 엔티티 타입을 통합합니다.

슈퍼 엔티티 타입에 서브 엔티티 타입에서 공통적으로 존재하는 속성만 포함시키고, 각 서브 엔티티 타입에 해당 서브 타입의 고유 속성을 추가합니다.

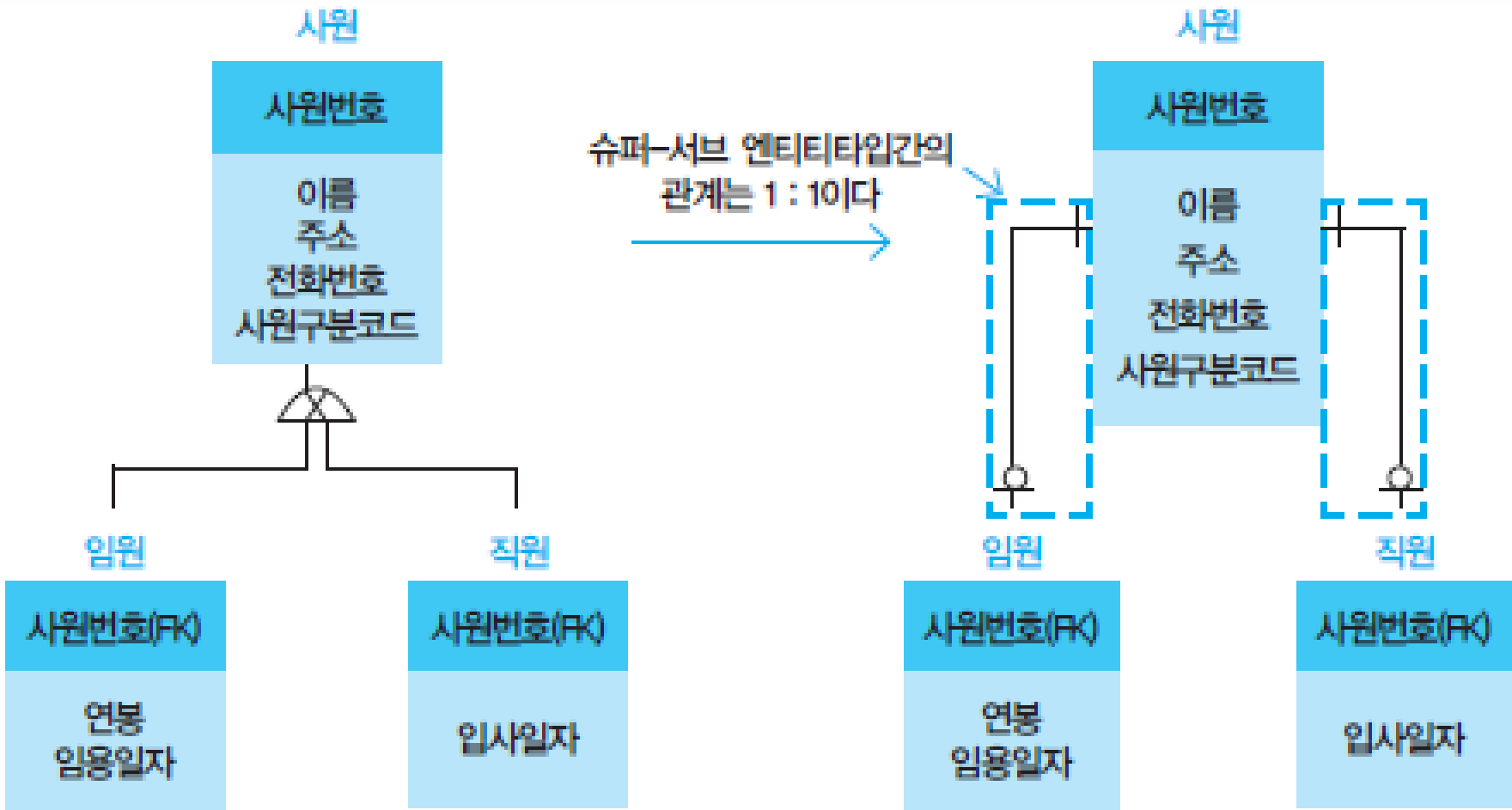
슈퍼-서브 엔티티타입 분리

슈퍼 엔티티타입

슈퍼 엔티티 타입을 별도로 유지하면서, 서브 엔티티 타입과의 관계를 'ISA' 관계로 표현합니다.

서브 엔티티타입

서브 엔티티 타입을 별도로 유지하면서, 슈퍼 엔티티 타입과의 관계를 'ISA' 관계로 표현합니다.



컴퓨터

비품번호

기종

CPU 종류

HDD 용량

구매일자

이력 관리 엔티티타입



이력 정보를 별도의 엔티티타입으로 관리합니다.



변경된 내용을 기록하고 추적할 수 있습니다.



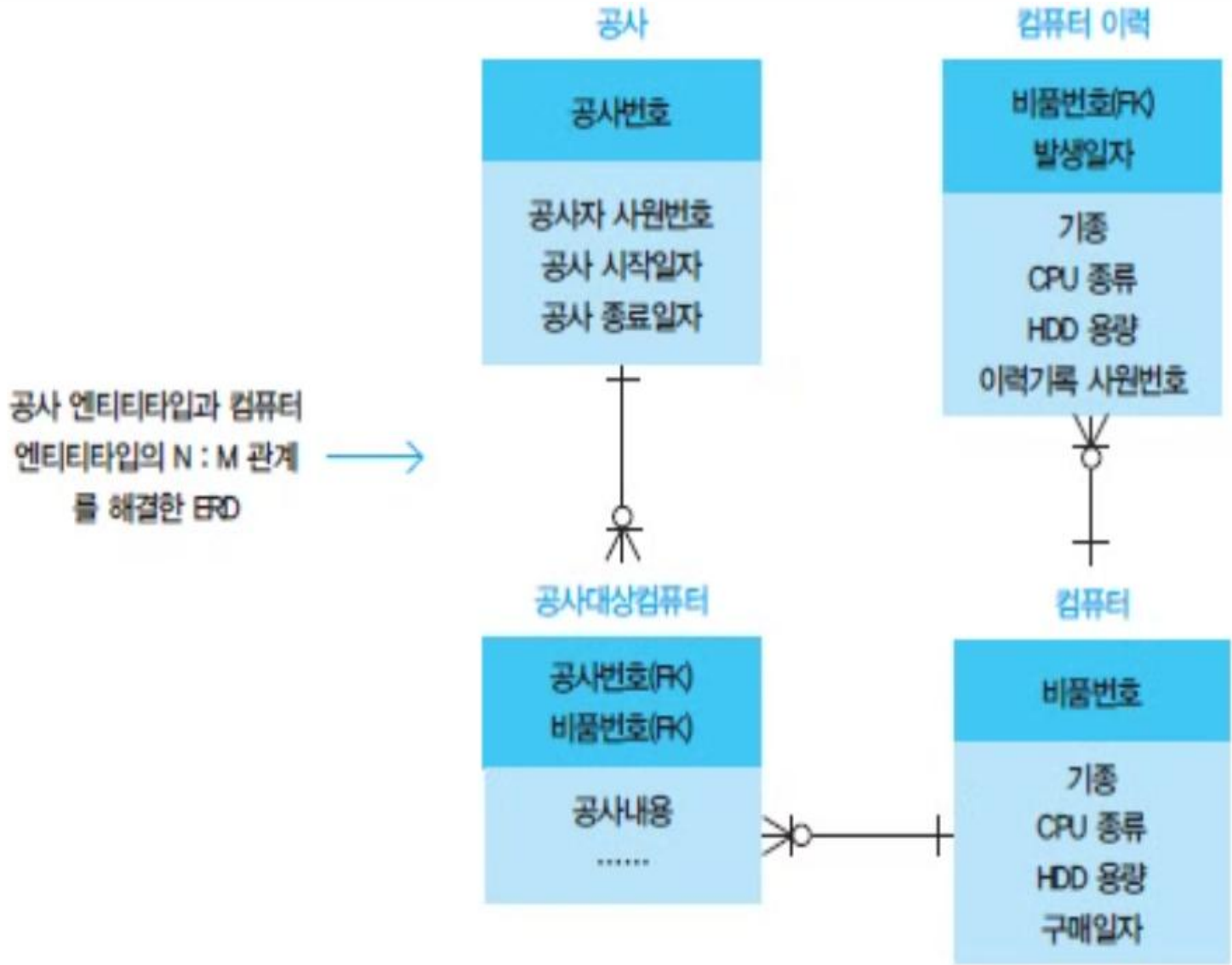
데이터 무결성을 유지하고 감사 추적을 용이하게 합니다.

이력 관리 엔티타입 구현



이력 관리 엔티타입

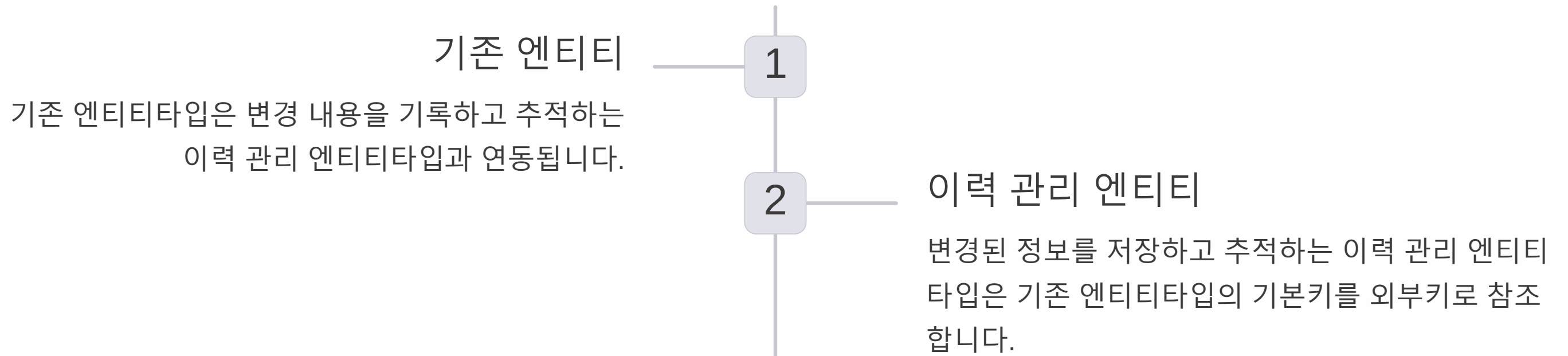
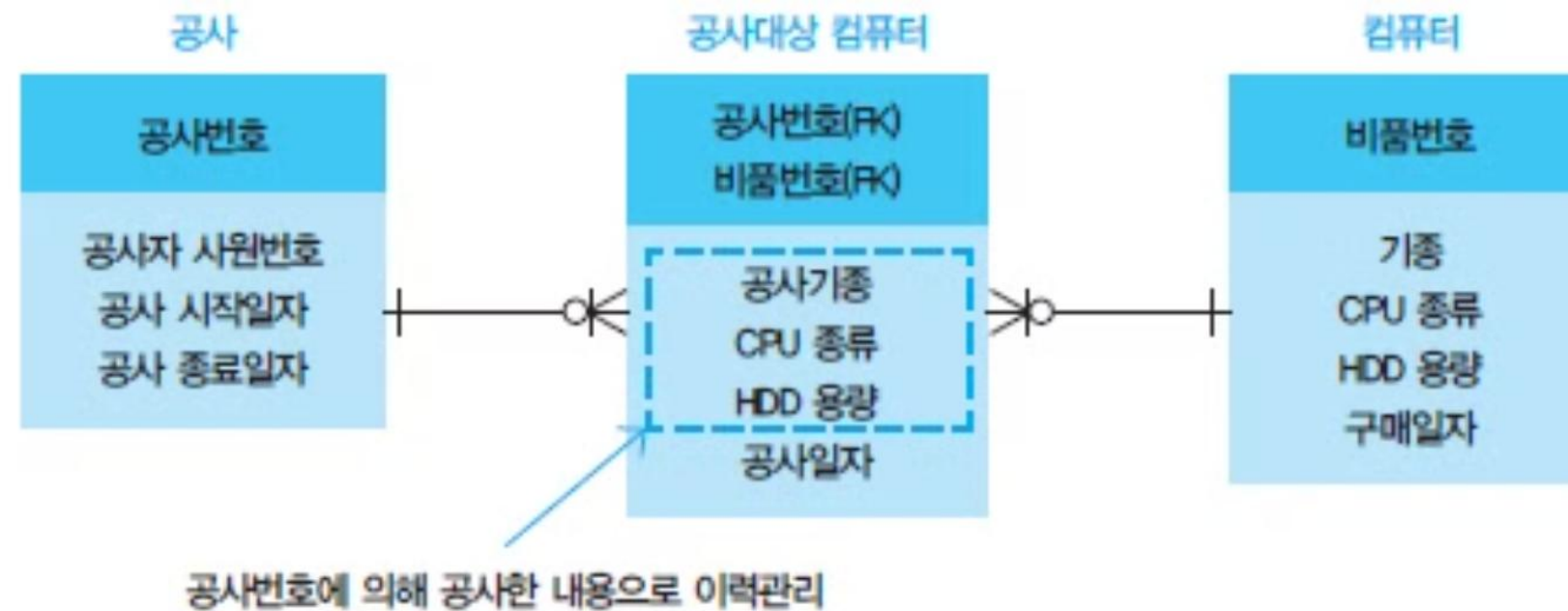
변경된 내용을 기록하고 추적할 수 있도록 기존 엔티타입과 연결됩니다. 이력 관리 엔티타입은 기존 엔티타입의 기본키를 외부키로 참조합니다.



추가 속성

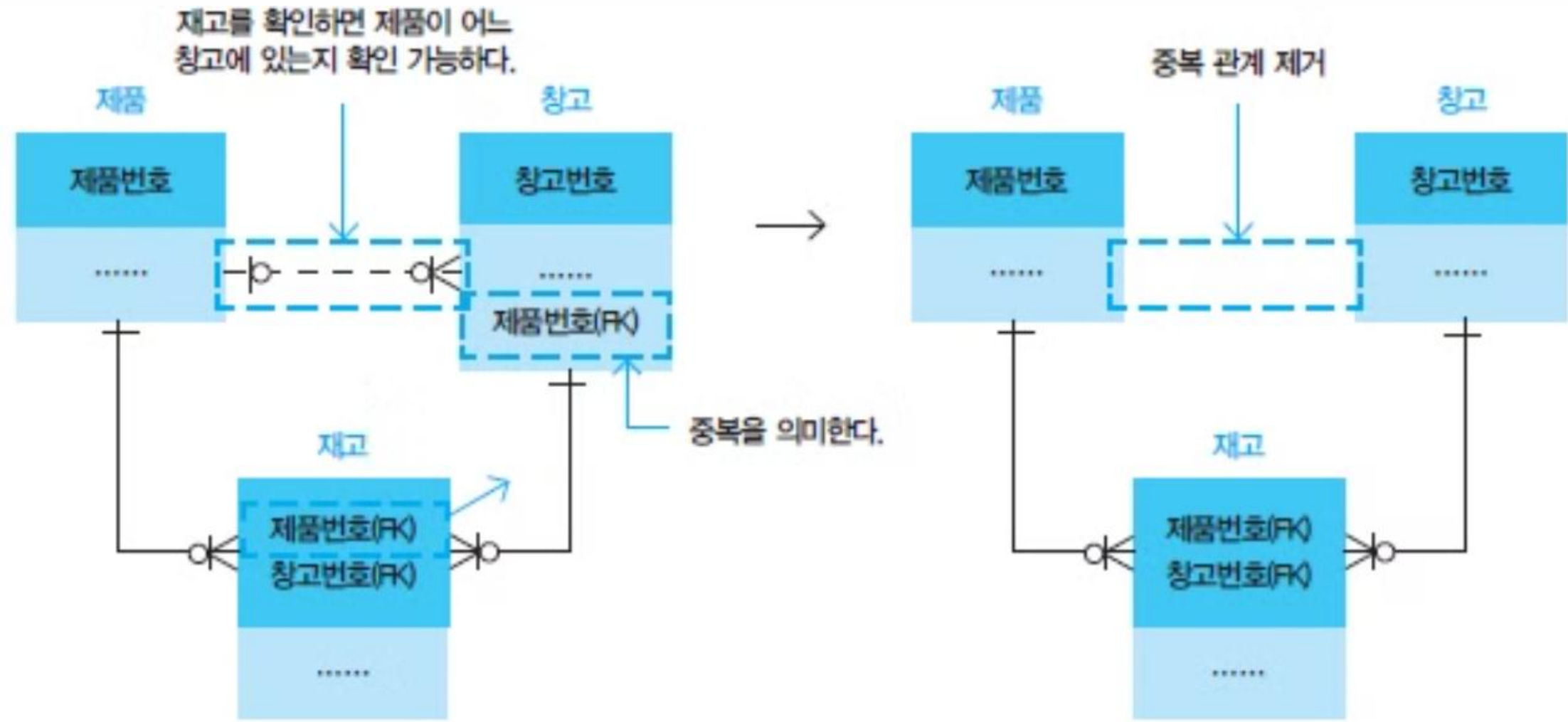
변경된 내용을 상세히 기록하기 위해 추가 속성을 포함합니다. 이러한 속성은 변경된 값, 변경 시간, 변경자 등을 저장합니다.

이력 관리 ERD 예시

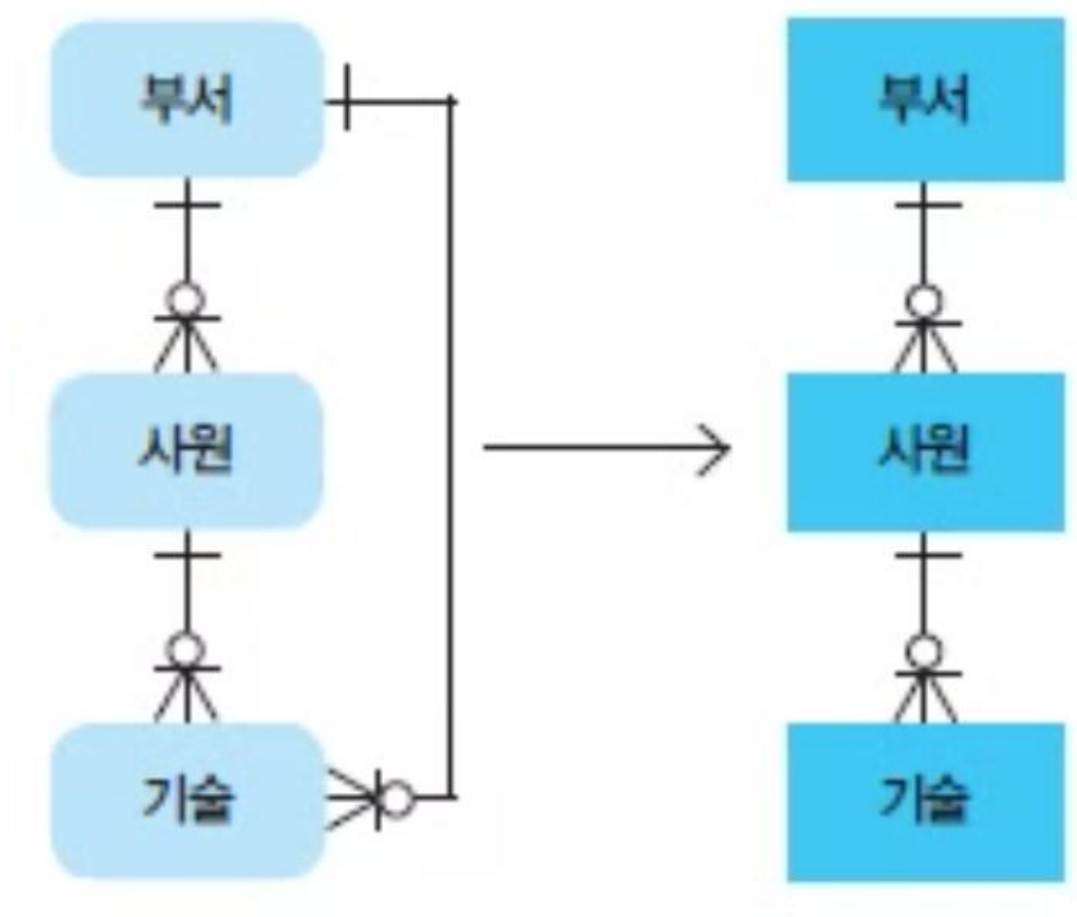


중복 관계 제거

- 원형 관계는 두 개 이상의 엔티티가 서로 순환적으로 연결되는 관계입니다.
- 사슬 관계는 세 개 이상의 엔티티가 순차적으로 연결되는 관계입니다.
- 중복 관계는 데이터 무결성을 저해하고 데이터 관리를 복잡하게 합니다.



중복 관계 제거



원형 관계

새로운 엔티티타입을 생성하여 관계를 단순화합니다.

사슬 관계

관계를 단순화하기 위해 중간 엔티티타입을 추가합니다.

중복 관계

관계를 재설계하여 중복을 제거합니다. 새로운 엔티티타입을 생성하거나 속성을 재분배할 수 있습니다.

엔티티 무결성

1 엔티티 무결성 규칙
데이터베이스에서 각 엔티티 타입의 기본키를 구성하는 속성에 적용되는 규칙입니다.

2 규칙 1
기본키를 구성하는 속성은 반드시 값을 가져야 합니다. 즉, null 값을 가질 수 없습니다.

3 규칙 2
기본키는 유일성을 보장해 주는 최소한의 집합입니다. 즉, 중복된 값이 존재할 수 없습니다.

규칙1과 규칙2의 조건 만족 →

사원번호
주민등록번호
사원번호, 퇴사일자
사원번호, 부서코드



규칙 1	규칙 2
0	0
0	0
X	0
0	X

엔티티 무결성 규칙

기본키는 데이터베이스에서 엔티티를 유일하게 식별하는 역할을 합니다.

기본키는 안정성, 최소성, 독립성, 확정성 및 유일성, 접속성 및 보안성을 만족해야 합니다.

기본키 속성의 순서는 Where 조건에 자주 검색되는 순서대로 지정하고, 분포도가 좋은 칼럼 순으로 배열하는 것이 일반적입니다.



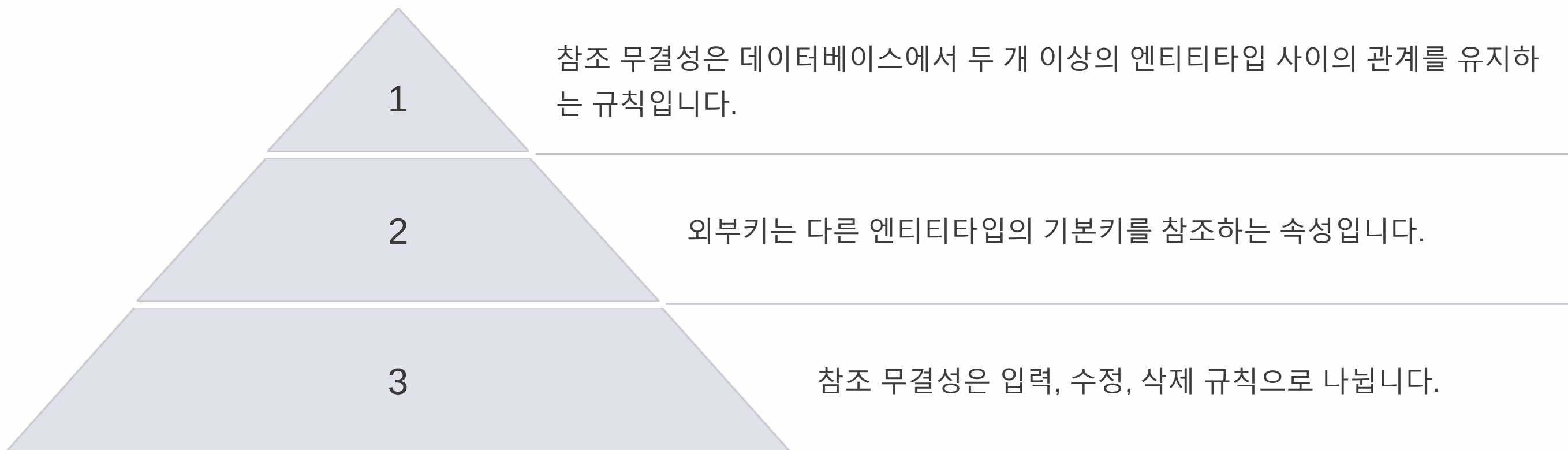
기본키 데이터 타입

가변 길이 문자는 가변형 문자
형으로 사용해야 합니다.

기본키는 추출 속성을 사용해서
는 안 됩니다. 추출 속성은 다른
속성에서 계산된 값이므로 안정
성이 떨어질 수 있습니다.

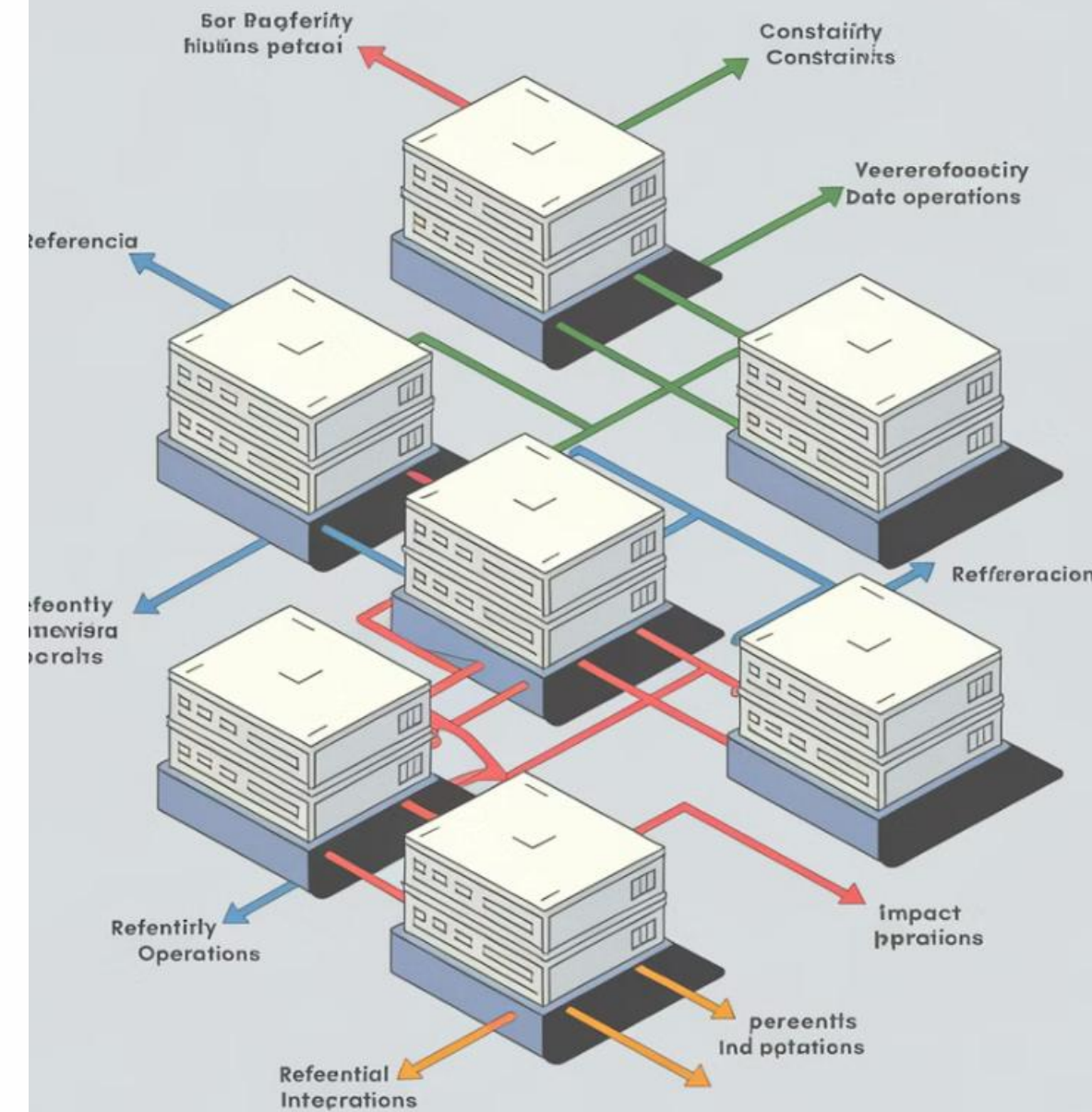
엔티티 무결성은 데이터베이스에서 엔티티의 유일성을 보장하는 규
칙이므로, 실세계의 엔티티와 대응해야 합니다.

참조 무결성



참조 무결성 규칙

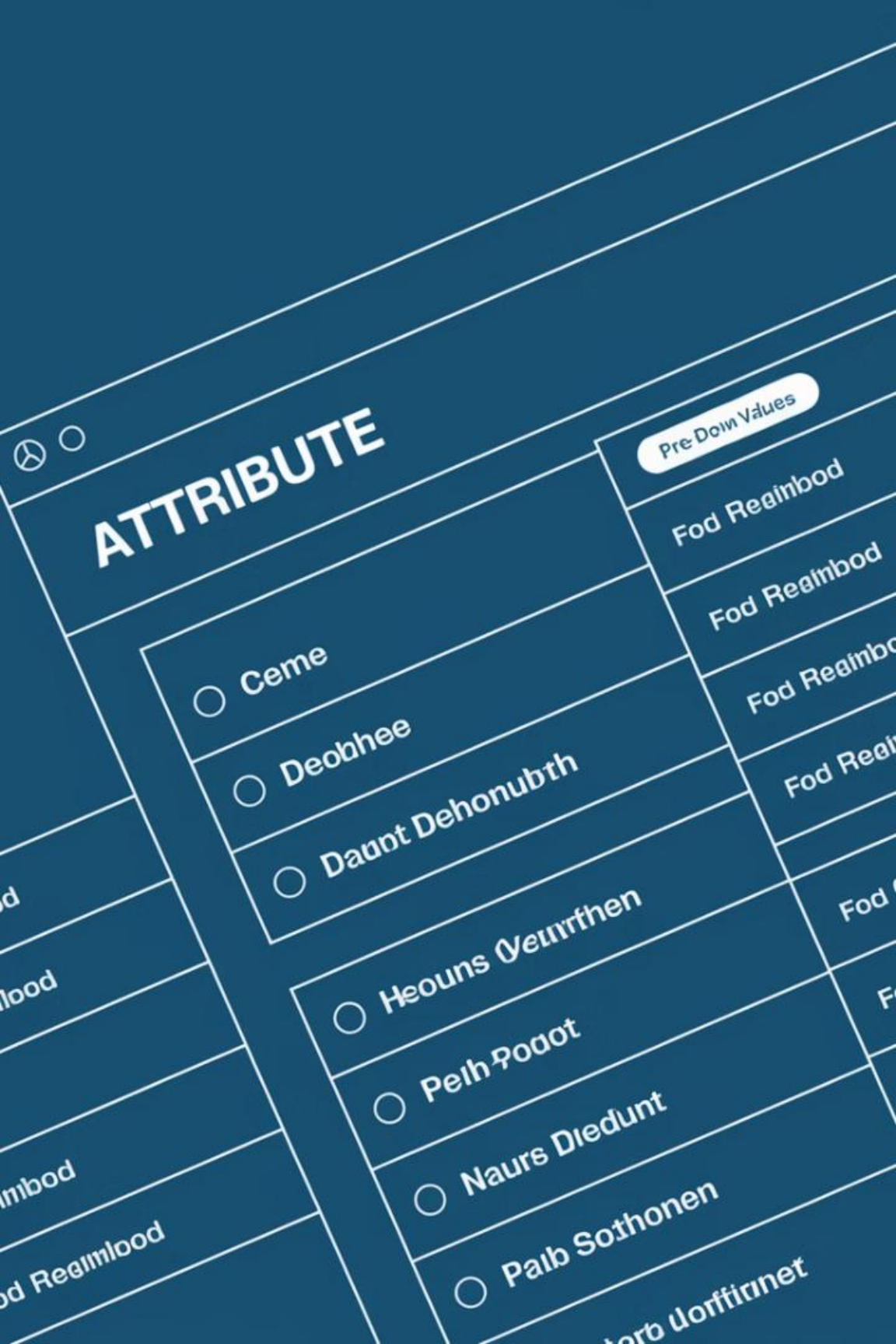
- | 입력 참조 무결성 | 수정 참조 무결성 |
|---|--|
| 새로운 데이터를 입력할 때 외부키의 값이 부모 엔티티타입의 기본키에 존재하는지 확인합니다. | 데이터를 수정할 때 외부키의 값이 부모 엔티티타입의 기본키에 존재하는지 확인합니다. |
| 삭제 참조 무결성 | |
| 부모 엔티티타입의 데이터를 삭제할 때 자식 엔티티타입의 외부키 값을 어떻게 처리할지 결정합니다. | |





참조 무결성 적용

- 참조 무결성은 프로그램 사양서에서 기술하여 SQL로 처리할 수 있습니다.
- 제약 조건을 이용하여 참조 무결성을 처리할 수도 있습니다.
- 참조 무결성은 데이터베이스 시스템에서 자동으로 검증됩니다.

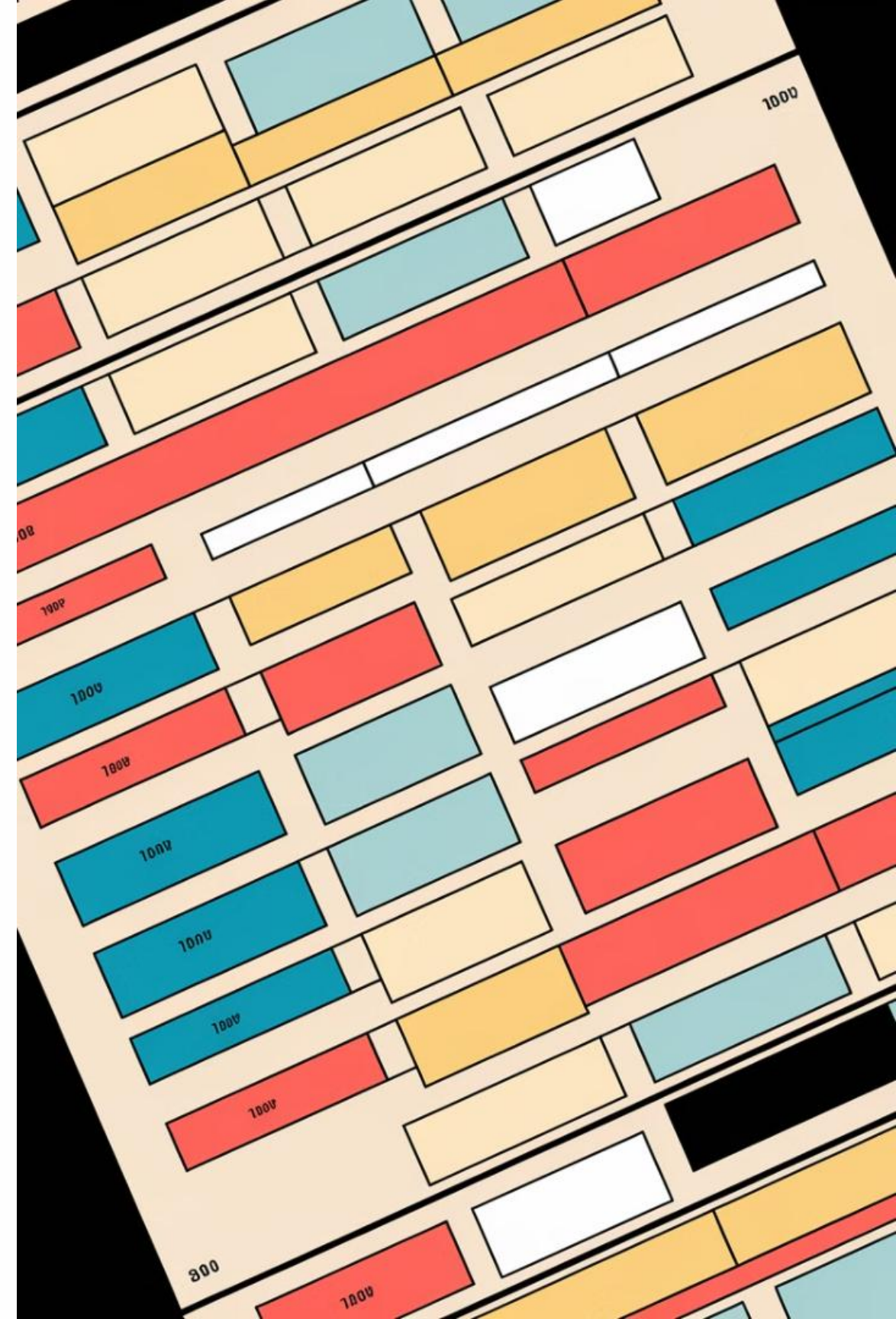


도메인

- 도메인은 속성이 가질 수 있는 값의 범위를 정의합니다.
- 도메인은 데이터 형태, 길이, 형식, 허용되는 값, 의미, 유일성, Null 여부, 초기값 등을 포함합니다.
- 도메인은 데이터 무결성을 보장하고 데이터 일관성을 유지하는 데 중요한 역할을 합니다.

도메인 검증

- 기본키의 도메인은 유일성을 보장해야 합니다. 단일 속성인 경우 해당 속성의 각 엔티티 값은 'Unique'해야 합니다.
- 외부키의 도메인은 부모 엔티티타입의 대응 속성과 일치해야 합니다.
- 추출 속성의 도메인은 원시 속성의 데이터 형태와 일치해야 합니다.



속성 무결성

- 속성 무결성은 데이터베이스에서 속성 값의 유효성을 보장하는 규칙입니다.
- 속성 무결성은 데이터 형태, 길이, 형식, 제약 조건 등을 포함합니다.
- 속성 무결성은 데이터의 정확성과 일관성을 유지하는 데 중요한 역할을 합니다.

속성 무결성 규칙

- 속성 무결성은 업무 규칙을 기반으로 정의됩니다.
 - 속성 무결성 규칙은 데이터 무결성을 보장하고 데이터의 정확성과 일관성을 유지하는 데 필수적입니다.
- 업무 규칙은 사용자 규칙, 사건, 실체명, 속성명, 조건, 연쇄 작용 등을 포함합니다.

Customer ID

Name

Email

☐ Name

Mrowe, X 010
01 (367 (2/1/))

fitL

NIO

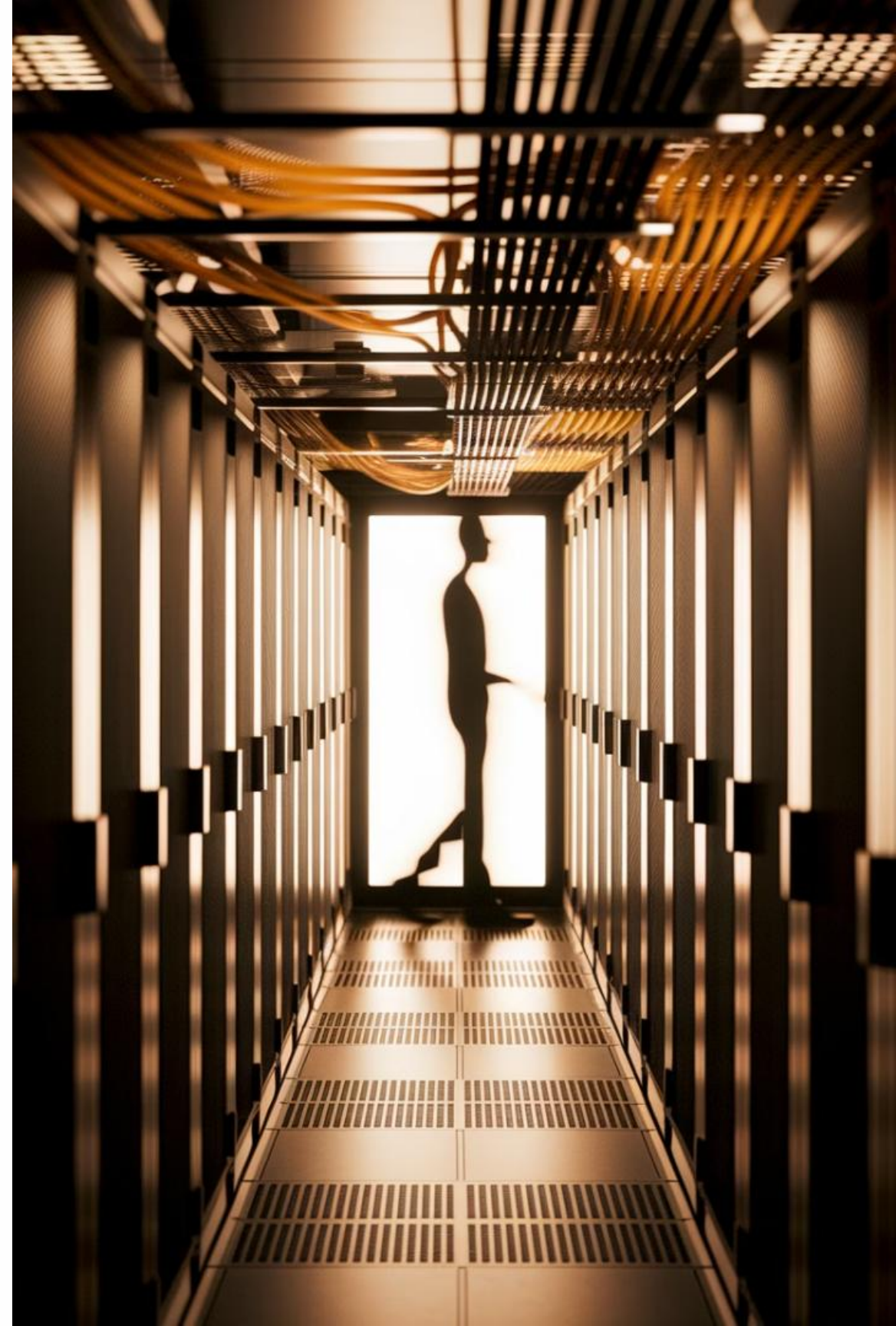
CUSC-1414

ADDRESS

데이터베이스 정규화(Normalization)

데이터베이스 설계 및 관리에서 정규화는 필수적인 과정으로, 데이터 무결성과 효율성을 향상시킵니다.

정규화의 개념과 절차를 심층적으로 살펴봅니다.



정규화의 목적

데이터 중복 최소화

중복된 데이터를 제거하여 저장 공간을 절약하고 데이터 일관성을 유지합니다.

데이터 무결성 보장

데이터 이상 현상을 방지하여 데이터의 정확성과 신뢰성을 높입니다.

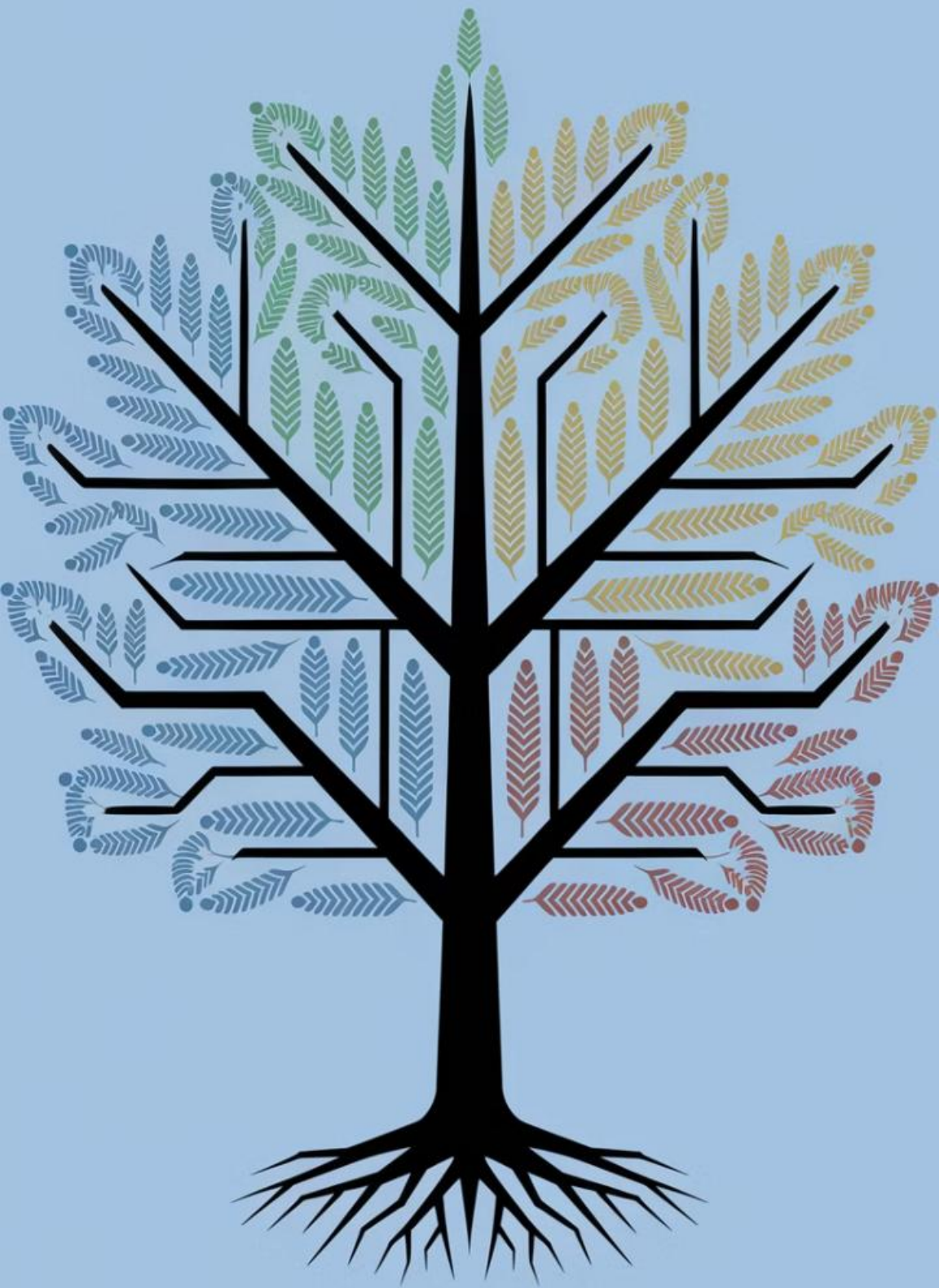
데이터 관리 효율성 향상

데이터베이스 설계 및 관리를 간소화하여 효율성을 높입니다.

FUNCTIONAL DEPENDENCY

함수적 종속성의 개념

- 1 속성 간 종속 관계를 나타내는 개념
- 2 하나의 속성 값이 다른 속성 값을 결정하는 관계
- 3 결정자(Determinant): 다른 속성 값을 결정하는 속성
- 4 종속자(Dependent): 결정자에 의해 결정되는 속성



함수적 종속성 종류

1

결정자 속성 하나만으로 종속자 속성 값이 결정됩니다.

2

결정자 속성의 부분집합이 종속자 속성 값을 결정합니다.

3

결정자 속성이 아닌 다른 속성에 의해 종속자 속성 값이 결정됩니다.

함수적 종속성 예시

학번(Student ID) -> 이름(Name)

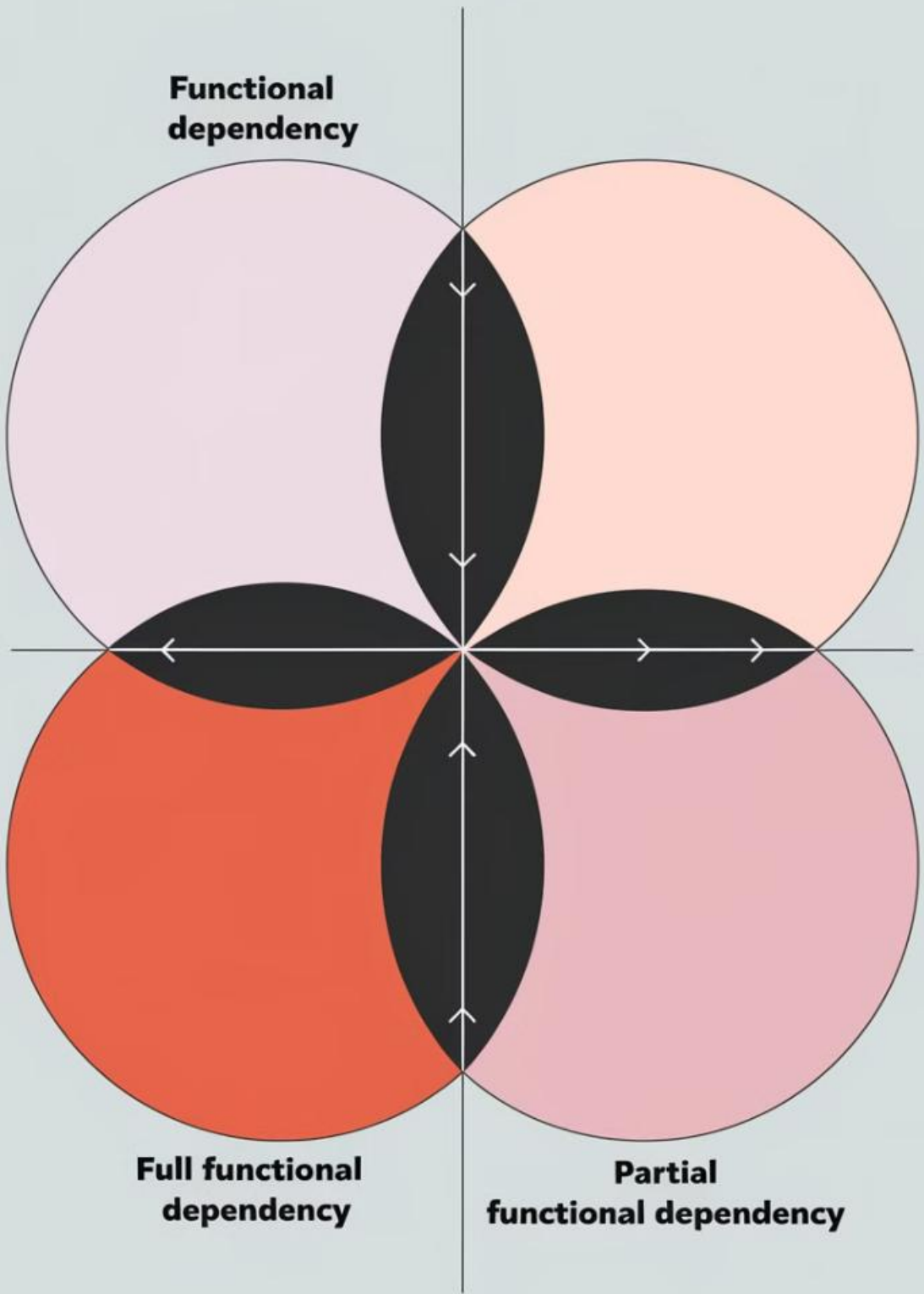
학번은 학생의 이름을 결정합니다.

학번(Student ID) -> 전공(Major)

학번은 학생의 전공을 결정합니다.

학번(Student ID), 과목(Subject) ->
성적(Grade)

학번과 과목은 학생의 성적을 결정합니다.



함수적 종속성 종류

완전 함수적 종속성

결정자가 키의 일부가 아닌 전체 키에 종속되는 관계

부분 함수적 종속성

결정자가 키의 일부에만 종속되는 관계

이행 함수적 종속성

키가 아닌 속성에 종속되는 관계



데이터 이상 현상



삽입 이상

원하는 데이터만 입력할 수 없고 다른 데이터도 함께 입력해야 하는 경우 발생합니다.



삭제 이상

특정 데이터를 삭제하면 다른 데이터도 함께 삭제되는 경우 발생합니다.



수정 이상

데이터가 중복되어 일부만 수정하면 다른 데이터와 일관성이 깨지는 경우 발생합니다.

데이터 이상 현상 사례

Order_ID	Customer_Name	Customer_Address	Products	Total_Amount
1	김철수	서울 강남구	노트북, 마우스	1500000
2	이영희	부산 해운대구	키보드, 모니터	500000
3	박민수	대구 수성구	스마트폰, 충전기	1200000

위 예시에서 하나의 주문에 제품이 여러 개 존재함

정규화 과정



정규화 단계별 설명

1

제1 정규화

각 속성은 단일 값만 가져야 합니다. 반복 값 또는 복수 값을 갖는 속성은 별도의 테이블로 분리합니다.

2

제2 정규화

기본 키에 완전 함수적으로 종속되지 않는 속성은 별도의 테이블로 분리합니다.

3

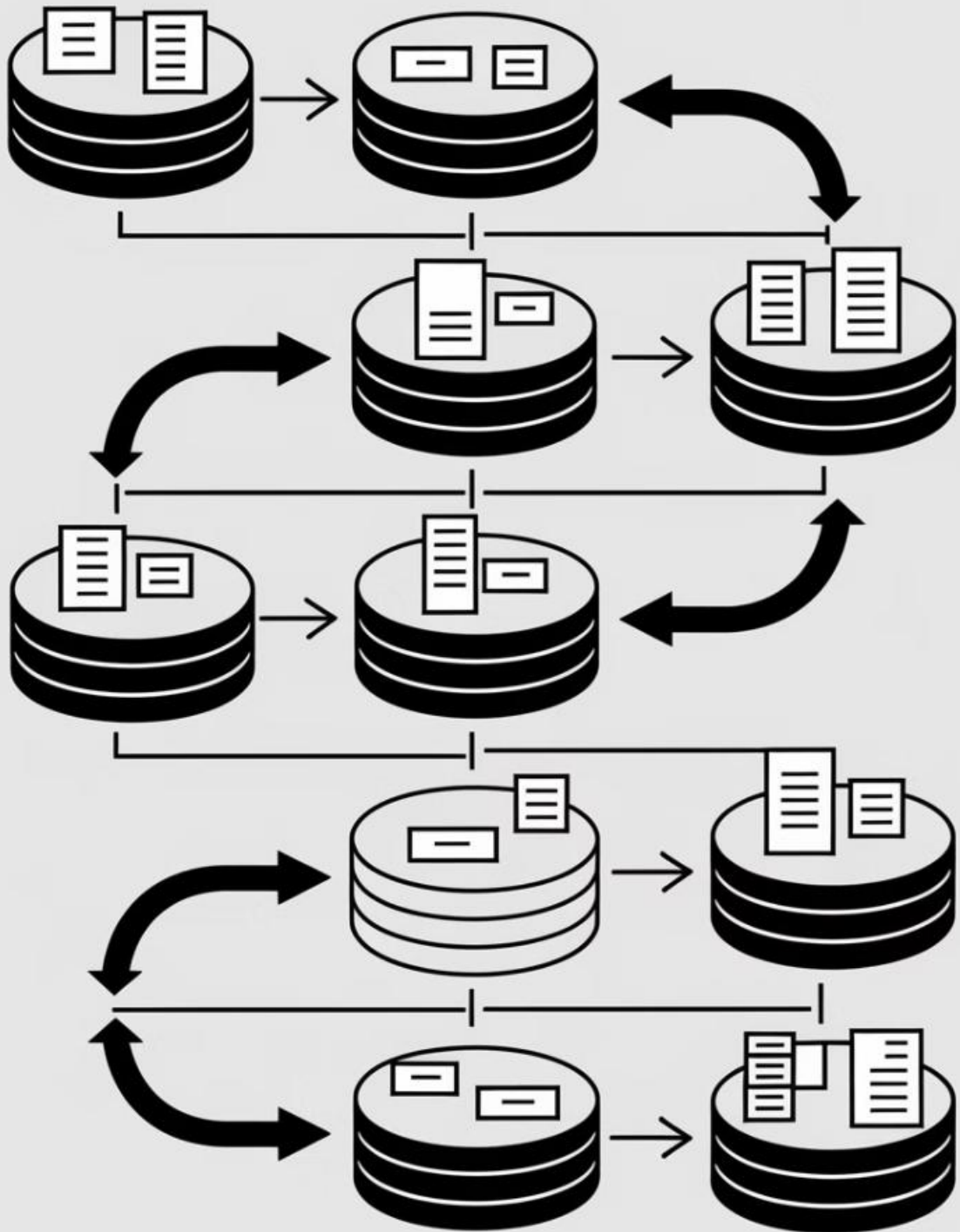
제3 정규화

기본 키가 아닌 속성에 종속된 속성은 별도의 테이블로 분리합니다.

4

BCNF

2NF, 3NF가 만족되더라도 이상 현상이 발생할 수 있으며, 모든 결정자가 키 속성으로 존재하도록 분리합니다.



Order_ID	Customer_Name	Customer_Address	Products	Total_Amount
1	김철수	서울 강남구	노트북, 마우스	1500000
2	이영희	부산 해운대구	키보드, 모니터	500000
3	박민수	대구 수성구	스마트폰, 충전기	1200000

데이터 이상 현상

삽입 이상

새로운 고객 '홍길동' '서울 송파구'의 정보를 추가하려면 주문 정보도 함께 입력해야 합니다.

삭제 이상

1번 주문 정보를 삭제하면 김철수 고객의 정보가 사라집니다.

수정 이상

김철수의 주소를 수정하면 주문정보에서 김철수의 모든 주문정보의 주소 정보도 함께 수정해야 합니다.

Customers

Customerid	110
Firstname	\$10
Lastname	\$30
Email	\$20
Phone	\$30

Orderdate	\$40
-----------	------

Customerid

Productid	150
Productid	\$40
Producane	\$30
Price	\$40
Orderdate	\$40

Orders

데이터 이상 현상 해결 방안

1 부분 함수 종속성을 제거하여 키의 일부분에만 종속되는 관계를 완전 종속으로 분리합니다.

2 이행 함수 종속성을 제거하여 결정자가 아닌 다른 속성에 의해 종속되는 현상을 제거합니다.

정규화 예시: 제1정규화

Order_ID	Customer_Name	Customer_Address	Products	Total_Amount
1	김철수	서울 강남구	노트북, 마우스	1500000
2	이영희	부산 해운대구	키보드, 모니터	500000
3	박민수	대구 수성구	스마트폰, 충전기	1200000

정규화 전

제1 정규화 후

Order_ID	Customer_Name	Customer_Address	Product	Price
1	김철수	서울 강남구	노트북	1300000
1	김철수	서울 강남구	마우스	200000
2	이영희	부산 해운대구	키보드	100000
2	이영희	부산 해운대구	모니터	400000
3	박민수	대구 수성구	스마트폰	1100000
3	박민수	대구 수성구	충전기	100000

정규화 예시: 제2정규화

주문 정보

Order_ID	Customer_Name	Customer_Address
1	김철수	서울 강남구
2	이영희	부산 해운대구
3	박민수	대구 수성구

Order_ID	Customer_Name	Customer_Address	Product	Price
1	김철수	서울 강남구	노트북	1300000
1	김철수	서울 강남구	마우스	200000
2	이영희	부산 해운대구	키보드	100000
2	이영희	부산 해운대구	모니터	400000
3	박민수	대구 수성구	스마트폰	1100000
3	박민수	대구 수성구	충전기	100000

주문 상세

Order_ID	Product_ID	Product	Price
1	101	노트북	1300000
1	102	마우스	200000
2	103	키보드	100000
2	104	모니터	400000
3	105	스마트폰	1100000
3	106	충전기	100000

정규화 예시: 제3정규화

고객

Customer_ID	Customer_Name	Customer_Address
1	김철수	서울 강남구
2	이영희	부산 해운대구
3	박민수	대구 수성구

제품

Product_ID	Product	Price
101	노트북	1300000
102	마우스	200000
103	키보드	100000
104	모니터	400000
105	스마트폰	1100000
106	충전기	100000

주문

Order_ID	Customer_ID
1	1
2	2
3	3

주문상세

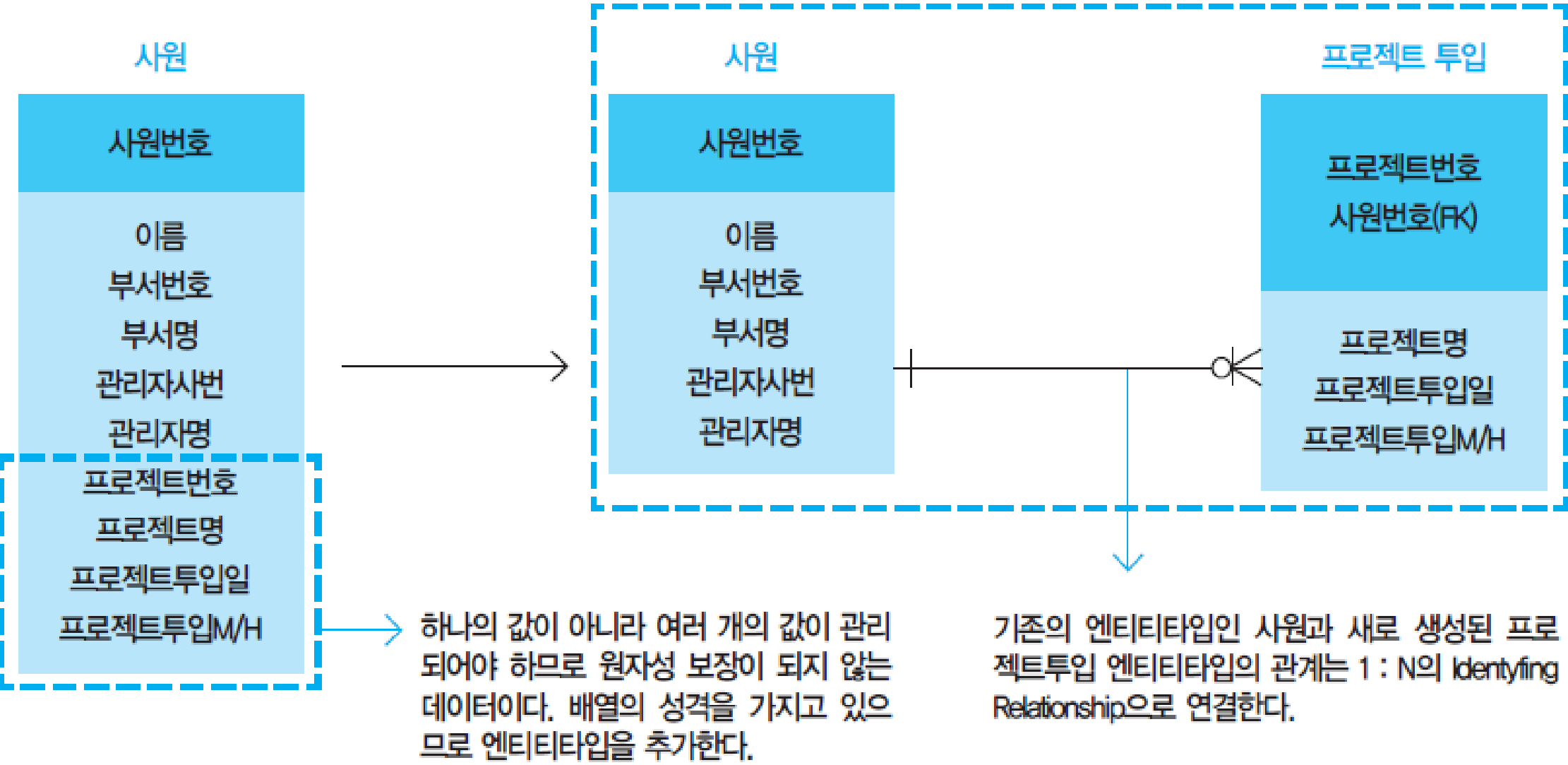
Order_ID	Product_ID
1	101
1	102
2	103
2	104
3	105
3	106



정규화 실습

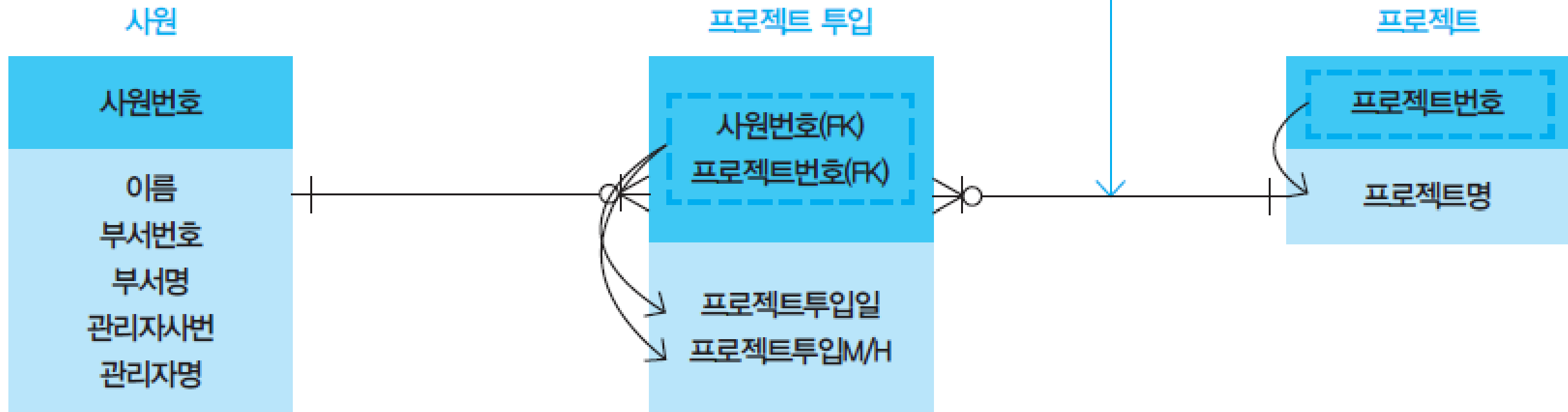
사원번호	이름	부서번호	부서명	관리자사번	관리자명	프로젝트번호	프로젝트명	투입일	투입M/H
PK									
7902	홍길동	10	영업부	7562	강동원	15	외환 전산망 구축	98/1/7	100
						35	정부기관 통합 홈페이지	97/10/5	100
						45	시군구 행정 전산망 구축	98/4/9	150
7988	오혜성	20	관리부	7699	김세종	15	외환 전산망 구축	98/1/10	200
						25	증권시장 분석 시스템	98/10/11	250
						45	시군구 행정 전산망 구축	98/5/1	200
7562	강동원	10	영업부	7099	이태백	25	증권시장 분석 시스템	98/9/1	150

정규화 실습: 1NF

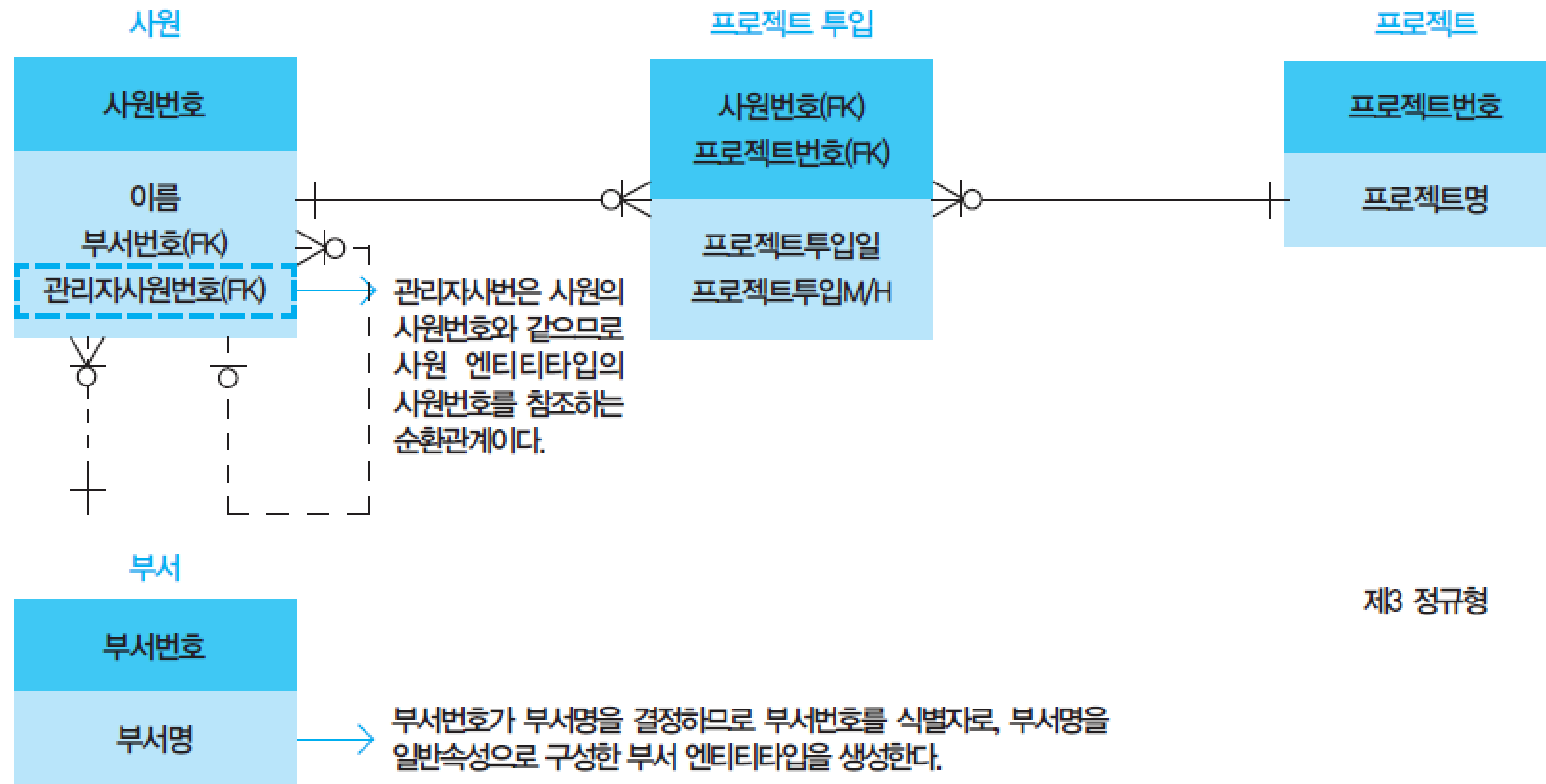


정규화 실습: 2NF

식별자에 완전종속인 프로젝트 엔티티타입을 추가한 후 기존의 엔티티타입과 1:N의 관계 설정



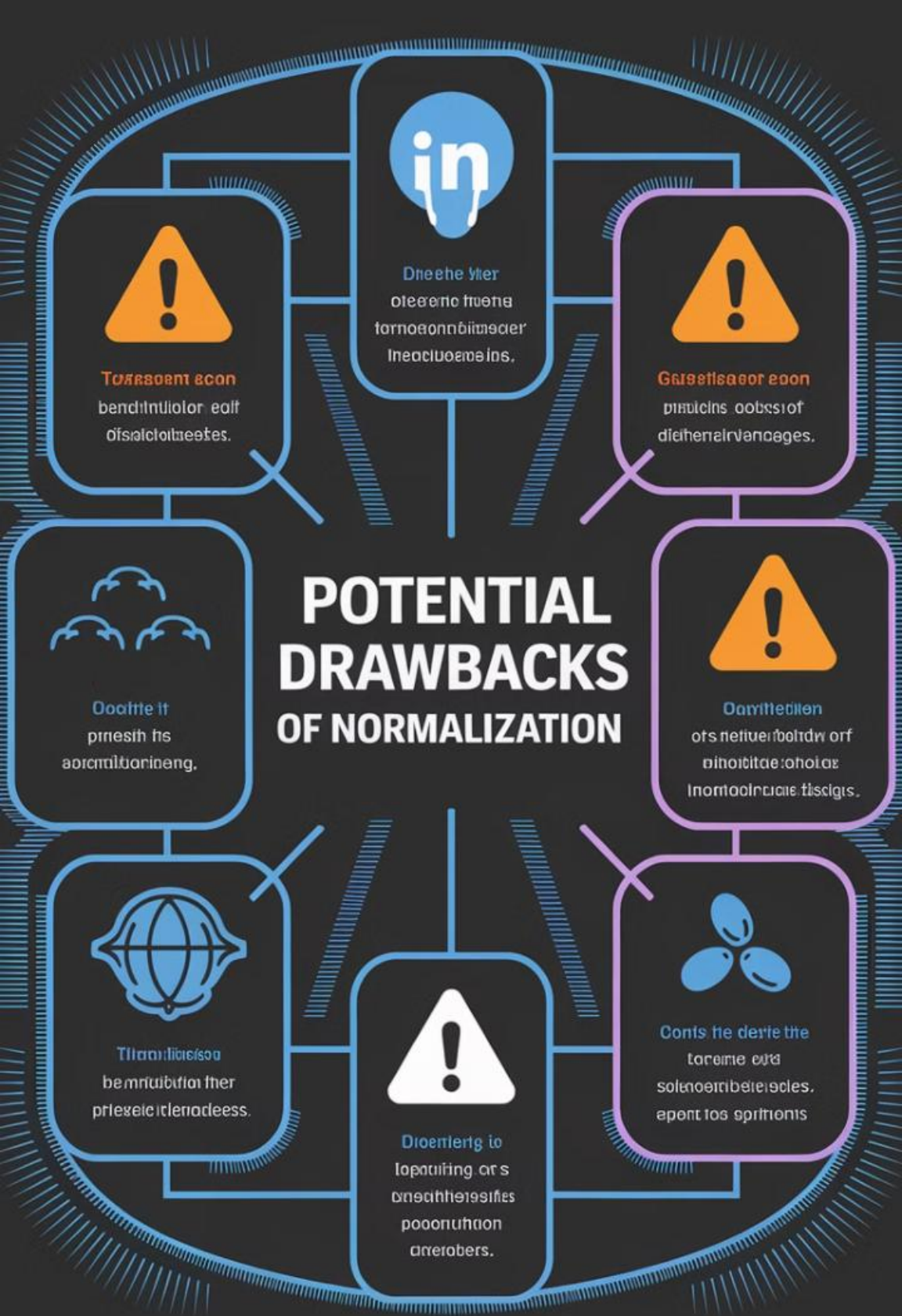
정규화 실습: 3NF





정규화 장점

- 1 데이터 중복을 줄여 저장 공간을 절약합니다.
- 2 데이터 일관성을 유지하여 데이터 무결성을 보장합니다.
- 3 데이터 관리 및 업데이트 작업을 효율적으로 수행할 수 있습니다.
- 4 데이터베이스 성능을 향상시켜 빠르고 효율적인 데이터 처리를 가능하게 합니다.



정규화 단점

1

데이터베이스 설계 및 구현 작업의 복잡성이 증가할 수 있습니다.

2

데이터 조회 및 조인 작업 시 성능 저하가 발생할 수 있습니다.

3

데이터베이스 설계 및 관리에 대한 전문 지식이 요구됩니다.



정규화 적용 사례

1

고객 관리 시스템: 고객 정보, 주문 정보, 배송 정보 등을 효율적으로 관리

2

전자 상거래 시스템: 상품 정보, 주문 정보, 결제 정보 등을 정확하게 관리

3

인사 관리 시스템: 사원 정보, 급여 정보, 근태 정보 등을 체계적으로 관리



정규화 적용시 고려 사항

- 1 데이터베이스 용도와 사용자 요구 사항을 고려합니다.
- 2 데이터베이스 성능과 데이터 무결성 간의 균형을 맞춥니다.
- 3 데이터베이스 관리 및 유지보수 비용을 고려합니다.

정규화, 최적의 데이터 관리를 위한 열쇠

정규화는 데이터베이스 설계 및 관리의 핵심 개념입니다.
적절한 정규화를 통해 데이터 무결성을 보장하고 데이터베이스 성능을 향상시킬 수 있습니다.



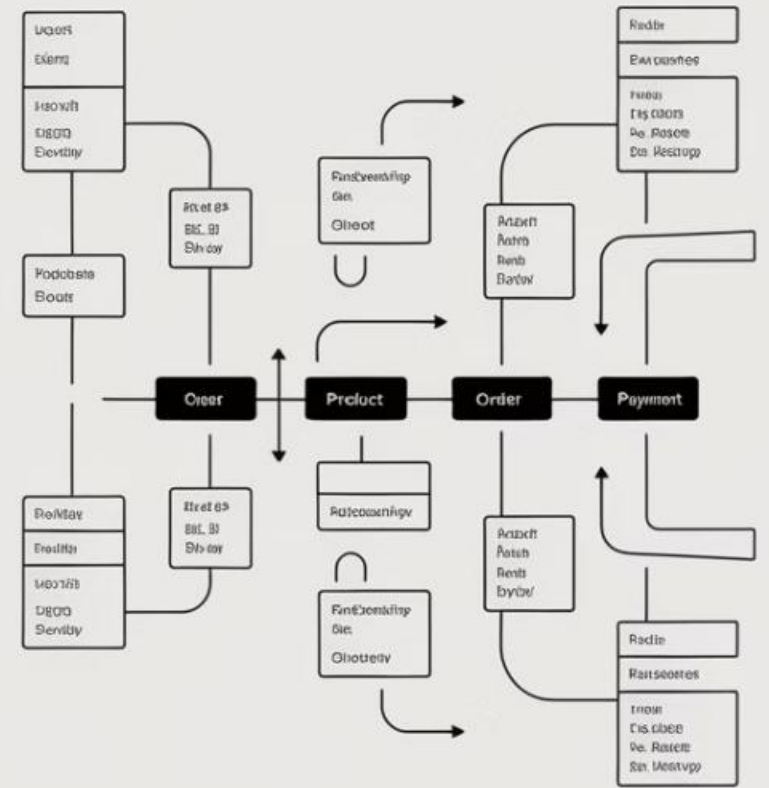


데이터 정규화는 필수

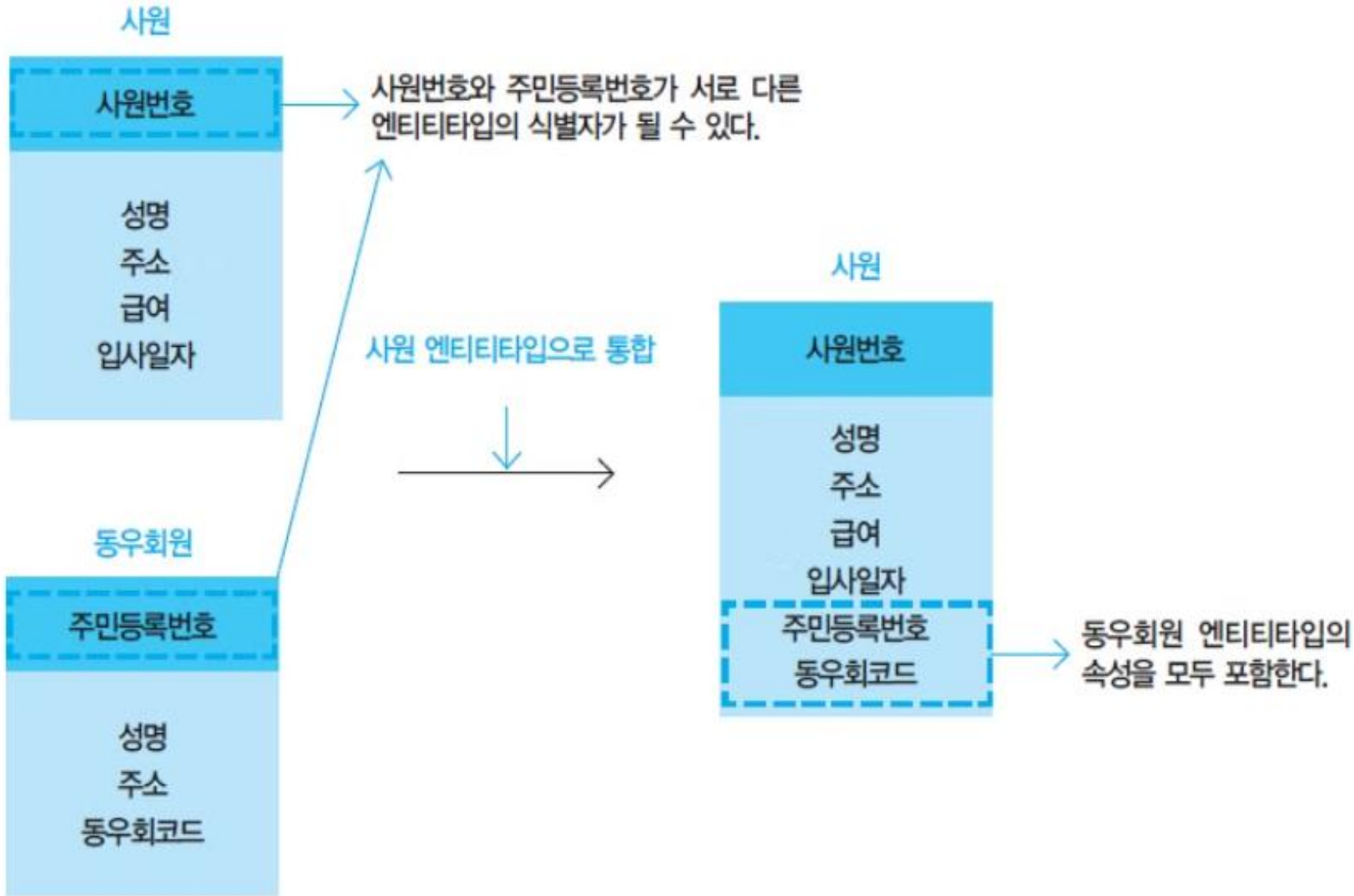
정규화는 데이터베이스 설계 및 관리의 필수적인 과정입니다.
데이터 이상 현상을 예방하고 데이터 무결성을 보장하여 데이터베이스
시스템의 안정성과 효율성을 향상시킵니다.

데이터 모델 통합

데이터 모델 통합은 복잡한 데이터 환경을 단순화합니다.
이 과정은 데이터 중복을 제거하고 이상 현상을 방지합니다.
엔터티 간의 관계를 명확히 하고 새로운 규칙을 추가합니다.
지금부터 데이터 모델 통합의 핵심을 알아보겠습니다.



식별자 기반 통합

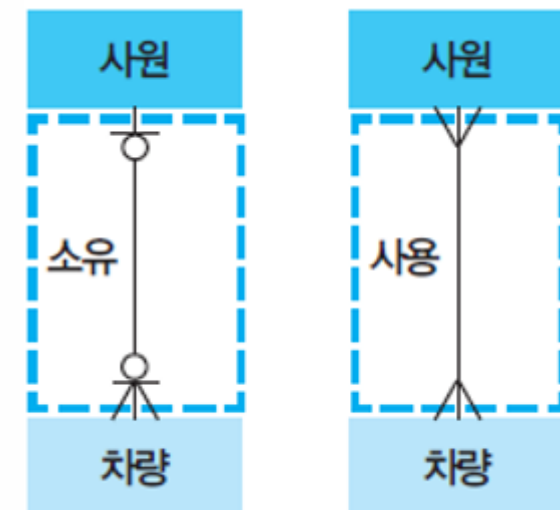


기본 키가 상호 식별 가능한 엔티티를 통합합니다.
이는 데이터 일관성을 유지하는 데 중요합니다

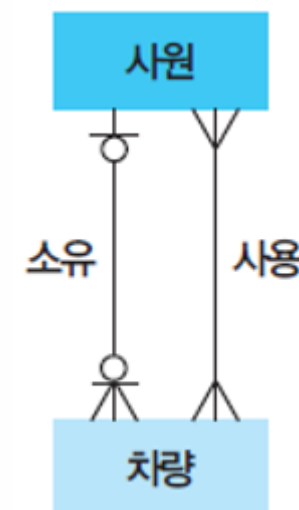
관계 기반 엔티티 통합

- 1 — 관계 분석
- 2 — 엔티티 검토
- 3 — 통합 수행

관계가 다른 엔티티 타입을 통합합니다. 이는 모델의 복잡성을 줄여줍니다.



↓
서로 다른 사용자 관점에서 같은
엔티티타입간의 다른 관계를 통합



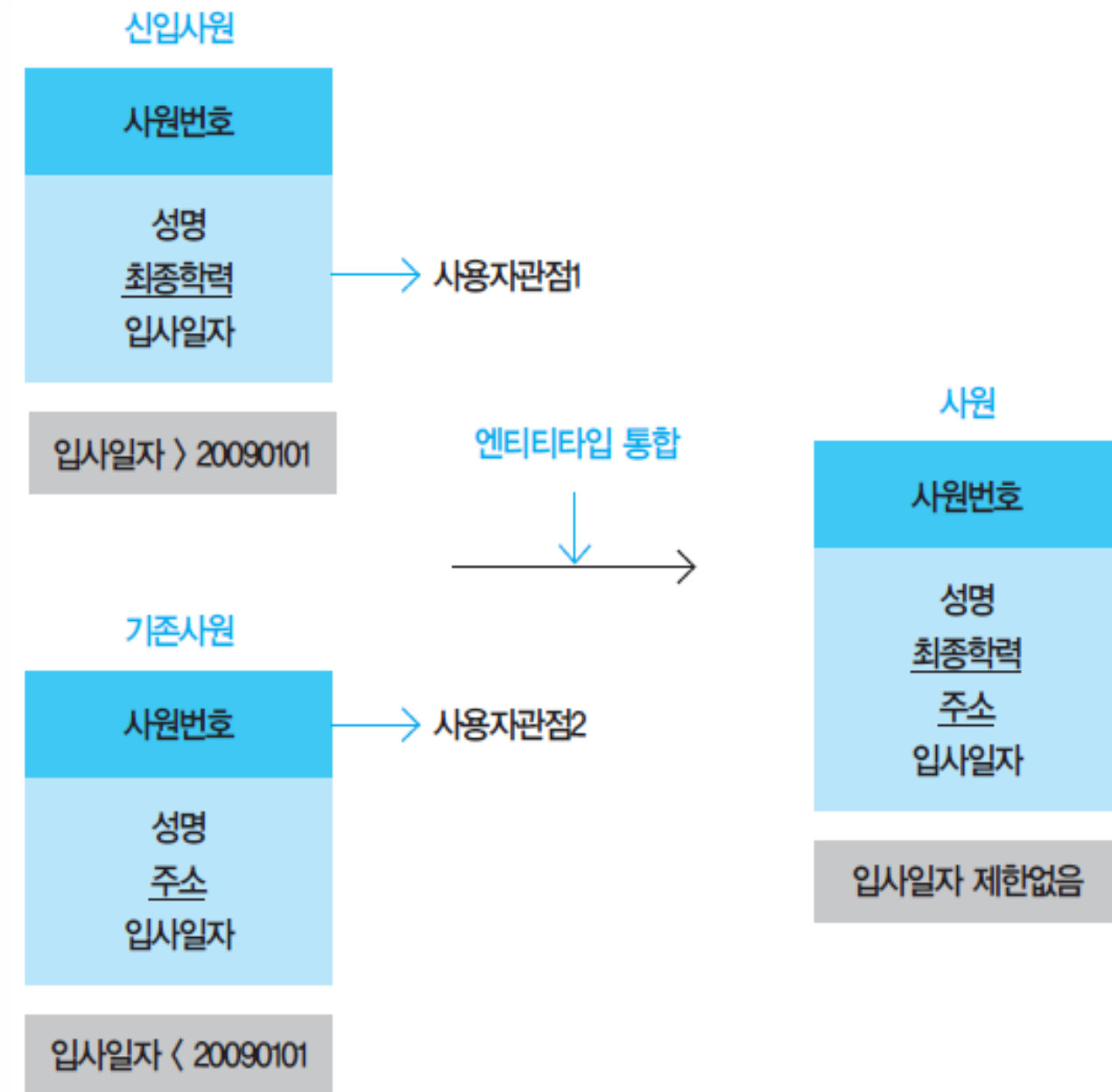
속성 및 규칙 통합

1 속성 분석
속성을 분석하여 중복을 확인합니다.

2 규칙 검토
업무 규칙을 검토하여 일관성을 유지합니다.

3 통합 수행
속성과 규칙을 통합합니다.

속성과 관련된 업무 규칙을 통합합니다. 이는 데이터 품질을 향상시킵니다.



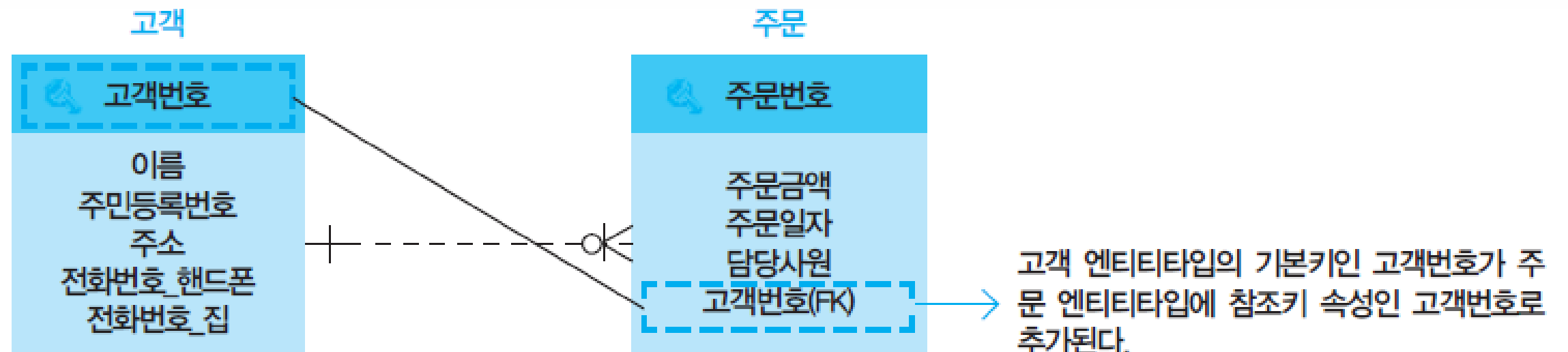
1:N, N:1 관계 변환

1 외래 키 설정

외래 키를 사용하여 관계를 설정합니다. 테이블 간 연결을 관리합니다.

2 참조 무결성

참조 무결성을 유지합니다. 데이터 일관성을 보장합니다.



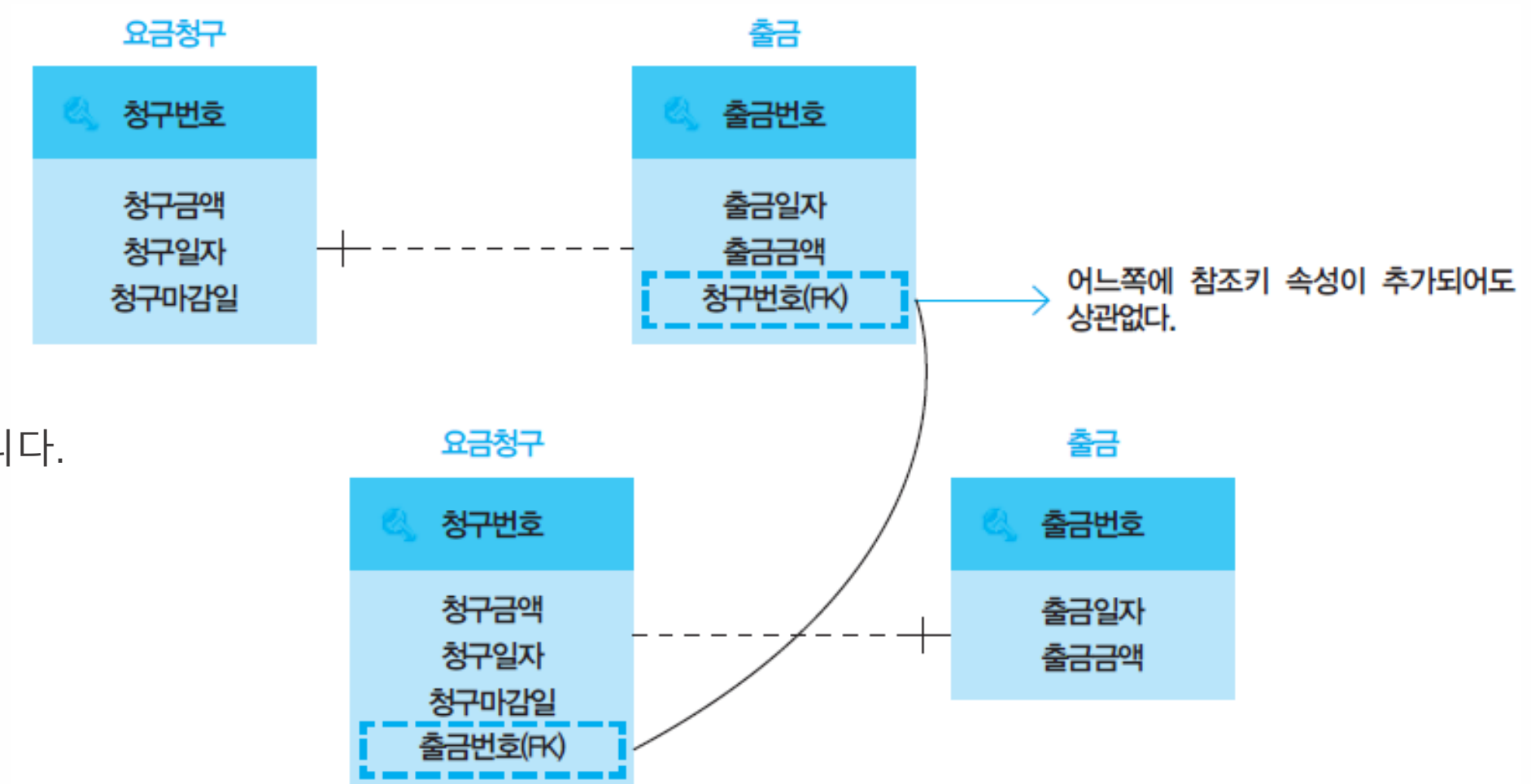
1:1 관계 변환



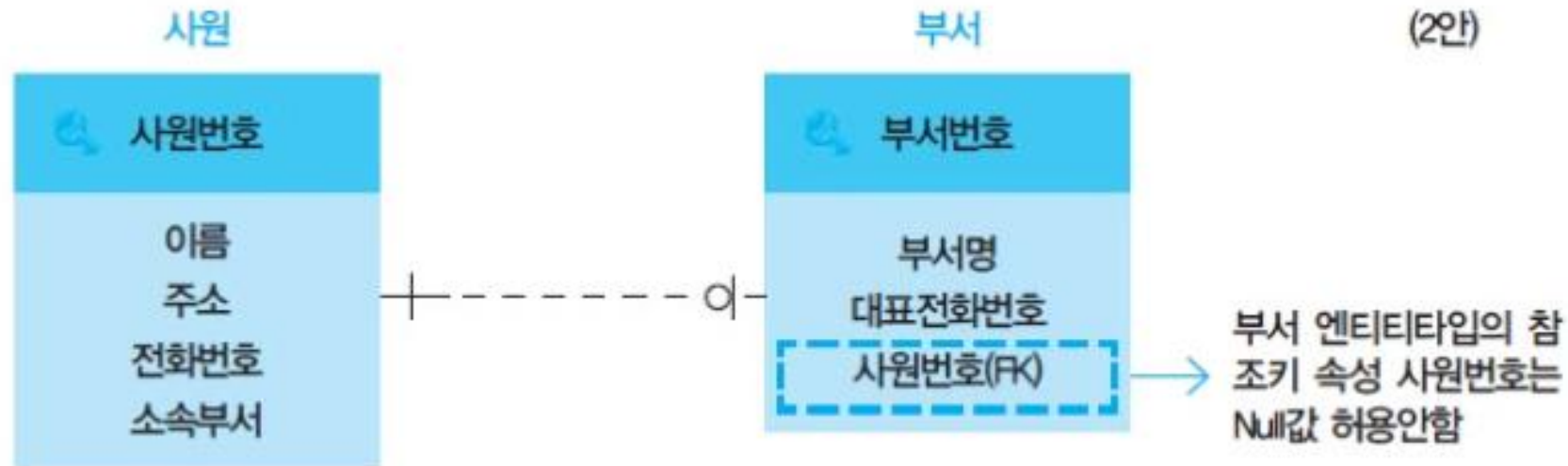
기본 키

기본 키를 확인합니다.

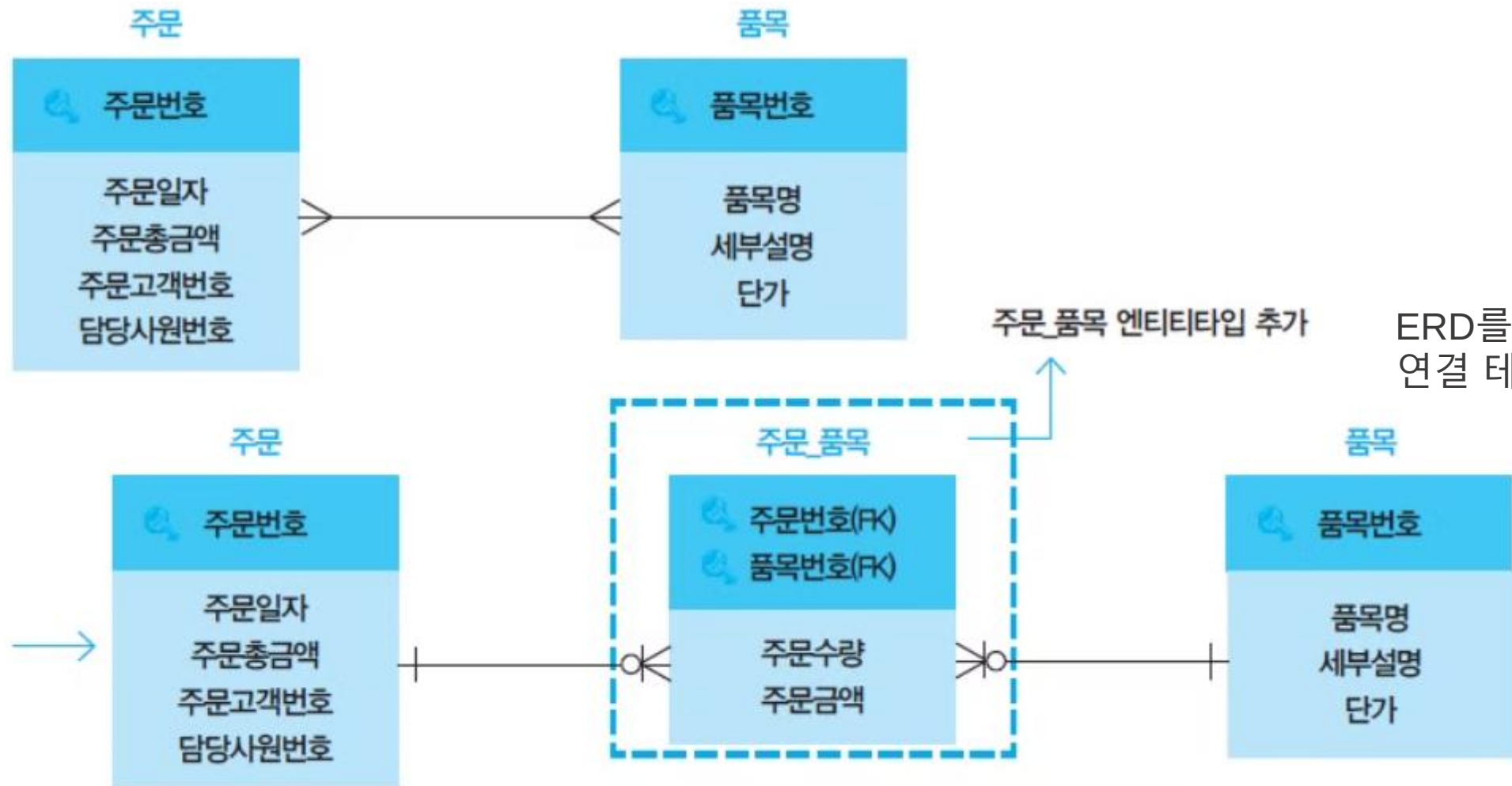
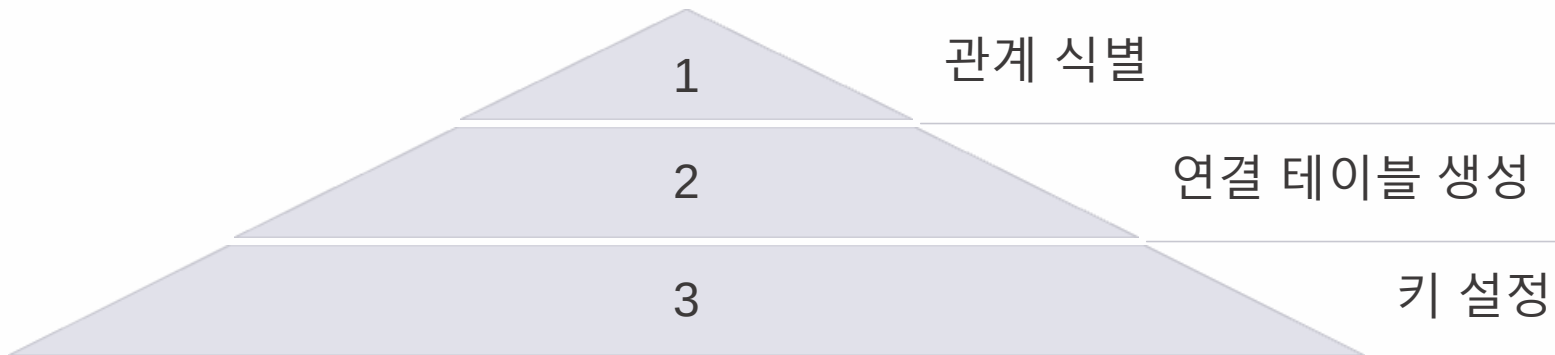
ERD를 테이블로 변환 시 1:1 관계를 처리합니다.
이는 테이블 구조를 단순화합니다.



1:1 관계 변환



M:M 관계 변환



ERD를 테이블로 변환 시 M:M 관계는
연결 테이블 생성으로 해결합니다.

다중 값 속성 변환

1

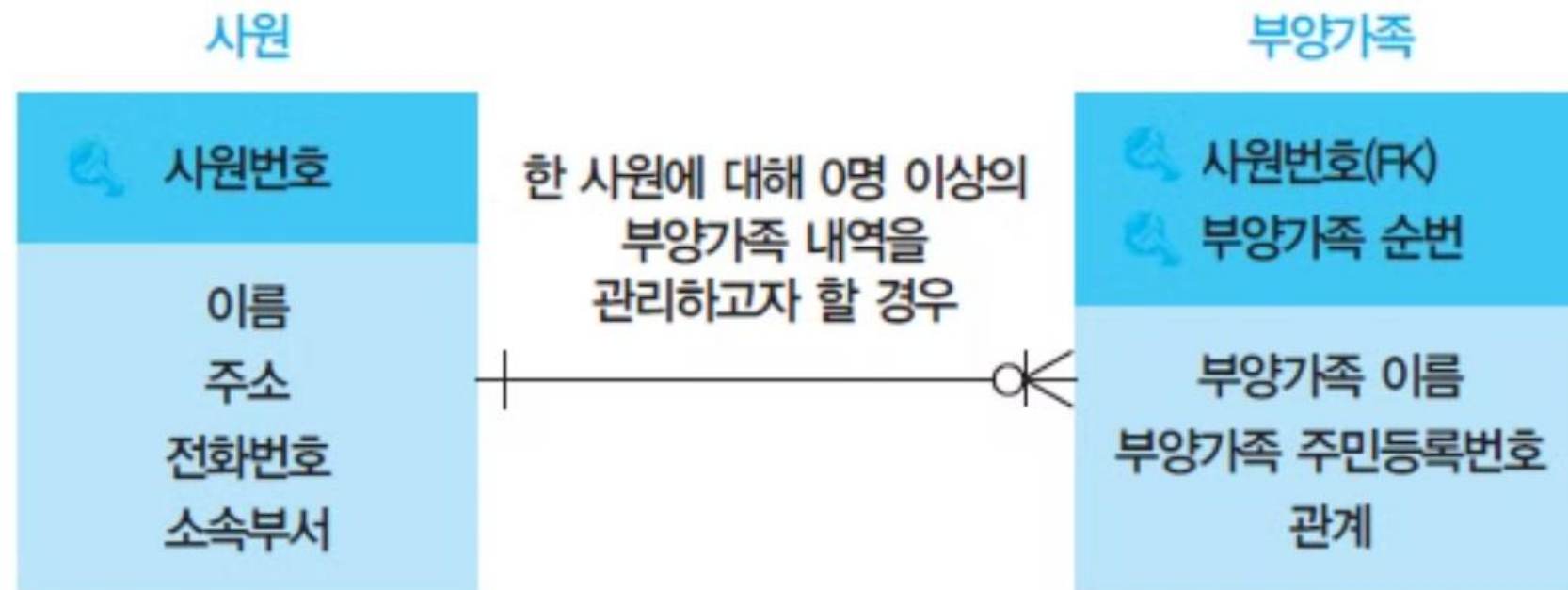
속성 식별

2

테이블 생성

3

관계 설정



다중 값 속성은 별도의 테이블로 변환합니다. 이는 데이터 무결성을 유지합니다.



ERD 테이블 변환 실습

단계	설명
1	M:M 관계 해결
2	일반화 관계 해결
3	데이터 모델 통합 고려
4	정규화 위배 속성 확인

ERD를 테이블로 변환하는 실습을 통해 이해를 높입니다.
M:M 관계, 일반화 관계 등을 해결합니다.

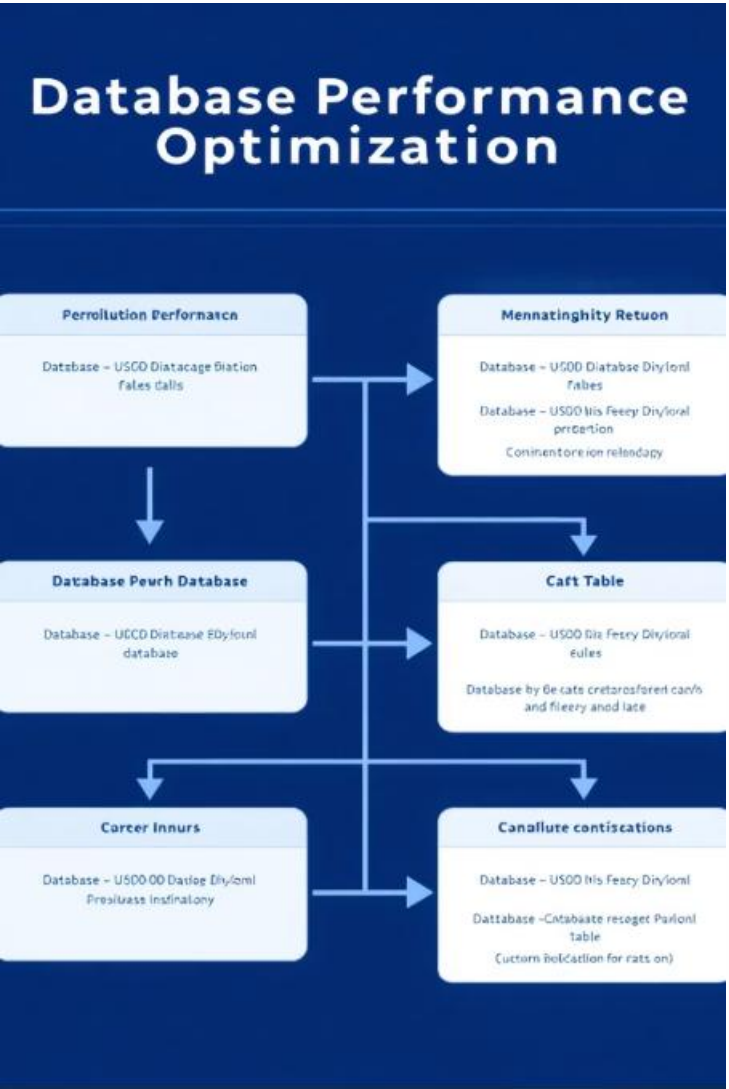
논리적 설계: 학과 시스템

1. M:M 해결
2. 다중 값 해결
3. 슈퍼-서브 타입 전개
4. 정규화 검증

논리적 설계 실습: e-마켓플레이스

1. M:M 해결
2. 다중 값 해결
3. 슈퍼-서브 타입 전개
4. 정규화 검증

성능을 고려한 데이터베이스 설계



효율적인 데이터베이스 구조를 통해 시스템 성능을 극대화하는 방법을 알아봅니다. 중복 데이터와 집계 데이터를 활용한 테이블 구조 재정의 기법을 통해 실제 성능 향상을 경험해 보세요.

1. 성능 향상을 위한 테이블 구조 재정의 고려사항
2. 중복 데이터를 추가한 테이블 구조 재정의
3. 집계 데이터를 추가한 테이블 구조 재정의
4. 성능을 고려한 ERD 재조정 실습

성능 향상을 위한 테이블 구조 재정의

다른 튜닝 메커니즘으로 해결되지 않는 경우

인덱스나 SQL 튜닝으로 해결할 수 없는 성능 문제가 있을 때 구조 재정의의 고려합니다.

테이블 구조 재정의의 장점이 많은 경우

구조 변경으로 인한 성능 향상이 유지보수 비용보다 크게 높을 때 재정의합니다.

데이터 무결성 손실 최소화

구조 변경 시 데이터 일관성과 정확성을 유지하는 방향으로 설계합니다.

테이블 구조변경에 대한 이력 관리

변경 사항을 문서화하고 추적하여 향후 관리와 유지보수를 용이하게 합니다.

시스템 복잡성과 오버헤드 고려

재정의로 인한 시스템 복잡도 증가와 성능 간의 균형을 신중히 평가합니다.

중복 데이터 추가한 테이블 구조 재정의



원본 테이블 구조

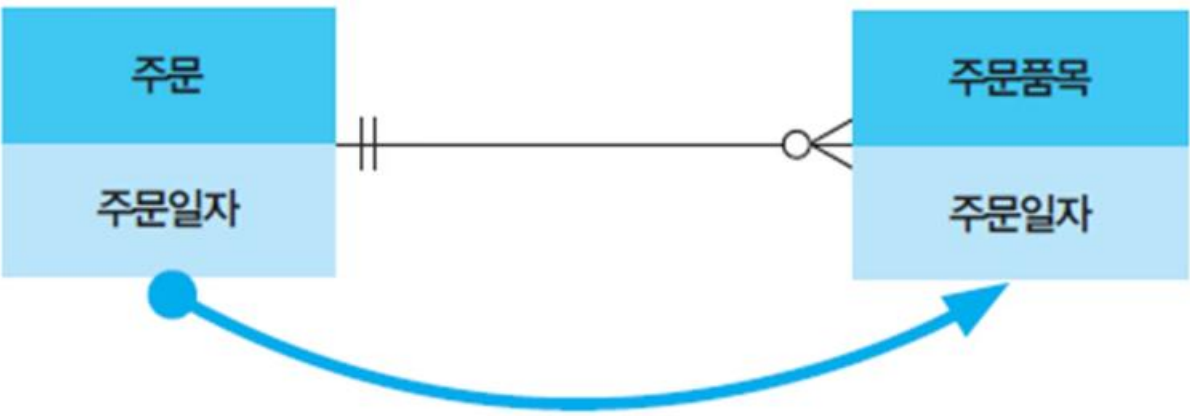
정규화된 상태의 테이블 구조입니다.
조인이 필요한 분리된 테이블들로 구성됩니다.

컬럼 복사

부모-자식 관계에서 필요한 컬럼을 복제합니다.
자주 조회되는 데이터를 중복 저장합니다.

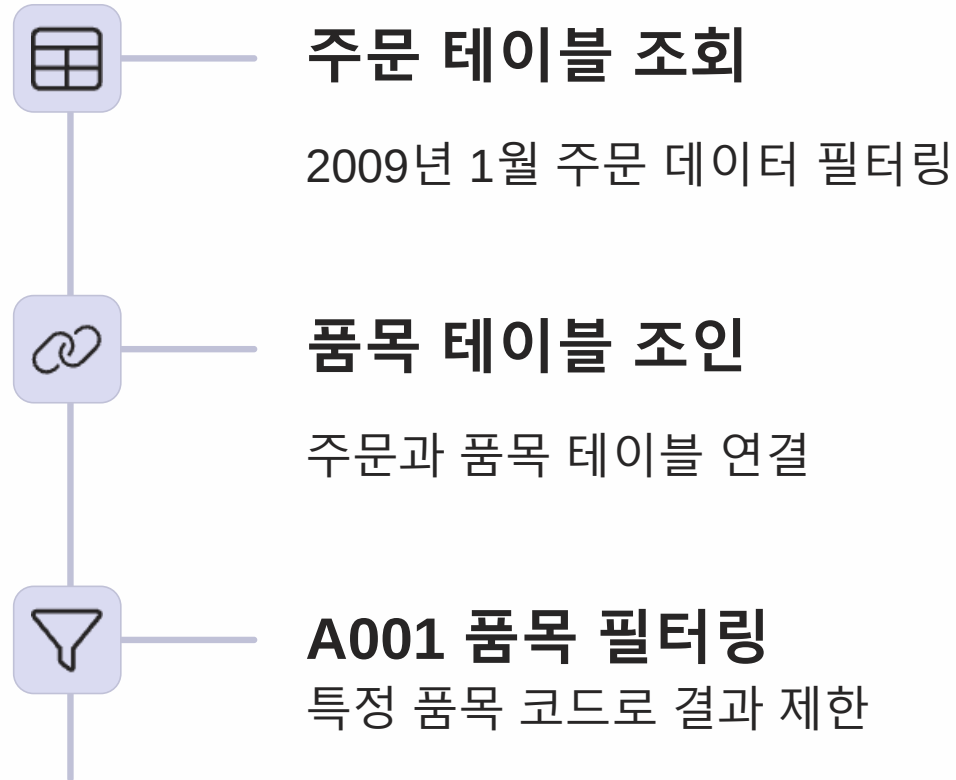
최적화된 구조

조인 감소로 쿼리 성능이 향상됩니다.
데이터 중복성 관리가 필요합니다.



중복 데이터 추가는 컬럼의 복사를 통해
부모와 자식 관계의 테이블 간 조인을 줄입니다.

중복 데이터 활용 쿼리 최적화



| 주문일자가 '2009년 1월'인 주문 |

주문일자	주문번호
20090101	200901001
20090102	200901002
20090102	200901003
20090103	200901004
20090104	200901005
.....	
20090201	200901001

| 품목번호가 'A001'인 주문품목 |

품목번호	주문번호
A001	200901001
A001	200901002
A001	200902001
A001	200902002
A001	200902003
.....	
A002	200901001

2009년 1월에 주문한 모든 품목을 조회한 후 그 중에서 'A001' 품목만 결과로 가져옵니다.

이 프로세스는 대량의 데이터를 처리할 때 성능 저하를 일으킬 수 있습니다.

중복 데이터 추가한 테이블 구조 재정의



주문월별 품목별 자료 정렬

주문 데이터를 월별, 품목별로 정렬하여 효율적인 접근을 제공합니다.



2009년 1월의 A001 품목 주문현황 조회

특정 기간과 품목에 대한 빠른 데이터 검색이 가능합니다.



주문번호와 품목테이블 최적화

자주 조회되는 패턴에 맞게 데이터 구조를 최적화했습니다.

중복 데이터를 활용하여 특정 조회 패턴에 최적화된 테이블 구조를 구현했습니다.

주문월	주문품목
200901	A001
200901	A001
200901	A001
200901	A001
200901	A001
200901	A001
200901	B001
200901	B001
200901	B001

1:1 관계 테이블 최적화

원본 구조

1:1 관계의 두 테이블

- 각 테이블에 분리된 데이터
- 조인이 필요한 쿼리 구조
- 정규화된 상태

최적화 방법

기본키가 아닌 컬럼 복사

- 자주 사용되는 컬럼 식별
- 선택적 컬럼 복제
- 데이터 중복 허용

최적화 결과

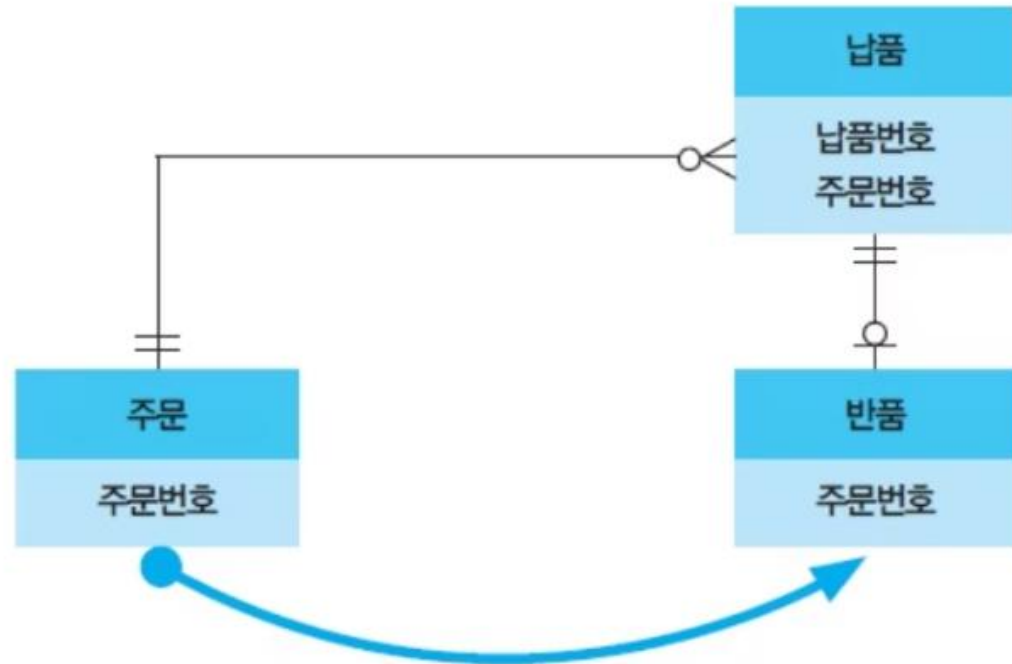
조인 없이 데이터 접근

- 쿼리 성능 향상
- I/O 감소
- 응답 시간 개선



1:1 관계의 테이블에서 기본키가 아닌 컬럼을 복사하여 조인 없이 데이터에 접근할 수 있습니다.

직접적 관계가 없는 테이블 최적화



직접적인 관계가 없는 테이블 간의 조인을 줄이기 위해 외부키를 복사합니다.
이를 통해 복잡한 다중 조인을 단순화하고 쿼리 실행 속도를 향상시킵니다.

원본 테이블 구조

직접적 관계가 없는 테이블들

외부키 식별

자주 사용되는 연결 키 분석

외부키 복사

필요한 테이블에 외부키 추가

최적화된 구조

직접 접근 가능한 테이블 구조

컬럼 복사를 위한 고려사항

빈번한 조인 발생 시

조인의 범위가 다량인 경우를 온라인화해야 하는 경우 컬럼 복사를 고려합니다.

자주 사용되는 액세스 조건

액세스 조건이 다른 테이블에 분산되어 있어 조건 부여에도 불구하고 범위를 줄이지 못하는 경우 컬럼 복사가 유용합니다.

컬럼 도메인 유지

복사된 컬럼의 도메인은 원 컬럼과 동일하게 유지하여 데이터 일관성을 보장해야 합니다.

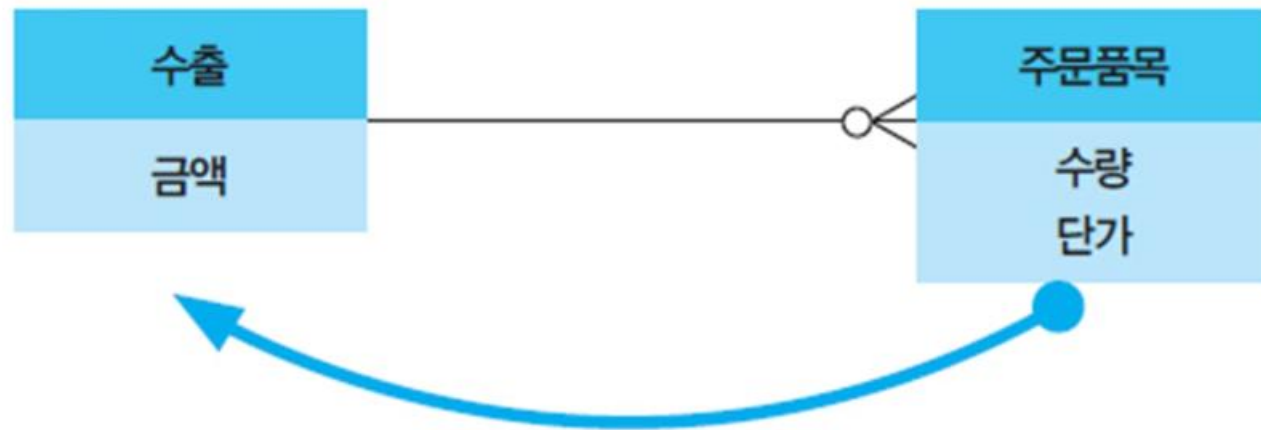
계층적 복사 전략

상위 레벨 테이블에 집계된 컬럼 추가(1:M 관계) 또는 하위 레벨 테이블로 중복 컬럼 복사(1:M 관계)를 선택적으로 적용합니다.

컬럼 복사는 액세스 경로의 단축을 위해 사용되며, 성능 향상과 데이터 무결성 사이의 균형을 고려해야 합니다.



컬럼 추출 기반 테이블 최적화



자주 조회되는 컬럼만 추출하여 별도의 테이블로 구성하면 데이터 접근 속도를 크게 향상시킬 수 있습니다.

이 방식은 대용량 테이블에서 특히 효과적입니다.

반복 그룹의 컬럼화 최적화

반복 그룹 식별

여러 행으로 표현된 유사 데이터 패턴

컬럼화 설계

행 데이터를 컬럼으로 변환하는 구조 설계

구조 변환

반복 그룹을 컬럼으로 재구성

연월	실적
1	
2	
3	
4	
⋮	
12	

→

연도	1월 실적	2월 실적		12월 실적
----	-------	-------	-------	--------

이 방식은 특히 고정된 개수의 반복 그룹이 있는 경우 효과적입니다.

중복 데이터를 추가한 테이블 구조 재정의 중 하나로, 특정 조회 패턴에 최적화된 성능을 제공합니다.

반복되는 행 데이터를 컬럼으로 변환하여 데이터 접근성을 향상시키는 기법입니다.

반복그룹 컬럼화의 장단점

컬럼으로 중복하는 경우의 장점

- DML의 조작이 용이해집니다
- 인덱스 활용이 효율적입니다
- 해시, 클러스터 등을 사용한 튜닝이 용이합니다
- 데이터 접근 속도가 향상됩니다
- 특정 데이터만 빠르게 조회할 수 있습니다

컬럼으로 중복하는 경우의 단점

- 기억장소의 낭비를 초래합니다(키 항목 관련)
- Row 수의 증가로 인해 성능이 저하될 수 있습니다
- 반복 그룹이 한 Row의 형식으로 보여져야 할 경우 구현이 복잡합니다
- 데이터 일관성 유지에 추가 작업이 필요합니다
- 변경 시 모든 중복 데이터 업데이트가 필요합니다

반복그룹의 컬럼화는 특정 상황에서 큰 성능 이점을 제공하지만, 저장 공간과 데이터 관리 측면의 단점도 고려해야 합니다.

집계 테이블 추가한 테이블 구조 재정의



집계 테이블 추가 고려사항

효율적인 집계 테이블 설계를 위한 핵심 요소

- 지나친 집계 테이블을 만들지 않습니다
- 여러 테이블 조인이 빈번한 경우 고려합니다
- M:M 관계 처리 과정을 추적할 때 유용합니다
- 복잡한 검색조건이 있는 경우 효과적입니다



응용 프로그램 활용

기존 시스템과의 통합 방안

- 추가된 집계 테이블을 기존 응용 프로그램이 이용할 수 있는지 확인합니다
- 시스템 변경 없이 활용 가능한 설계를 지향합니다



데이터 일관성

트리거 오버헤드와 일관성 관리 방안

- 데이터베이스 트리거 오버헤드에 주의합니다
- 데이터 일관성을 보장하는 메커니즘을 구현합니다

집계 테이블은 데이터 양이 적절하고 활용도가 좋아지도록 기본키를 선정하여 재정의해야 합니다.

집계 테이블 추가 방법론



특정 부분만 포함하는 테이블 추가

넓은 범위로 자주 처리되며 특정한 컬럼들만 자주 사용되는 경우 최적화합니다.

예: 주문 테이블에서 주문번호, 주문일자, 총액만 포함된 주문요약 테이블을 생성하여 보고서 쿼리 속도 향상



자주 사용되는 컬럼 분석

특정 부분만 주로 사용되는 경우를 파악하고 해당 컬럼을 집중 분석합니다.

예: 고객 테이블에서 이름, 등급, 가입일만 자주 조회된다면 이 세 컬럼으로 구성된 고객요약 테이블 구성



보고서 최적화

보고서 양식을 위해 경우에 따라 코드 명칭만 모은 테이블을 추가합니다.

예: 상품코드-상품명-카테고리명을 한 테이블에 모아 복잡한 조인 없이 보고서 생성 가능하도록 설계



테이블 재정의 유형

테이블을 분할하거나 두 개의 테이블을 조합해서 하나의 테이블로 관리하는 방법을 선택합니다.

예: 회원-주문 테이블을 조합하여 회원별 주문통계 테이블 생성 또는 대용량 로그 테이블을 기간별로 분할

집계 테이블 추가는 데이터 접근 패턴에 따라 전략적으로 적용해야 하며, 테이블 구조의 복잡성과 성능 향상 사이의 균형을 고려해야 합니다.

테이블 구조 재정의

구분	테이블 분할	테이블 조합
적용대상	· 다른 사용자 그룹이 테이블 내의 특정 부분집합에만 지속적으로 접근하는 경우 · 요구사항이 다중 테이블을 읽어야 하는 일이 드문 경우 · 다중 테이블에 동시 입력·수정이 없는 경우	· 1:1 관계의 테이블이 동시에 이용되는 경우 · 같은 상부 유형을 공유하는 하부 유형 테이블 · 거의 동일한 컬럼을 가지는 별도의 테이블인 경우 · 별도로 접근하는 경우가 거의 없는 경우
장점	· 스캔 처리시 액세스의 양이 감소 · 테이블 관리가 용이 (Backup, Recovery, 재생성) · 보안유지가 용이	· 데이터 액세스가 간편 · 뷰(View)를 활용하여 별도의 테이블처럼 사용 · 부분범위 처리로 유도 가능 · 복잡한 처리를 하나의 SQL로 통합 용이
단점	· 테이블 구분 없이 전체적인 데이터를 처리해야 하는 경우 Unjoin/Join 발생 · 처리속도가 증가하는 경우 발생 · 트랜잭션 처리시 여러 테이블을 동시에 처리하는 경우가 증가 · 복잡한 처리의 SQL 통합이 어려워짐 · 부분범위 처리가 불가능해질 수 있음 · 기본 키의 유일성 유지가 어려움	· Row 수가 증가하여 스캔 처리시 처리량 증가 · 입력, 갱신, 삭제 규칙이 복잡해질 수 있음 · Not Null, Default, Check 등의 Constraint를 완벽하게 지정하기 어려워짐 · 액세스 경로의 조건 증가

집계 테이블 성능 향상 사례

실제 구현 사례

- 금융 시스템: 거래 내역 집계 테이블 도입으로 일일 결산 처리 시간 42% 감소
- 물류 관리: 배송 상태 요약 테이블 구현으로 실시간 조회 응답 시간 28% 개선
- 이커머스 플랫폼: 주문 상세 테이블에서 주문 요약 테이블을 분리하여 보고서 생성 시간 35% 단축

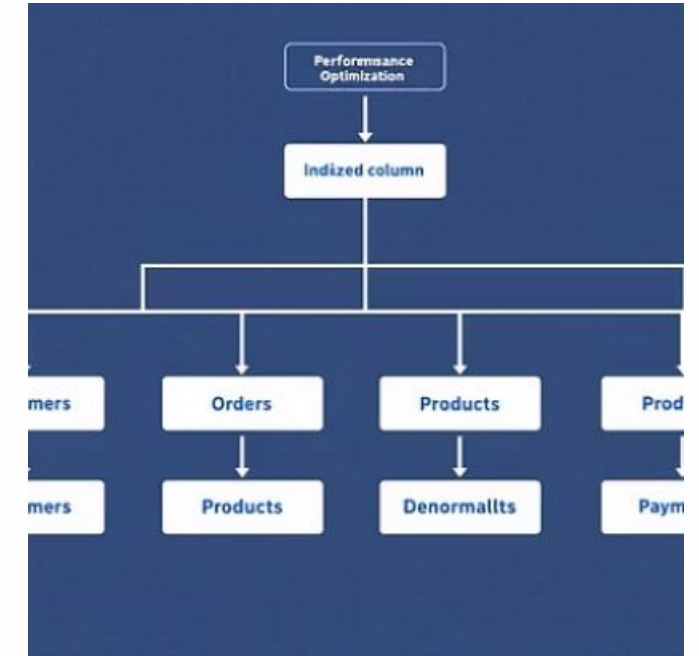
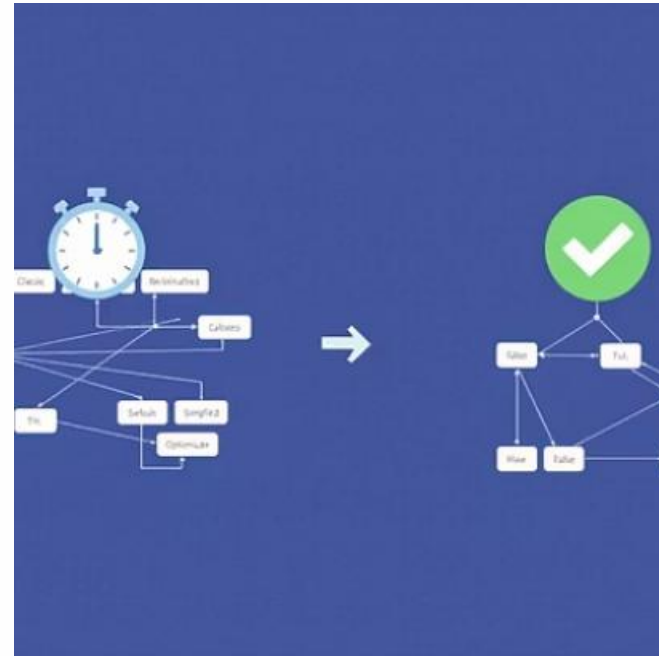
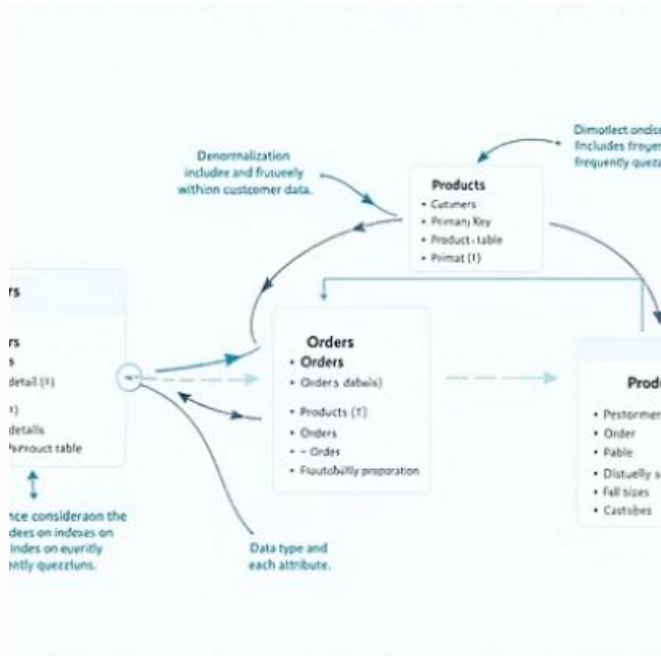
기대 효과 및 주의점

집계 테이블 추가를 통해 평균 20%의 쿼리 성능 향상을 기대할 수 있습니다.

데이터 양과 쿼리 복잡성에 따라 최적의 테이블 유형과 최적화 방식을 선택해야 합니다.

성능 향상은 시스템 환경, 데이터 특성, 쿼리 패턴에 따라 달라질 수 있습니다.

성능을 고려한 ERD의 재조정 실습



실제 ERD를 성능 중심으로 재조정하는 실습을 통해 이론을 실제로 적용해 봅니다.

테이블 간의 관계, 중복 데이터 추가, 집계 테이블 구성 등을 종합적으로 고려하여 최적화된 데이터베이스 설계를 완성합니다.

실습 결과를 분석하여 성능 향상 효과를 측정하고 개선점을 도출합니다.

물리적 데이터베이스 설계와 최적화

데이터베이스의 물리적 설계와 성능 최적화 방법에 대해 알아보겠습니다.
효율적인 설계를 통해 응답시간 최소화, 저장 공간의 효율화, 트랜잭션 처리량
증가를 달성하는 방법을 소개합니다.

Database Physical Design Optimization

is database optimizations of in tinctration design physica the lieme actualal sizes in terogated of the ubaloes form the
mab a upsign precat and oletiplatatins suononation logianing your and a ociemancs deening. Tels and equiplance to
dis your iandrsiert for the databsags.

is scal ctur the cand onlt of the rernent.

Payrical Payticut Test (PAY PRG UP)



Practure Performance Metrics



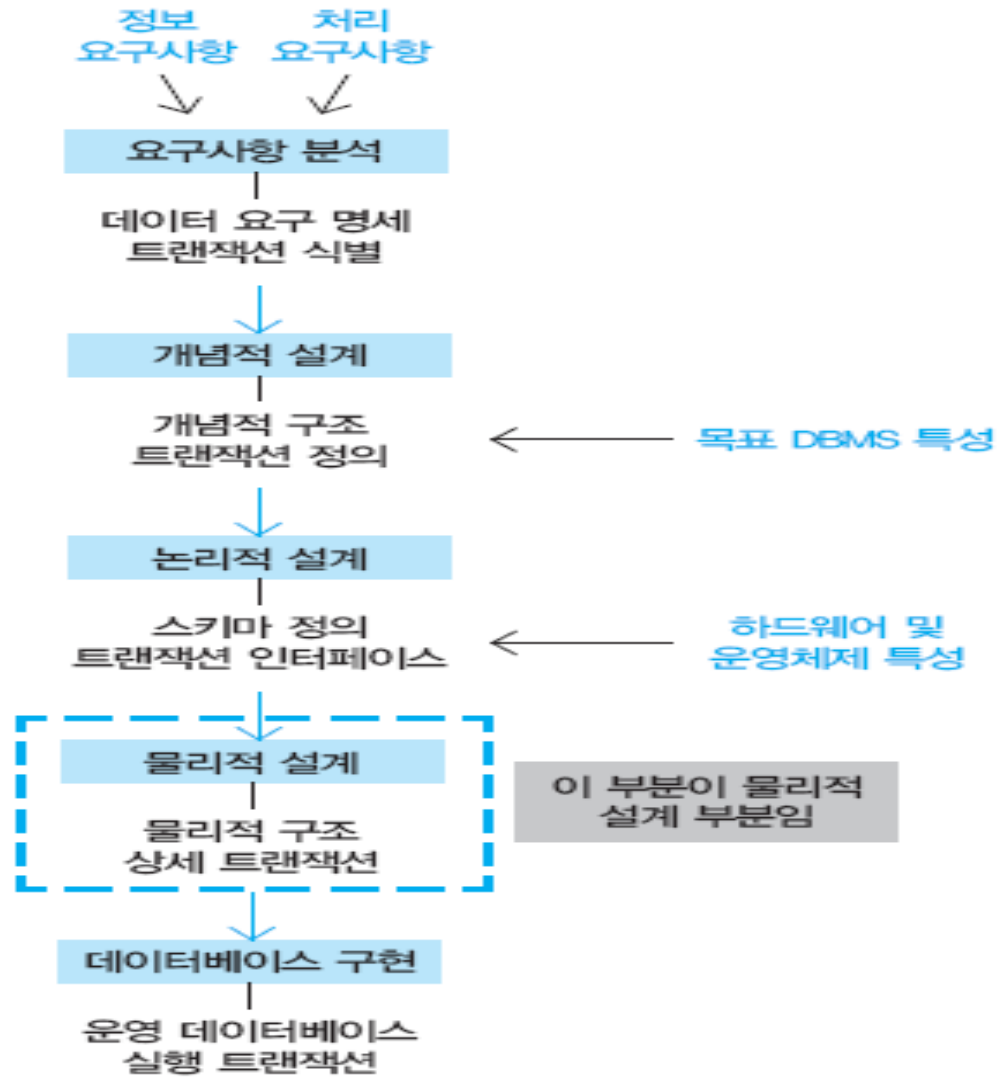
Improrvainge Founatiny Metrics

Uiver canistared in AUSE
sayiploc dent danc ionterete war
cotetes and overloun engon nant,
eccenstetans ovations

\$, 17P9, 500 Database Pberitces
Onk T1B for canto p rreslon a of in the
arphory and in conrt inatims

\$110, Colliorer By \$1500 M TEIS
8x. 079, 5TIX KOC, 00M teE6

데이터베이스 구축 과정



응답시간 최소화

사용자 요청에 대한 빠른 응답 보장



저장 공간의 효율화

데이터 중복 최소화 및 자원 효율적 활용



트랜잭션 처리량 증가

시스템의 전반적인 처리 능력 향상

물리적 설계 과정에서는 위 세 가지 요소를 균형 있게 고려해야 합니다. 적절한 균형점을 찾는 것이 성공적인 데이터베이스 구축의 핵심입니다.

성능을 위한 테이블 구성 - 테이블 유형

정규 테이블

가장 일반적으로 사용되는 기본 테이블 형태로 데이터를 행과 열 형태로 저장합니다.

파티션 테이블

하나 이상의 파티션을 가지며 각 파티션은 지정된 범위의 키값을 갖는 행들을 저장합니다.

클러스터 테이블

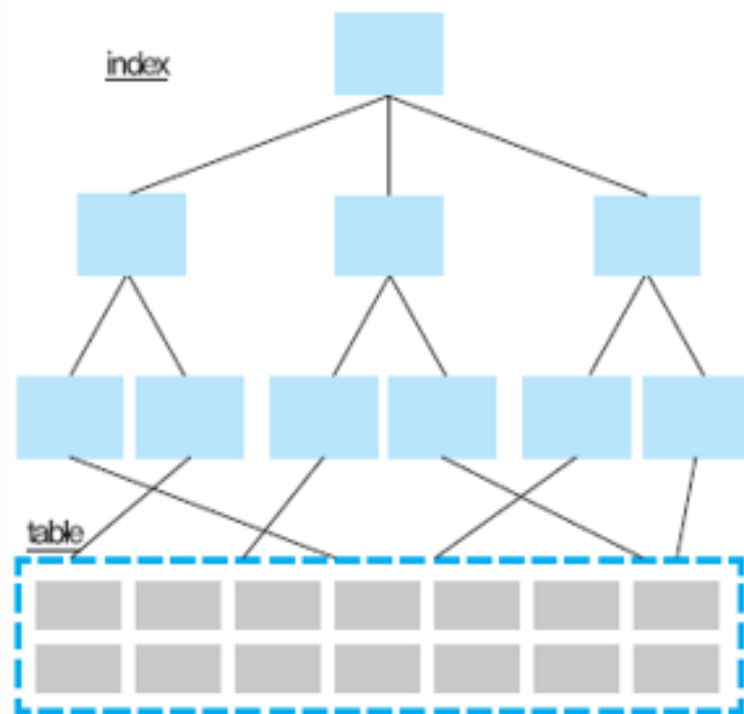
지정된 클러스터 키 칼럼값의 순서대로 행을 저장시키는 방법을 사용합니다.

IOT (Index-Organized Table)

모든 테이블 데이터를 B-트리 구조로 저장하여 접근 속도를 향상시킵니다.

일반 테이블과 IOT의 저장 구조 비교

| 일반 테이블 |

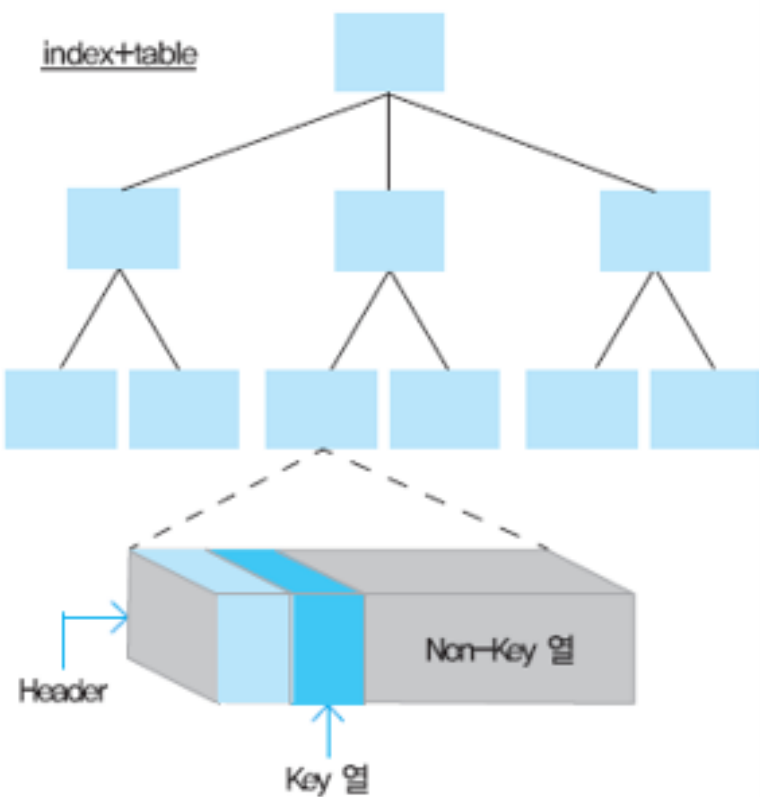


일반 테이블 구조

데이터는 힙 구조로 저장되며, 별도의 인덱스를 통해 접근합니다.

ROWID를 통한 직접 접근이 가능하며, 데이터와 인덱스가 분리되어 있습니다.

| Index Organized Table |



IOT 구조

데이터가 B-트리 인덱스 구조 자체에 저장됩니다
키 값에 따라 데이터가 정렬되어 저장되므로 특정 범위 검색에 효율적입니다.



일반 테이블과 IOT 상세 비교

구분	일반 테이블	IOT
식별 방식	유일 식별자 또는 ROWID로 식별	기본키로 식별
ROWID	암시적 ROWID 여러 인덱스 지원	ROWID 없음
전체 검색	FTS시 Random하게 Return	FTS시 기본키 순서로 Return
제약 조건	유일 제약 조건 허용	유일 제약 조건 허용하지 않음
구조	인덱스와 테이블로 구분됨	인덱스 구조를 가진 테이블임

파티션(Partition) 테이블 구조



파티션의 개념

대용량 테이블을 여러 개의 작은 물리적 조각으로 나누어 관리하는 기법입니다.



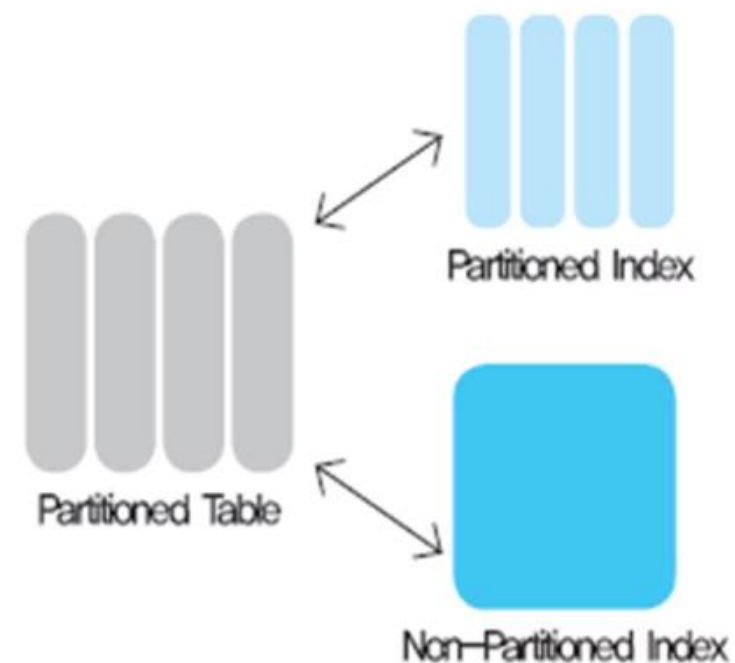
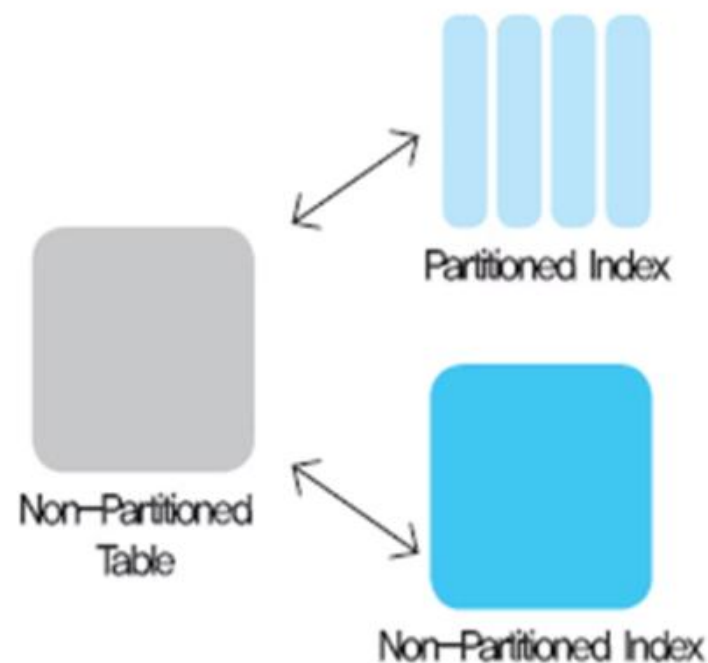
성능 향상

검색 시 관련 파티션만 접근하여 처리 속도를 높이고 불필요한 I/O를 줄입니다.



관리 용이성

파티션 단위로 백업, 복구, 로드 작업이 가능하여 관리가 편리합니다.



FTS(Full Table Scan) 검색 방법



테이블 전체 읽기

테이블의 모든 데이터 블록을 순차적으로 읽습니다.



조건 필터링

모든 행에 대해 조건을 적용하여 필터링합니다.



결과 반환

조건에 맞는 행만 결과로 반환합니다.

EMP 테이블

EMPNO	ENAME	JOB
7369	SMITH	CLERK
7698	BLAKE	MANAGER
7566	JONES	MANAGER
7654	MARTIN	SALESMAN
7782	CLARK	MANAGER
7499	ALLEN	SALESMAN
7521	WARD	SALESMAN

결과

EMPNO	ENAME	JOB
7698	BLAKE	MANAGER
7566	JONES	MANAGER
7654	MARTIN	SALESMAN
7521	WARD	SALESMAN

FTS는 대량의 데이터를 읽어야 할 때 효율적이지만, 소량의 데이터만 필요한 경우에는 성능이 저하될 수 있습니다.

인덱스 스캔 검색 방법

- 1

인덱스 접근

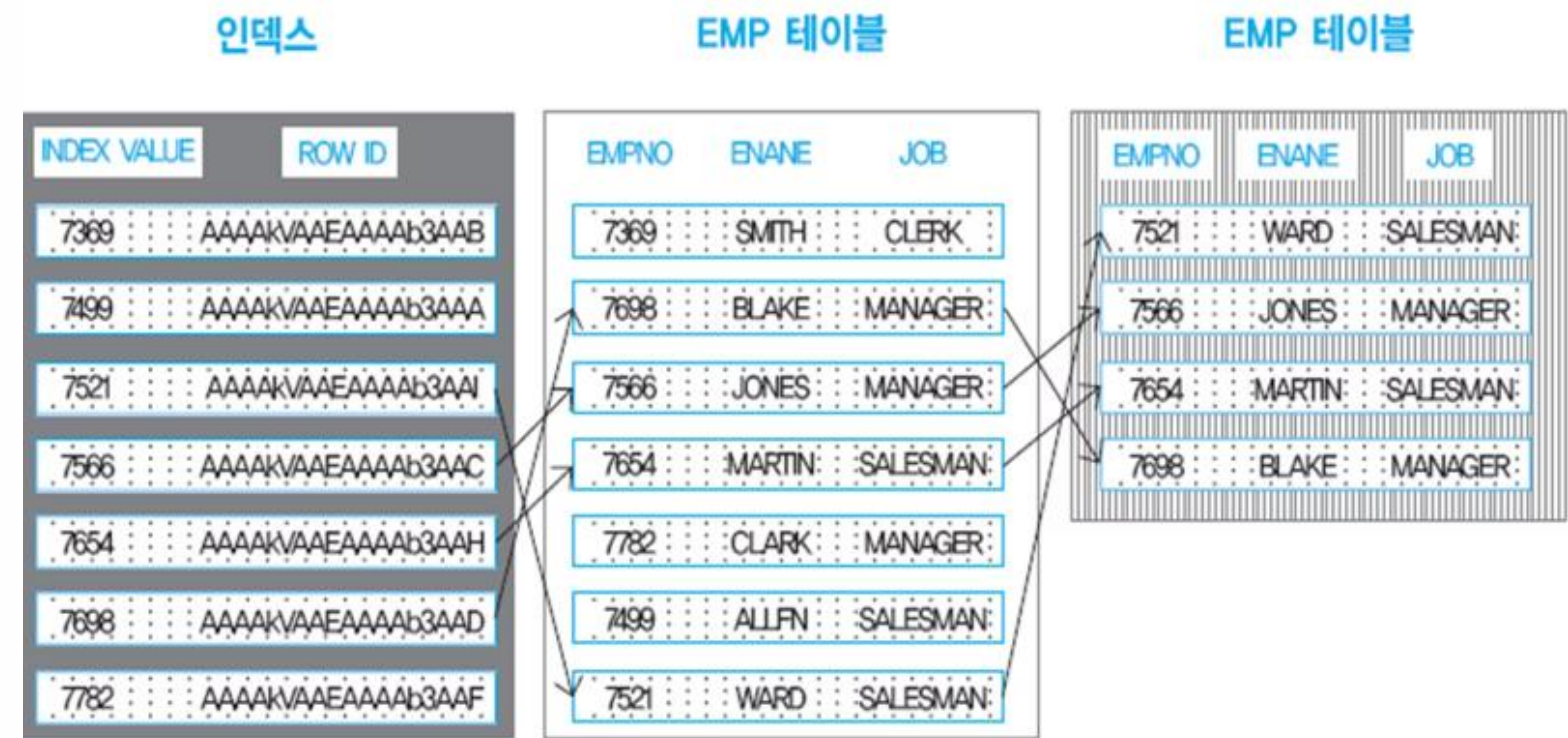
조건에 맞는 인덱스 키를 찾아 B-트리 구조를 탐색합니다.
- 2

ROWID 획득

인덱스에서 해당 데이터의 ROWID 정보를 얻습니다.
- 3

테이블 접근

ROWID를 통해 테이블의 특정 데이터에 직접 접근합니다.



인덱스 스캔은 전체 테이블의 일부만 검색할 때 효율적이며, 적절한 인덱스 설계가 성능에 큰 영향을 미칩니다.

인덱스의 논리적 구분

단일/결합 인덱스

단일 컬럼 인덱스는 하나의 컬럼만 사용하고, 결합 인덱스는 여러 컬럼을 함께 사용합니다.

유일성 인덱스

유일한(Unique) 인덱스는 중복값을 허용하지 않고, Non-Unique 인덱스는 중복을 허용합니다.

함수기반 인덱스

컬럼에 함수를 적용한 결과값으로 인덱스를 생성하여 특수한 검색 패턴에 최적화합니다.

적절한 인덱스 유형 선택은 쿼리 패턴과 데이터 특성을 고려하여 결정해야 합니다.

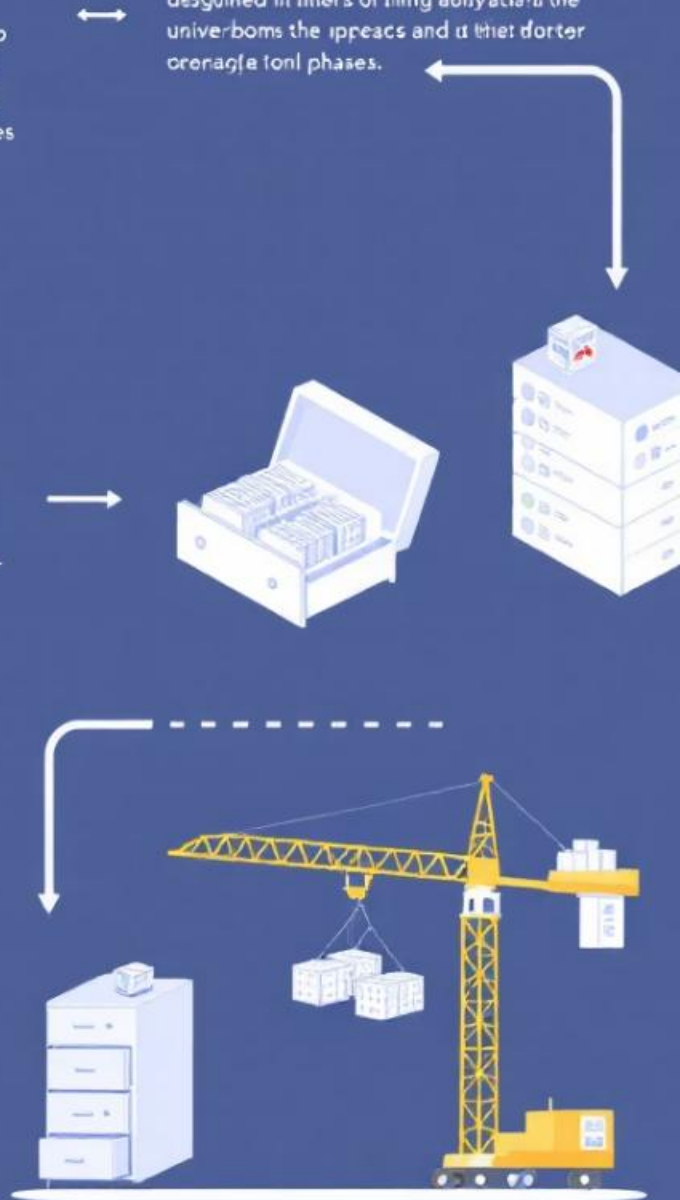
Database Physical Design

1. Performance optimization
performance optimization. Illustration
performance optimization for a crown to
today the for this data target of filled
having an entire ment is sophisticated
denaaged to needdilly inat docks
data stages.

A banghous perlization in clarather's
desguined in mters of filling aollyatib the
univerboms the appeacs and it thet dotor
orenagfe tonl phases.

2. Normalization
a well-organized filling cabinet
callied capn an at uabied in the
imporized acoes and inabding to ties of
explereang and freeclonges and bljvice.

3. Implementation phases
Los a imolization parsons ant data
eany the preeced in the dabas its
catlorate pnciets cereclaren. proant
your need an the bean ation on a
crecfile dote if the dact gffer, desical
tolocks.



성능을 고려한 물리적 설계 및 데이터베이스 구축



성능을 위한 선행 조건

데이터베이스 정규화, 인덱스 최적화, 성능 분석 방법을 준비합니다.



반정규화를 통한 성능향상

필요한 경우 테이블 통합이나 분할을 통해 성능을 개선합니다.



PK (Primary Key) 작성

효율적인 기본키 설계로 데이터 접근 성능을 향상시킵니다.



데이터베이스 구축

설계된 내용을 바탕으로 실제 데이터베이스를 구현합니다.

Checklist Database Performance Optimization



Normalization



Indexing



Analysis Tools



성능을 위한 선행 조건



데이터베이스 정규화

적절한 정규화를 통해 데이터 중복을 제거하고 일관성을 유지합니다.



인덱스 최적화

좁게 구성한 인덱스와 클러스터 인덱스를 적절히 활용하여 성능을 개선합니다.



값의 행 유일성 검사

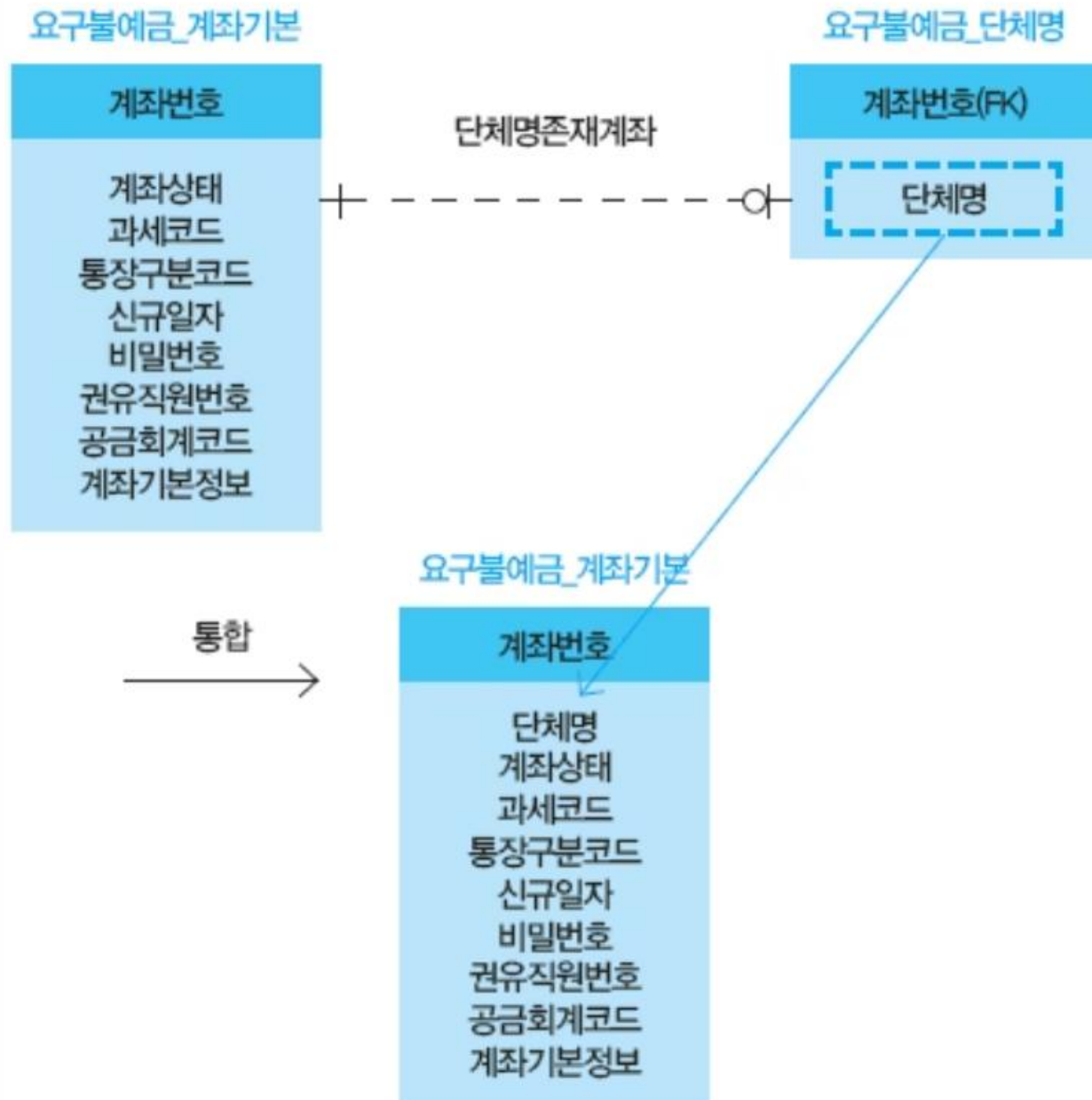
중복 데이터를 방지하여 데이터 품질과 검색 성능을 향상시킵니다.



성능 저하 분석

다양한 분석 도구를 활용하여 성능 병목 현상을 식별하고 해결합니다.

동일한 기본키를 갖는 테이블의 통합



문제 인식

동일한 기본키를 가진 테이블 간 잦은 조인 발생

통합 설계

공통 기본키를 중심으로 테이블 통합

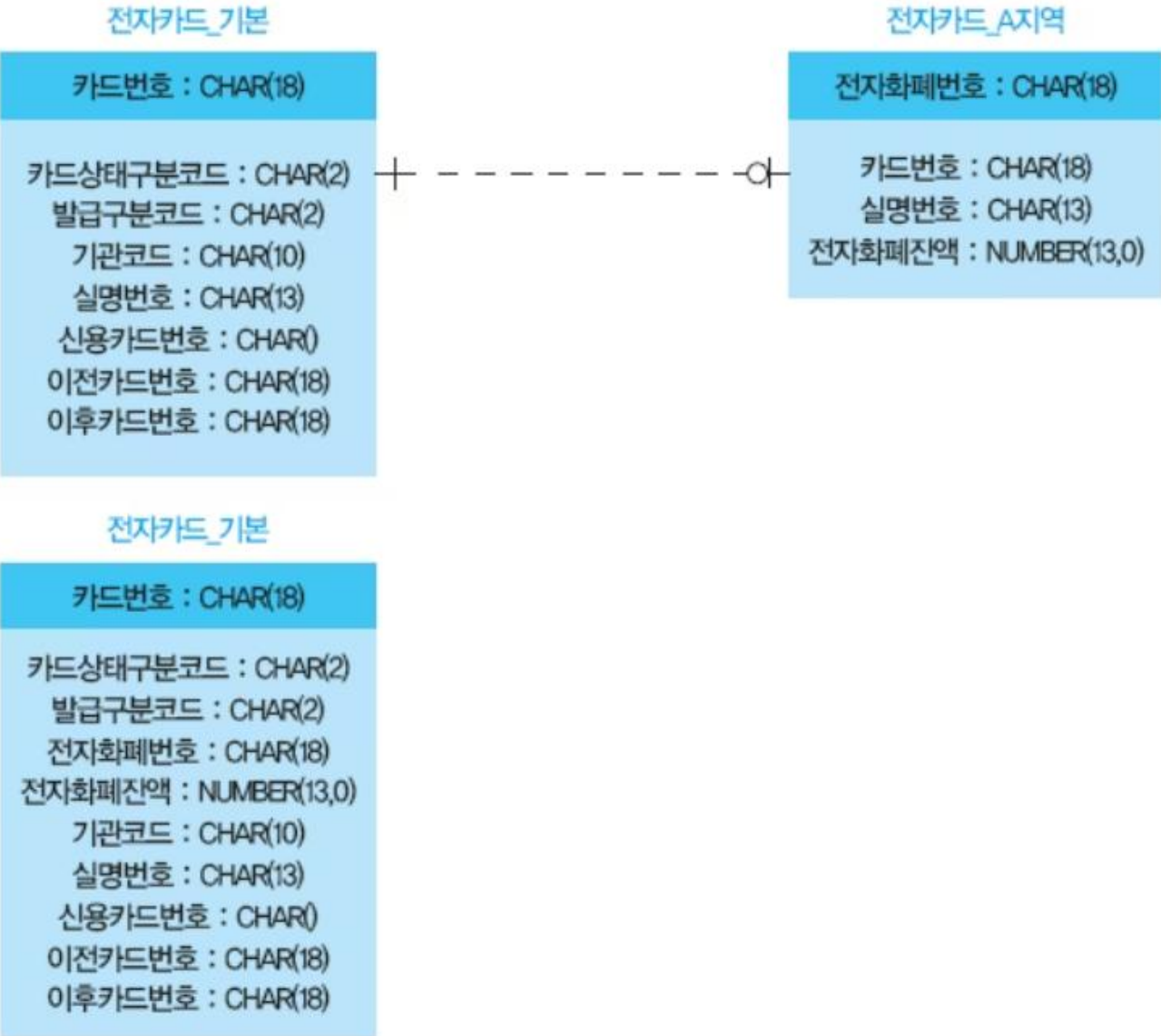
성능 향상

조인 연산 감소로 검색 속도 개선

동일한 기본키를 갖는 여러 테이블을 하나로 통합하면 조인 연산이 줄어 검색 성능이 향상됩니다.

특히 자주 함께 조회되는 테이블에 효과적입니다.

1:1 관계 테이블의 통합



통합 전

서로 다른 식별자를 가진 테이블이 1:1 관계로 연결되어 있습니다.
조회 시 항상 조인이 발생하여 성능 저하가 발생합니다.

통합 후

두 테이블의 모든 컬럼을 하나의 테이블로 통합합니다.
조인 없이 한 번의 테이블 접근으로 모든 데이터를 조회할 수 있습니다.
1:1 관계의 테이블은 통합을 통해 조인 오버헤드를 제거하고 데이터 접근 경로를 단순화할 수 있습니다.

유사 의미 테이블의 통합

유사 테이블 식별

의미가 유사하거나 속성의 성격이 비슷한 테이블을 식별합니다.

유형 구분

구분자 컬럼을 추가하여 각 레코드의 유형을 식별합니다.

구조 통합

공통 속성을 중심으로 테이블 구조를 통합합니다.

관리 단순화

유사한 여러 테이블을 하나로 관리하여 복잡성을 줄입니다.

전자카드_A가맹점이용실적

가맹점번호 : VARCHAR2(12)
거래년월 : CHAR(6)

월이용건수 : NUMBER(6)
월이용금액 : NUMBER(15,0)

전자카드_B가맹점이용실적

가맹점번호 : VARCHAR2(12)
거래년월 : CHAR(6)

월이용건수 : NUMBER(6)
월이용금액 : NUMBER(15,0)

전자카드_C가맹점이용실적

가맹점번호 : VARCHAR2(12)
거래년월 : CHAR(6)

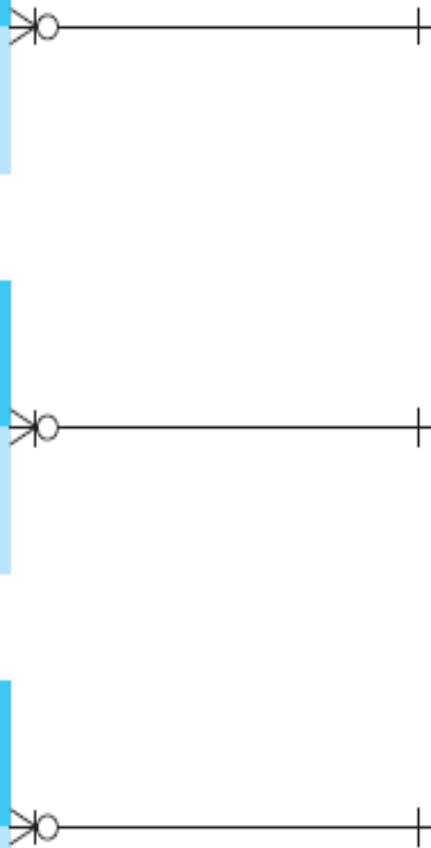
월이용건수 : NUMBER(6)
월이용금액 : NUMBER(15,0)

전자카드_가맹점기본

가맹점번호 : VARCHAR2(12)

가맹점구분코드 : CHAR(1)
해외이용여부 : CHAR(1)
전출여부 : CHAR(1)
전입여부 : CHAR(1)

가맹점전화번호 : VARCHAR2(14)
가맹점상호명 : VARCHAR2(40)
가맹점대표자명 : VARCHAR2(10)
가맹점자본 : NUMBER(15,0)
입금계좌번호 : CHAR(13)
투자계좌번호 : CHAR(13)
사업자등록번호 : VARCHAR2(10)
가맹점해지일자 : CHAR(8)
가맹점신규등록일자 : CHAR(8)



유사 의미 테이블 통합 사례

1

개별 테이블 관리

업체, 직원, 고객이 각각 별도의 테이블로 관리됩니다.

2

공통 속성 분석

세 테이블의 공통 속성과 특화된 속성을 구분합니다.

3

통합 테이블 설계

공통 속성을 기반으로 통합 테이블을 설계합니다.

4

구분자 추가

유형 구분자를 추가하여 레코드의 출처를 식별합니다.

전자카드_가맹점이용실적

가맹점번호 : VARCHAR2(12)
거래년월 : CHAR(6)
가맹점구분코드 : CHAR(1)

월이용건수 : NUMBER(6)
월이용금액 : NUMBER(15,0)



전자카드_가맹점기본

가맹점번호 : VARCHAR2(12)

가맹점구분코드 : CHAR(1)

해외이용여부 : CHAR(1)

전출여부 : CHAR(1)

전입여부 : CHAR(1)

가맹점전화번호 : VARCHAR2(14)

가맹점상호명 : VARCHAR2(40)

가맹점대표자명 : VARCHAR2(10)

가맹점자본 : NUMBER(15,0)

입금계좌번호 : CHAR(13)

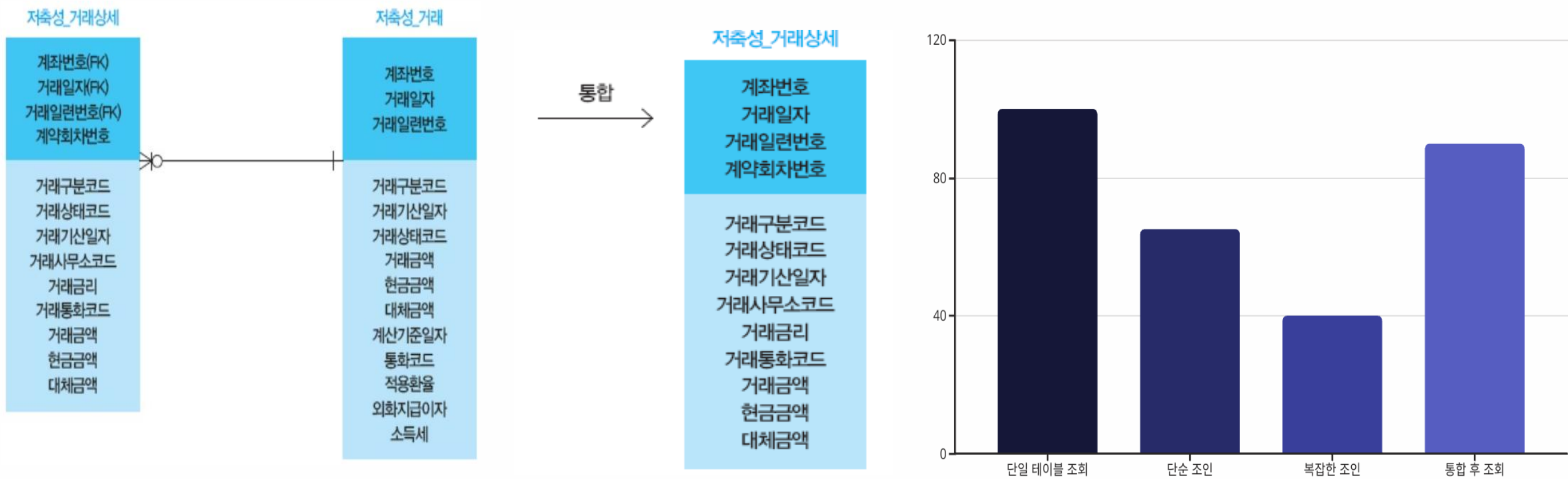
투자계좌번호 : CHAR(13)

사업자등록번호 : VARCHAR2(10)

가맹점해지일자 : CHAR(8)

가맹점신규등록일자 : CHAR(8)

자주 함께 사용되는 테이블의 통합



둘 이상의 테이블이 자주 함께 사용되는 경우 테이블을 통합하면 조인 연산이 제거되어 성능이 크게 향상됩니다.
통합 테이블은 단일 테이블 조회에 가까운 성능을 제공합니다.

테이블 분리 기준

1 객체 타입 컬럼

Object Type의 컬럼이 존재하는 테이블은 분리가 필요합니다.

3 수평 분할

기간에 따른 수평 분할로 데이터 접근 효율성을 높입니다.

5 보안 강화

보안이 필요한 민감한 데이터는 별도 테이블로 분리하여 관리합니다.

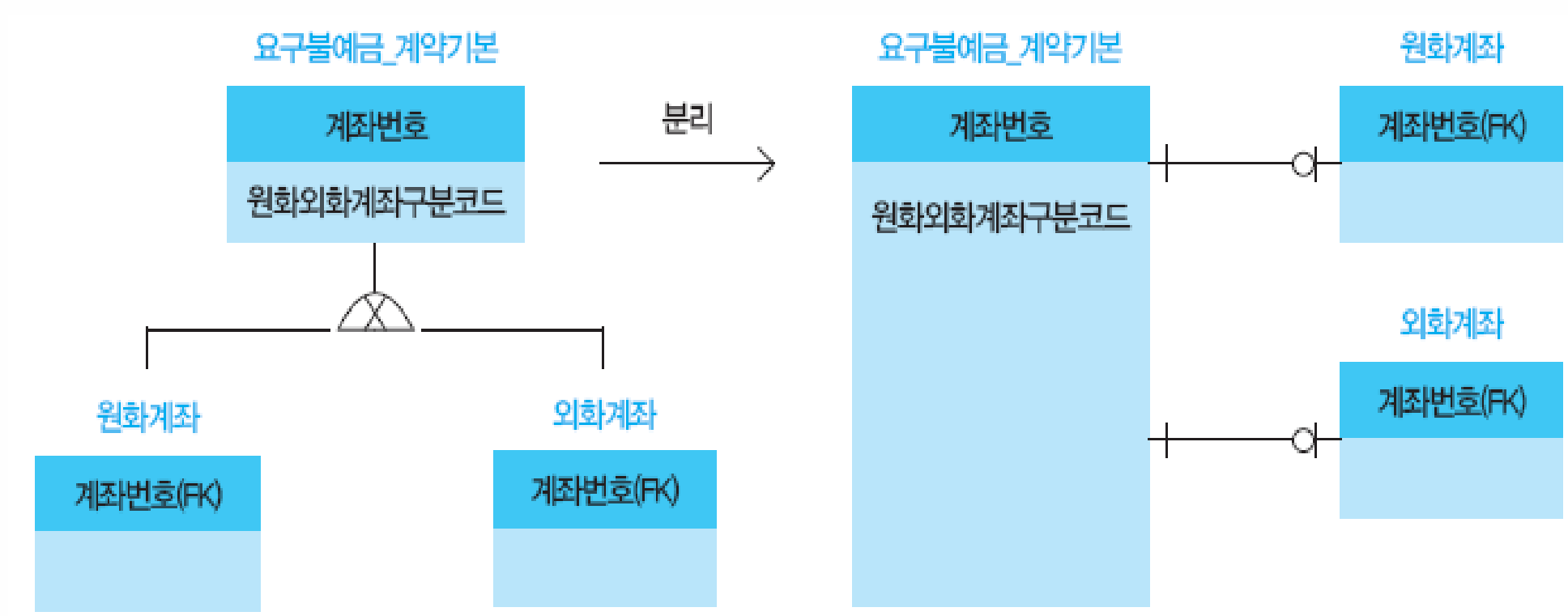
2 수직 분할

컬럼의 수가 많은 경우 수직 분할로 성능을 개선할 수 있습니다.

4 시스템 안정성

시스템과 데이터의 안정성을 확보하기 위해 테이블을 분리합니다.

수직 분할 테이블 구조



장점

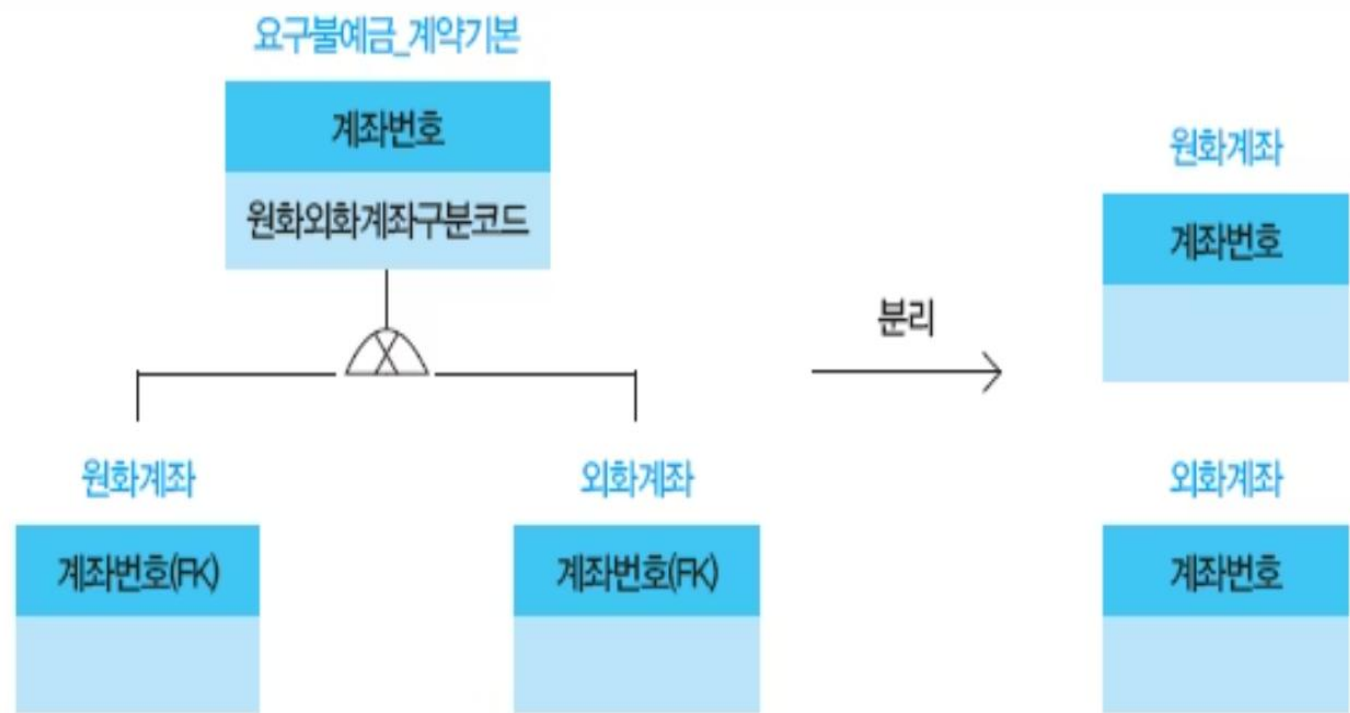
- 속도 향상: 필요한 컬럼만 조회하므로 I/O 성능 개선
- 보안 강화: 민감 정보 분리로 접근 제한 가능
- 관리 편리: 테이블이 가볍고 유지보수 쉬움



단점

- 조인 성능 저하: 나뉘진 테이블을 다시 합쳐야 함
- 트랜잭션 복잡: 여러 테이블에 걸쳐 작업해야 함
- 설계 어려움: 인덱스·ORM 등 설계 복잡성 증가

수평 분할 테이블 구조



수평 분할은 동일한 구조를 가진 데이터를 여러 테이블로 분산시켜 각 테이블의 크기를 줄이고 접근 성능을 향상시킵니다.



날짜 기준 분할

특정 날짜나 기간을 기준으로 데이터를 여러 테이블로 분할합니다.



지역 기준 분할

지역 코드나 위치 정보를 기준으로 데이터를 분리하여 관리합니다.



속성값 기준 분할

고객 유형이나 상품 카테고리 등 특정 속성값에 따라 테이블을 분할합니다.

Primary Key



기본 키(PK)의 특성과 중요성



변경되지 않는 속성

기본 키는 한번 설정되면 변경되지 않는 안정적인 컬럼이어야 합니다.



최소화 원칙

최소한의 컬럼으로 구성하여 효율성을 극대화해야 합니다.



유일성 보장

모든 레코드를 고유하게 식별할 수 있어야 합니다.



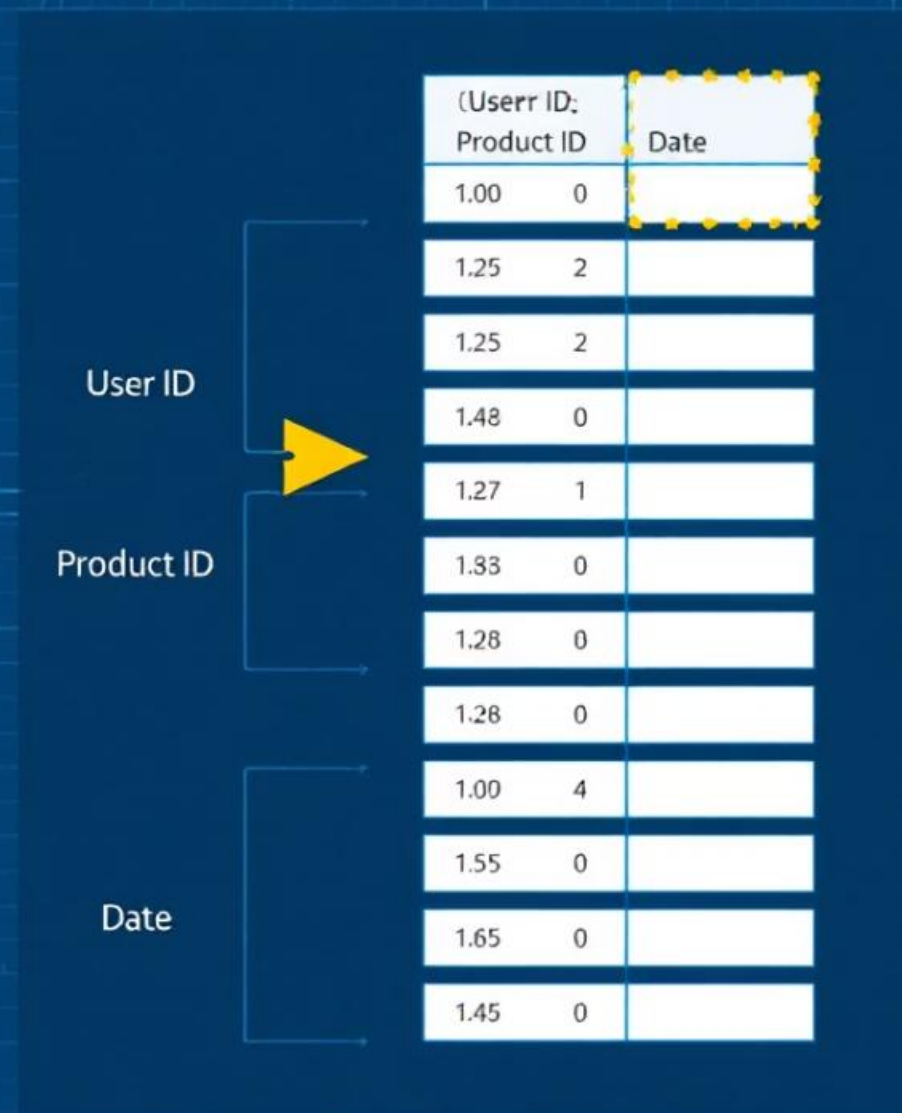
접근성

모든 사용자가 접근 가능해야 합니다.

Database Index Optimization

Throumy column order

Search by arattrons by the percomtions in search pattum index performances, and lulters if you cutum, index order in index column index, ant they ttable anels of ditution.



(User ID; Product ID	Date
1.00 0	
1.25 2	
1.25 2	
1.48 0	
1.27 1	
1.33 0	
1.28 0	
1.28 0	
1.00 4	
1.55 0	
1.65 0	
1.45 0	

기본 키 순서 결정 방법



검색 빈도 우선

자주 검색되는 컬럼을 우선적으로 배치합니다.



분포도 고려

데이터 분포가 균일한 컬럼을 앞쪽에 배치합니다.



외래 키 순서 유지

부모 테이블에서 상속받은 FK는 원래 PK 순서를 유지합니다.



가변길이 타입 지양

고정 길이 데이터 타입을 사용하는 것이 좋습니다.

기본 키 설계 실전 예시

schedule_t

1. 검색 내용 : K-리그 삼성PAVV 경기일정을 경기장, 경기일자순으로 정렬하여 출력
2. WHERE 조건 : 삼성PAVV배 → gubun = 'S'
3. 조회 칼럼 : gubun, stadium_id, sche_date
4. 참고사항 : gubun, stadium_id, sche_date 순으로 자료정렬

위의 조건으로 검색을 자주 할 때 PK 컬럼의 순서는 gubun + stadium_id + sche_date 순으로 구성



요구사항 분석

K-리그 삼성PAVV 경기일정을 경기장, 경기일자순으로 정렬



조건 확인

삼성PAVV배 → gubun = 'S'



최적 PK 설계

gubun + stadium_id + sche_date 순으로 구성

이 예시는 검색 패턴에 최적화된 PK 설계의 전형적인 사례입니다.

물리적 데이터베이스 구축 과정

논리 모델 전환

ERD를 테이블간 관계도로 변환합니다.

물리적 구현

분산 데이터베이스 설계를 고려한 실제 구현을 진행합니다.



테이블 정의

엔티티를 테이블로, 속성을 컬럼으로 변환합니다.

식별자 변환

주식별자를 PK로, 관계를 외부키로 변환합니다.

테이블 정의서						
시스템명				작성일		
테이블 ID				작성자		
테이블명						
번호	컬럼 ID	컬럼명	형태	길이	NULL	비고
인덱스						
업무규칙						

테이블 정의서 작성 방법

기본 정보 영역

테이블명, 설명, 소유자 등 테이블의 기본 정보를 기록합니다.

컬럼 정의 영역

각 컬럼의 이름, 데이터 타입, 길이, NULL 허용 여부를 정의합니다.

제약조건 영역

PK, FK 등의 제약조건과 인덱스 정보를 명시합니다.

데이터베이스 무결성 규칙 적용

도메인 무결성

컬럼 값이 정의된 도메인 내에 존재해야 함

참조 무결성

자식 엔티티 입력 및 부모 엔티티 삭제
·수정 규칙 적용

엔티티 무결성

모든 테이블은 고유한 기본 키를 가져야 함

무결성 규칙은 데이터베이스의 정확성과 일관성을 보장하는 핵심 요소입니다. 설계 단계에서 철저히 적용해야 합니다.



데이터베이스 설계 정리

1

데이터베이스 설계는 사용자의 요구사항을 수집하고 분석하여 데이터베이스를 구축하는 과정입니다.

2

설계 단계에서는 개념적 설계, 논리적 설계, 물리적 설계를 통해 데이터베이스 구조를 정의합니다.

3

구현 단계에서는 DBMS를 사용하여 데이터베이스를 생성하고 테이블, 뷰, 인덱스 등의 데이터베이스 객체를 생성합니다.

4

테스트 및 배포, 유지보수 단계를 거쳐 데이터베이스를 완성합니다.



Thank you